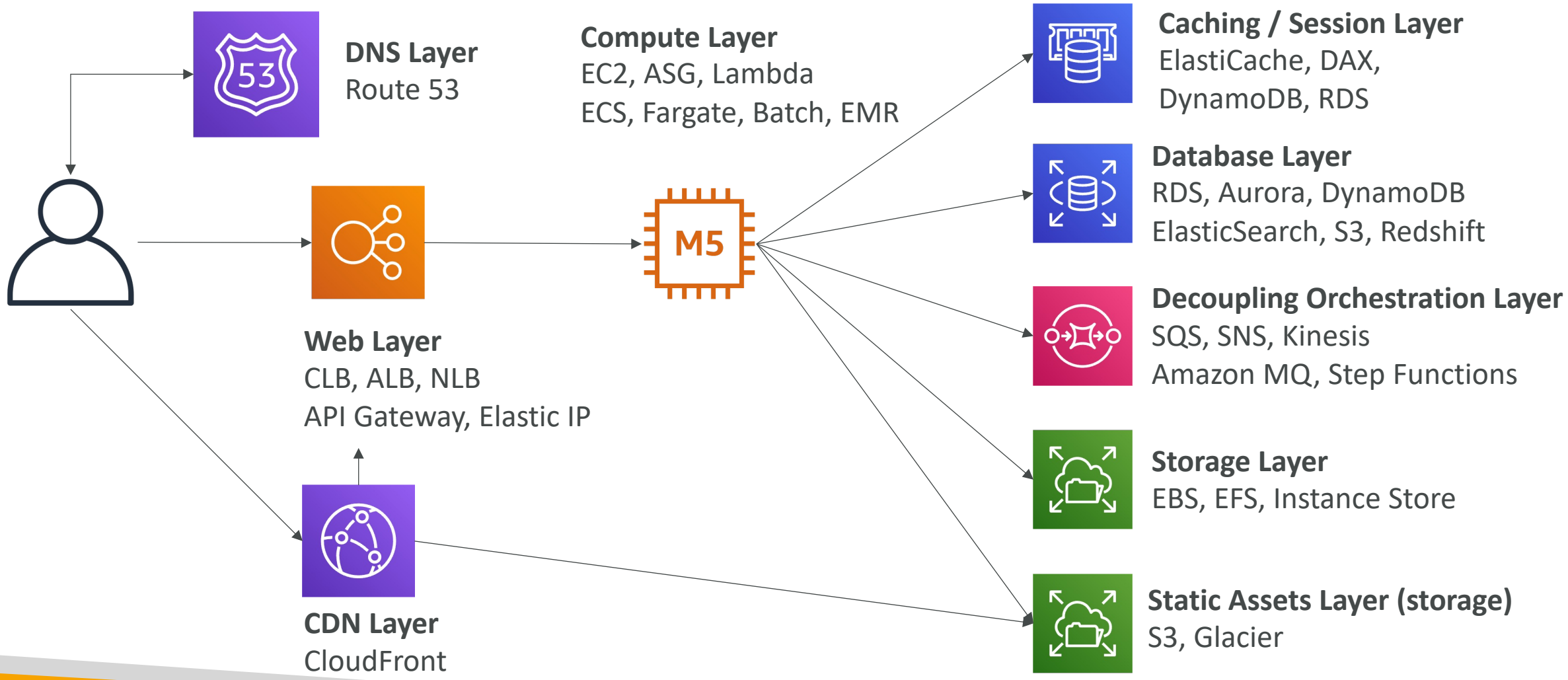
Compute & Load Balancing

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Solution Architecture on AWS



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

EC2 Instance Types – Main ones

- R: applications that needs a lot of RAM – in-memory caches
- C: applications that needs good CPU – compute / databases
- M: applications that are balanced (think "medium") – general / web app
- I: applications that need good local I/O (instance storage) – databases
- G: applications that need a GPU – video rendering / machine learning

- T2 / T3: burstable instances (up to a capacity)
- T2 / T3 - unlimited: unlimited burst

- Real-world tip: use https://www.ec2instances.info

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

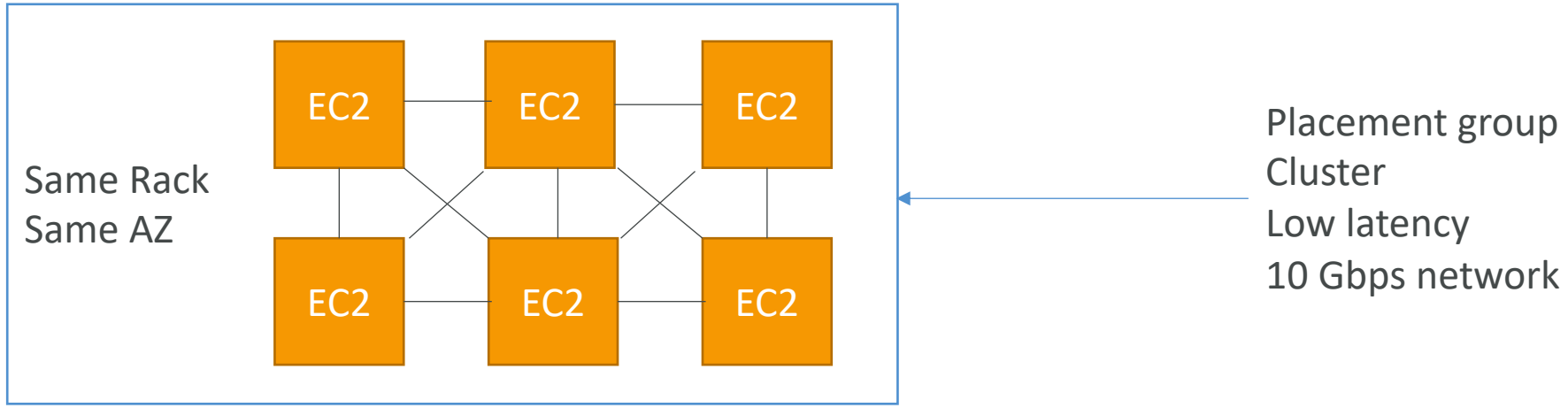
EC2 - Placement Groups

- Control the EC2 Instance placement strategy using placement groups
- Group Strategies:
  - *Cluster*—clusters instances into a low-latency group in a single Availability Zone
  - *Spread*—spreads instances across underlying hardware (max 7 instances per group per AZ) – critical applications
  - *Partition*—spreads instances across many different partitions (which rely on different sets of racks) within an AZ. Scales to 100s of EC2 instances per group (Hadoop, Cassandra, Kafka)
- **You can move** an instance into or out of a placement group
  - Your first need to **stop** it
  - You then need to use the CLI (modify-instance-placement)
  - You can then **start** your instance

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Placement Groups
Cluster

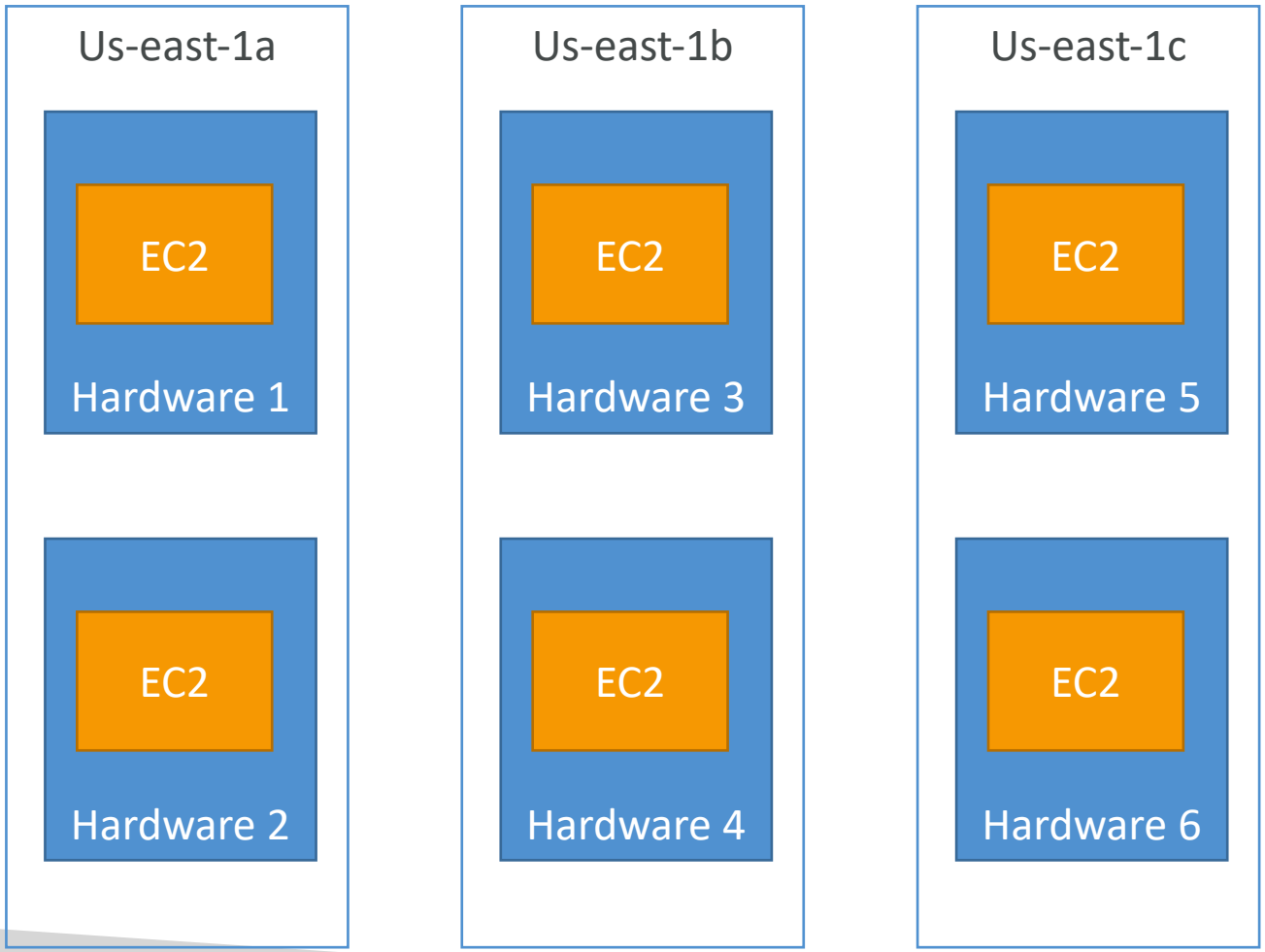


- Pros: Great network (10 Gbps bandwidth between instances with Enhanced Networking enabled - recommended)
- Cons: If the rack fails, all instances fails at the same time
- Use case:
  - Big Data job that needs to complete fast
  - Application that needs extremely low latency and high network throughput

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Placement Groups Spread

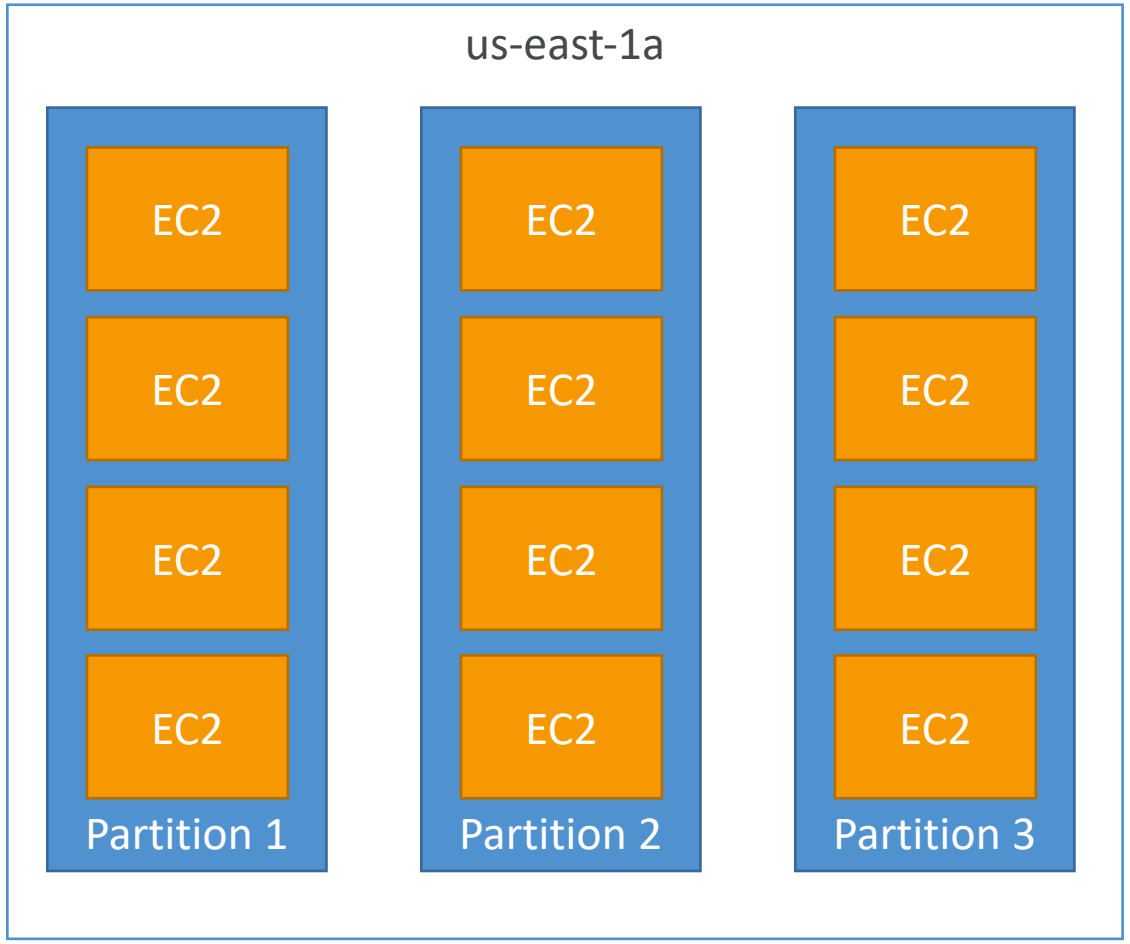


- Pros:
  - Can span across Availability Zones (AZ)
  - Reduced risk is simultaneous failure
  - EC2 Instances are on different physical hardware
- Cons:
  - Limited to 7 instances per AZ per placement group
- Use case:
  - Application that needs to maximize high availability
  - Critical Applications where each instance must be isolated from failure from each other

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Placements Groups Partition



- Up to 7 partitions per AZ
- Up to 100s of EC2 instances
- The instances in a partition do not share racks with the instances in the other partitions
- A partition failure can affect many EC2 but won't affect other partitions
- EC2 instances get access to the partition information as metadata
- Use cases: HDFS, HBase, Cassandra, Kafka

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

EC2 Instance Launch Types

- **On Demand Instances:** short workload, predictable pricing, reliable
- **Spot Instances:** short workloads, for cheap, can lose instances (not reliable)
- **Reserved:** (MINIMUM 1 year)
  - **Reserved Instances:** long workloads
  - **Convertible Reserved Instances:** long workloads with flexible instances
  - **Highest to lowest discount:** All Upfront payment, Partial Upfront payment, no Upfront
- **Dedicated Instances:** no other customers will share your hardware
- **Dedicated Hosts:** book an entire physical server, control instance placement
  - Great for software licenses that operate at the core, or CPU socket level
  - Can define **host affinity** so that instance reboots are kept on the same host

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

EC2 Graviton



- AWS Graviton Processors deliver the best price performance
- Supports many Linux OS, Amazon Linux 2, RedHat, SUSE, Ubuntu
- Not available for Windows instances

- **Graviton2** – 40% better price performance over comparable 5th generation x86-based instances
- **Graviton3** – Up to 3x better performance compared to Graviton2
- Use cases: app servers, microservices, HPC, CPU-based ML, video encoding, gaming, in-memory caches, …

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

EC2 included metrics

- CPU: CPU Utilization + Credit Usage / Balance
- Network: Network In / Out
- Status Check:
  - Instance status = check the EC2 VM
  - System status = check the underlying hardware
- Disk: Read / Write for Ops / Bytes (only for instance store)

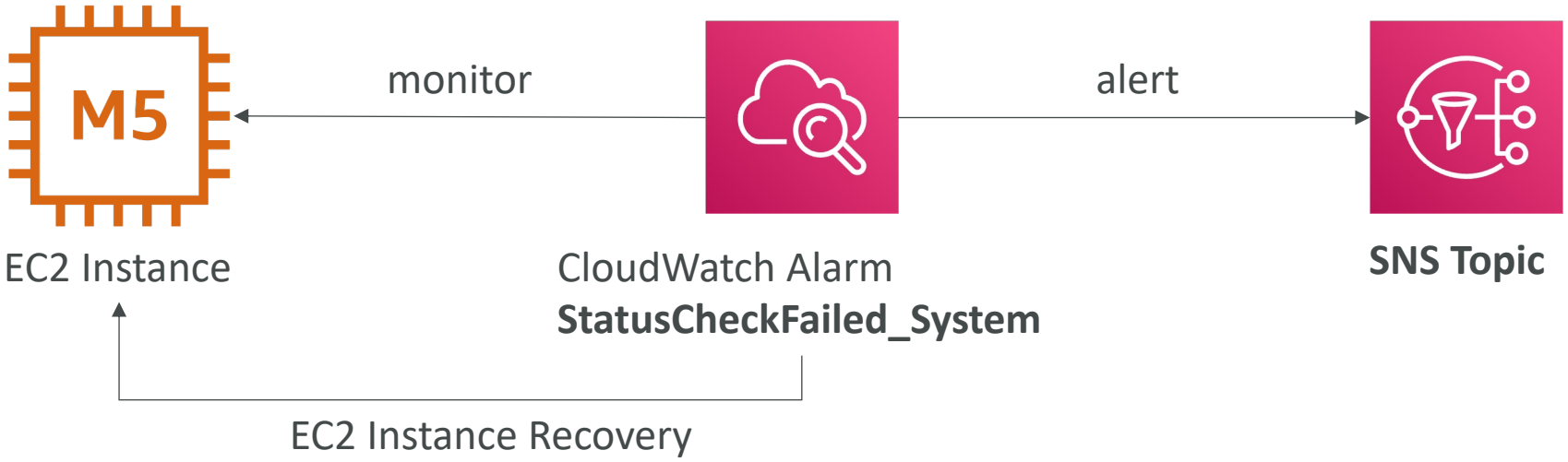
- RAM is NOT included in the AWS EC2 metrics

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

EC2 Instance Recovery

- Status Check:
  - Instance status = check the EC2 VM
  - System status = check the underlying hardware



- **Recovery:** Same Private, Public, Elastic IP, metadata, placement group

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

High Performance Computing (HPC)

- The cloud is the perfect place to perform HPC
- You can create a very high number of resources in no time
- You can speed up time to results by adding more resources
- You can pay only for the systems you have used

- Perform genomics, computational chemistry, financial risk modeling, weather prediction, machine learning, deep learning, autonomous driving

- Which services help perform HPC?

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Data Management & Transfer

- AWS Direct Connect:
  - Move GB/s of data to the cloud, over a private secure network
- Snowball
  - Move PB of data to the cloud
- AWS DataSync
  - Move large amount of data between on-premise and S3, EFS, FSx for Windows

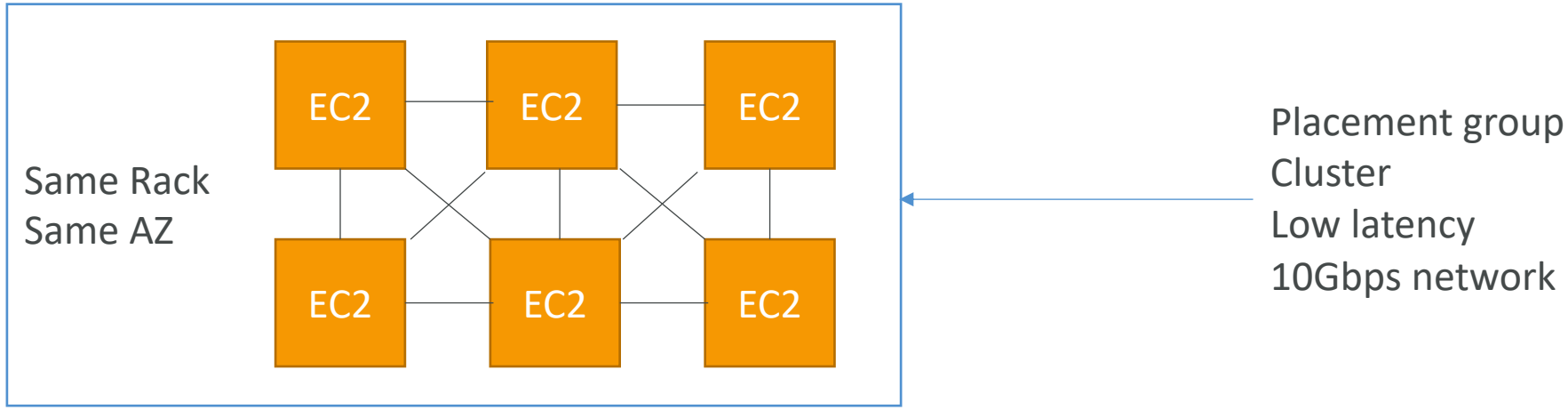
© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Compute and Networking

- EC2 Instances:
  - CPU optimized, GPU optimized
  - Spot Instances / Spot Fleets for cost savings + Auto Scaling

- EC2 Placement Groups: **Cluster** for good network performance



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Compute and Networking

- EC2 Enhanced Networking (SR-IOV)
  - Higher bandwidth, higher PPS (packet per second), lower latency
  - Option 1: **Elastic Network Adapter (ENA)** up to 100 Gbps
  - Option 2: Intel 82599 VF up to 10 Gbps – LEGACY

- **Elastic Fabric Adapter (EFA)**
  - Improved ENA for **HPC**, only works for **Linux**
  - Great for inter-node communications, **tightly coupled workloads**
  - Leverages Message Passing Interface (MPI) standard
  - Bypasses the underlying Linux OS to provide low-latency, reliable transport

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Storage

- Instance-attached storage:
  - **EBS:** scale up to 256,000 IOPS with io2 Block Express
  - **Instance Store:** scale to millions of IOPS, linked to EC2 instance, low latency

- Network storage:
  - **Amazon S3:** large blob, not a file system
  - **Amazon EFS:** scale IOPS based on total size, or use provisioned IOPS
  - **Amazon FSx for Lustre:**
    - HPC optimized distributed file system, millions of IOPS
    - Backed by S3

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Automation and Orchestration

- AWS Batch
  - **AWS Batch** supports multi-node parallel jobs, which enables you to run single jobs that span multiple **EC2** instances.
  - Easily schedule jobs and launch EC2 instances accordingly

- AWS ParallelCluster
  - Open source cluster management tool to deploy HPC on AWS
  - Configure with text files
  - Automate creation of VPC, Subnet, cluster type and instance types

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Auto Scaling Groups – Dynamic Scaling Policies

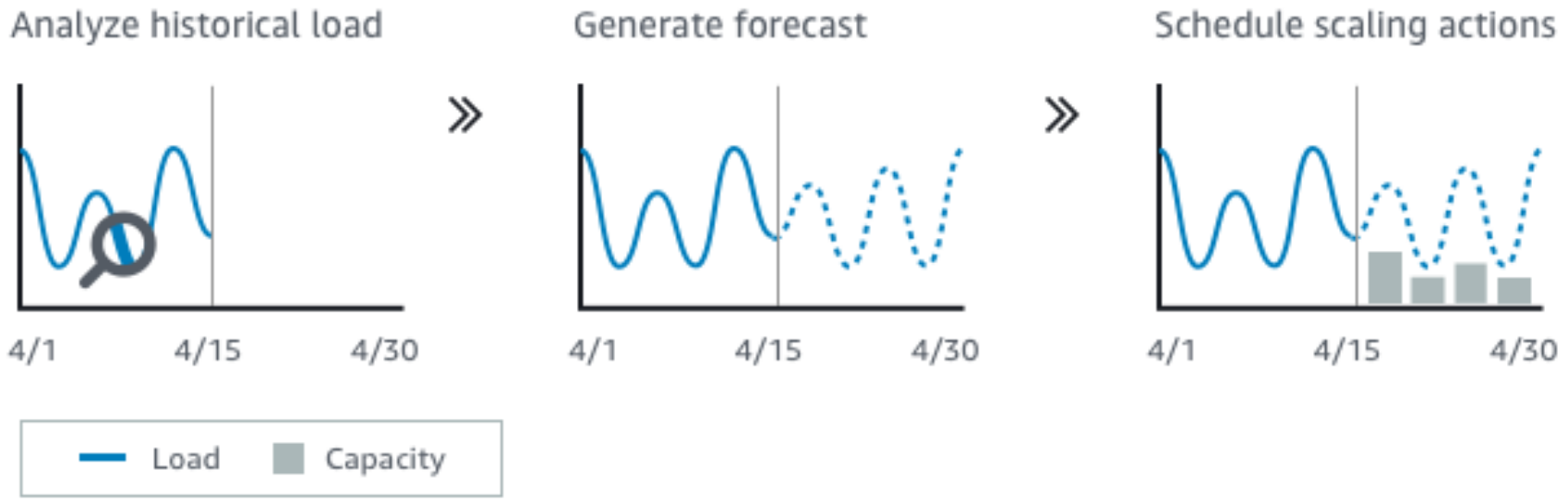
- Target Tracking Scaling
  - Most simple and easy to set-up
  - Example: I want the average ASG CPU to stay at around 40%
- Simple / Step Scaling
  - When a CloudWatch alarm is triggered (example CPU > 70%), then add 2 units
  - When a CloudWatch alarm is triggered (example CPU < 30%), then remove 1
- Scheduled Actions
  - Anticipate a scaling based on known usage patterns
  - Example: increase the min capacity to 10 at 5 pm on Fridays

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Auto Scaling Groups – Predictive Scaling

- **Predictive scaling:** continuously forecast load and schedule scaling ahead

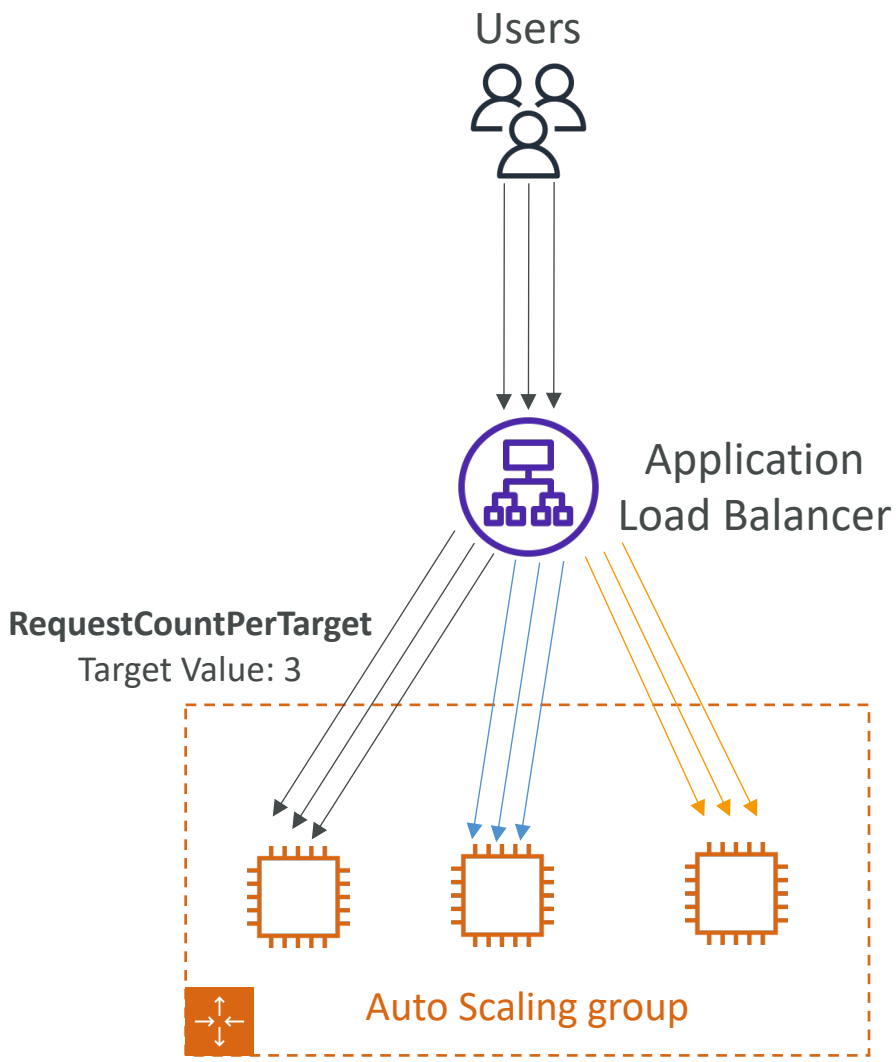


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Good metrics to scale on

- **CPUUtilization**: Average CPU utilization across your instances
- **RequestCountPerTarget**: to make sure the number of requests per EC2 instances is stable
- **Average Network In / Out** (if you're application is network bound)
- **Any custom metric** (that you push using CloudWatch)



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Auto Scaling – Good to know

- **Spot Fleet support** (mix on Spot and On-Demand instances)

- **Lifecycle Hooks**:
  - Perform actions before an instance is in service, or before it is terminated
  - Examples: cleanup, log extraction, special health checks

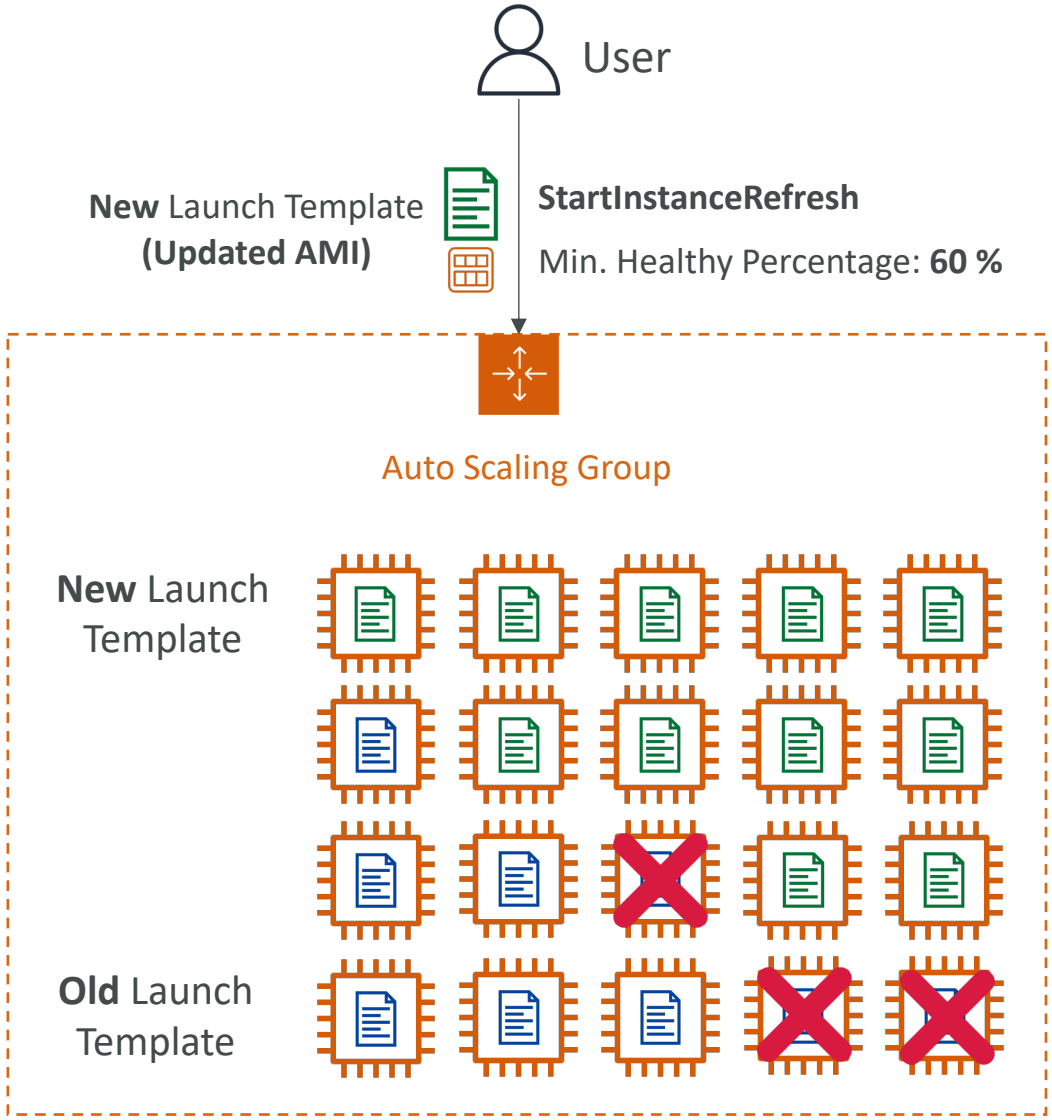
- To upgrade an AMI, must update the **launch configuration / template**
  - Then terminate instances manually (CloudFormation can help)
  - Or use EC2 Instance Refresh for Auto Scaling

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Auto Scaling – Instance Refresh

- Goal: update launch template and then re-creating all EC2 instances
- For this we can use the native feature of Instance Refresh
- Setting of minimum healthy percentage
- Specify warm-up time (how long until the instance is ready to use)



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Auto Scaling – Scaling Processes

- **Launch:** Add a new EC2 to the group, increasing the capacity
- **Terminate:** Removes an EC2 instance from the group, decreasing its capacity.
- **HealthCheck:** Checks the health of the instances
- **ReplaceUnhealthy:** Terminate unhealthy instances and re-create them
- **AZRebalance:** Balancer the number of EC2 instances across AZ
- **AlarmNotification:** Accept notification from CloudWatch
- **ScheduledActions:** Performs scheduled actions that you create.
- **AddToLoadBalancer:** Adds instances to the load balancer or target group
- **InstanceRefresh**: Perform an instance refresh

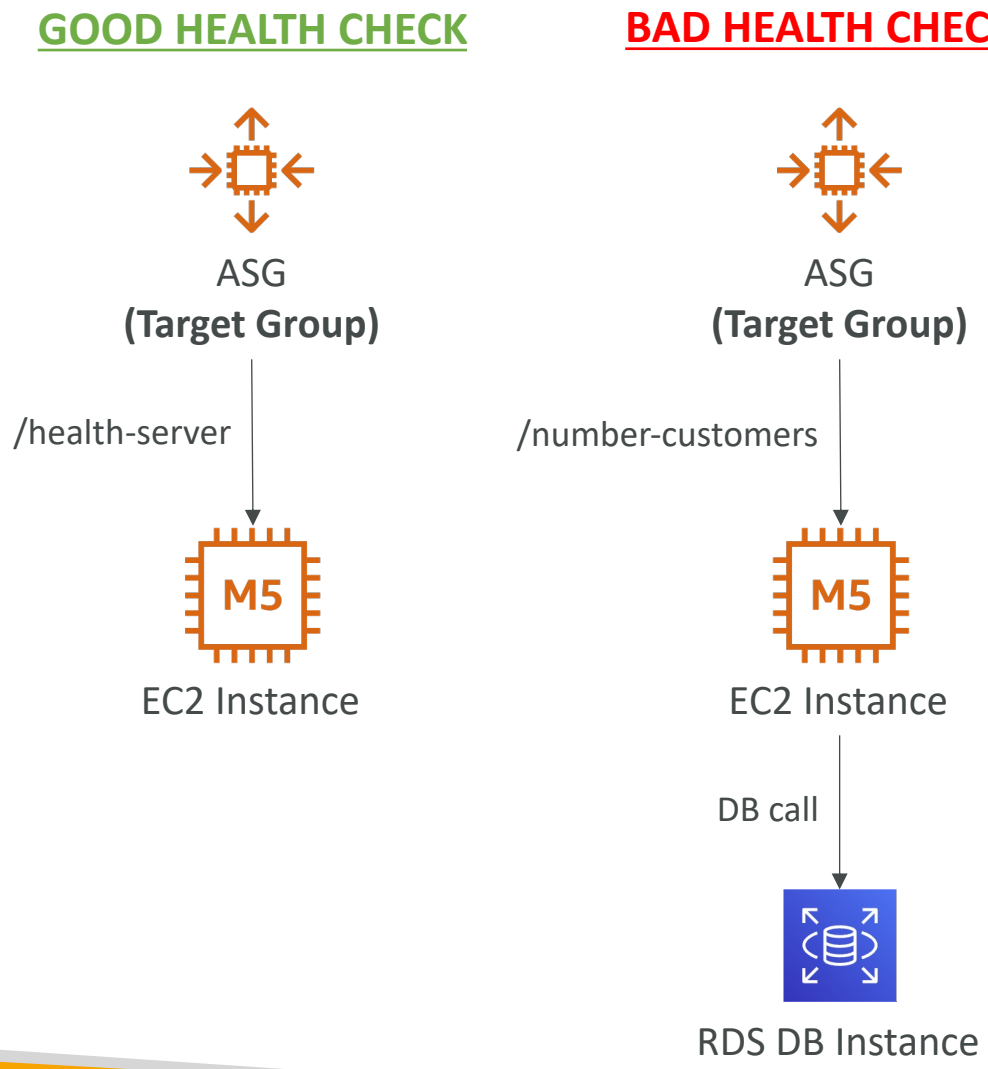
- We can suspend these processes!

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Auto Scaling – Health Checks

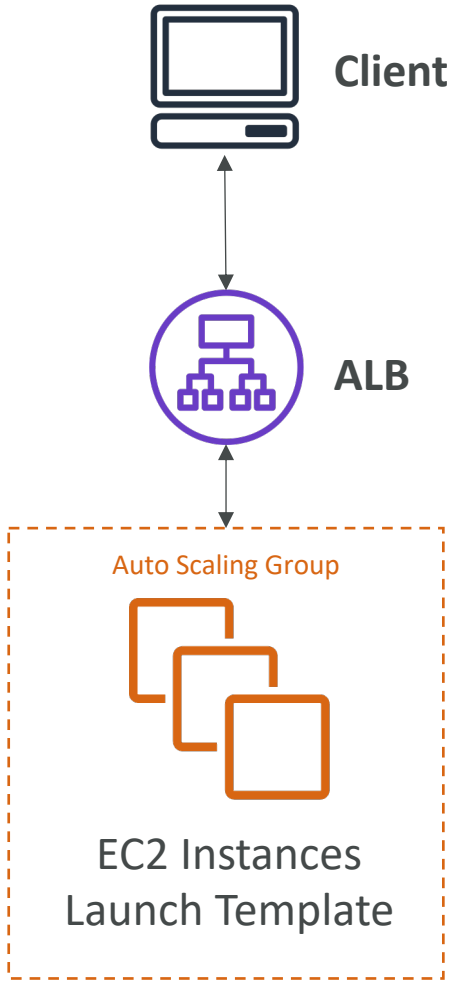
- Health checks available:
  - **EC2 Status Checks**
  - **ELB Health Checks (HTTP)**
  - **Custom Health Checks** – send instance's health to an ASG using AWS CLI or AWS SDK (**set-instance-health**)
- ASG will launch a new instance after terminating an unhealthy one
- Make sure the health check is simple and checks the correct thing



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

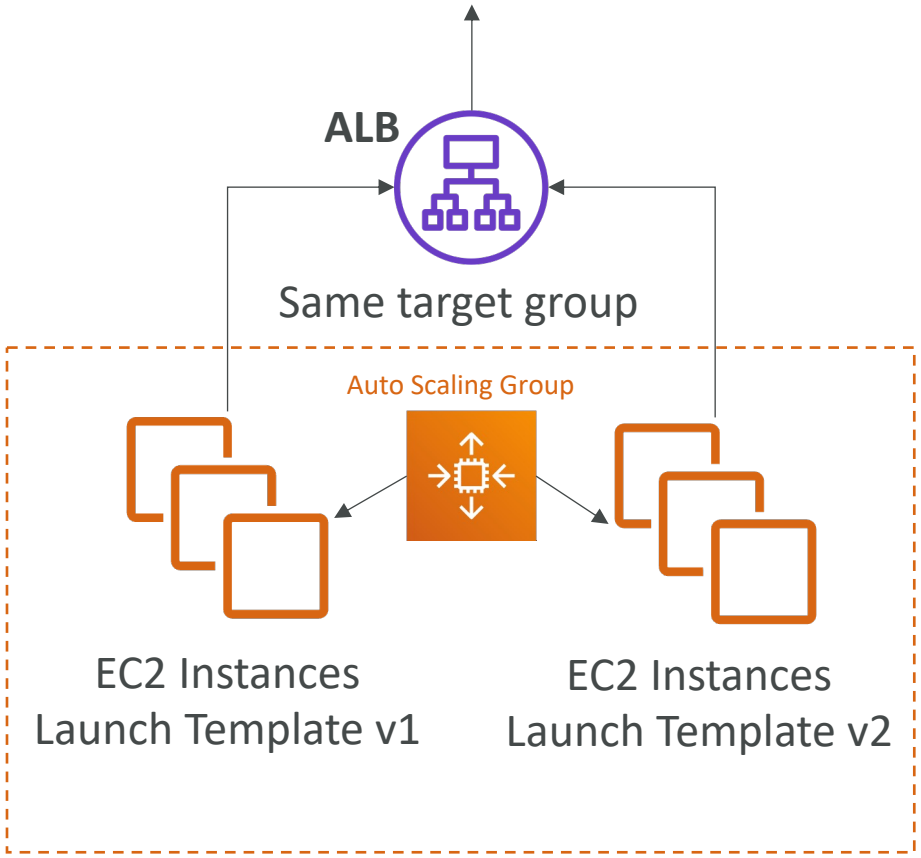
Auto Scaling – Updating an application

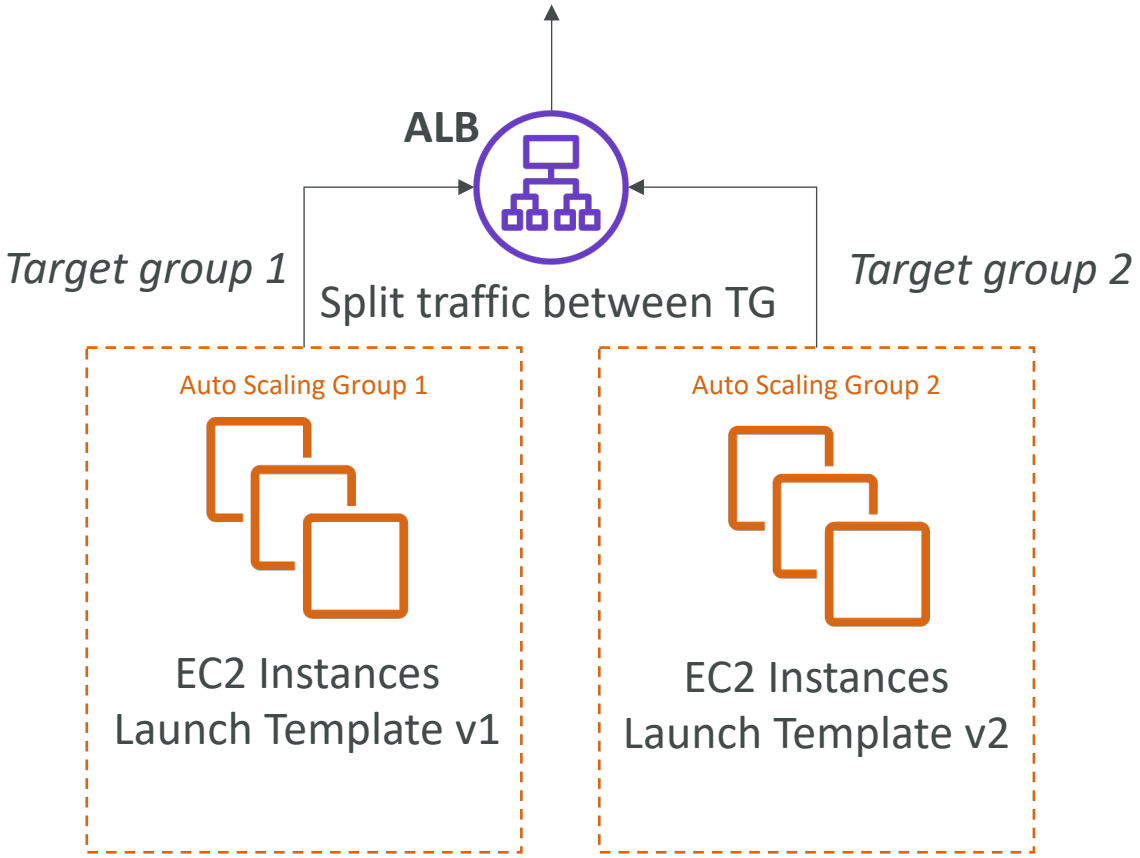


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Auto Scaling – Solution Architecture

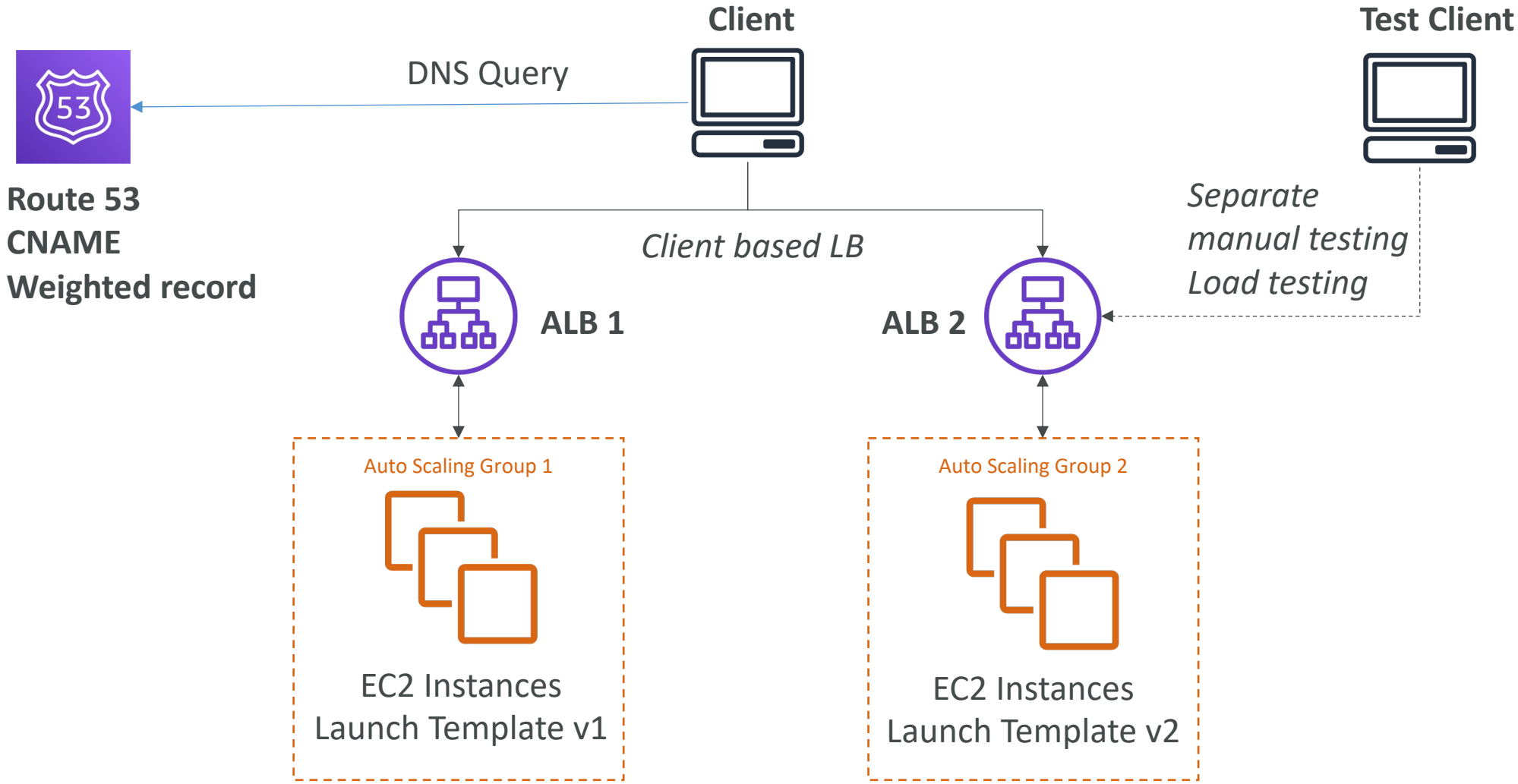




© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Auto Scaling – Solution Architecture



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

EC2 Spot Instances



- Can get a discount of up to 90% compared to On-Demand
- Define **max spot price** and get the instance while **current spot price < max**
  - The hourly spot price varies based on offer and capacity
  - If the current spot price > your max price you can choose to **stop** or **terminate** your instance with a 2 minutes grace period.

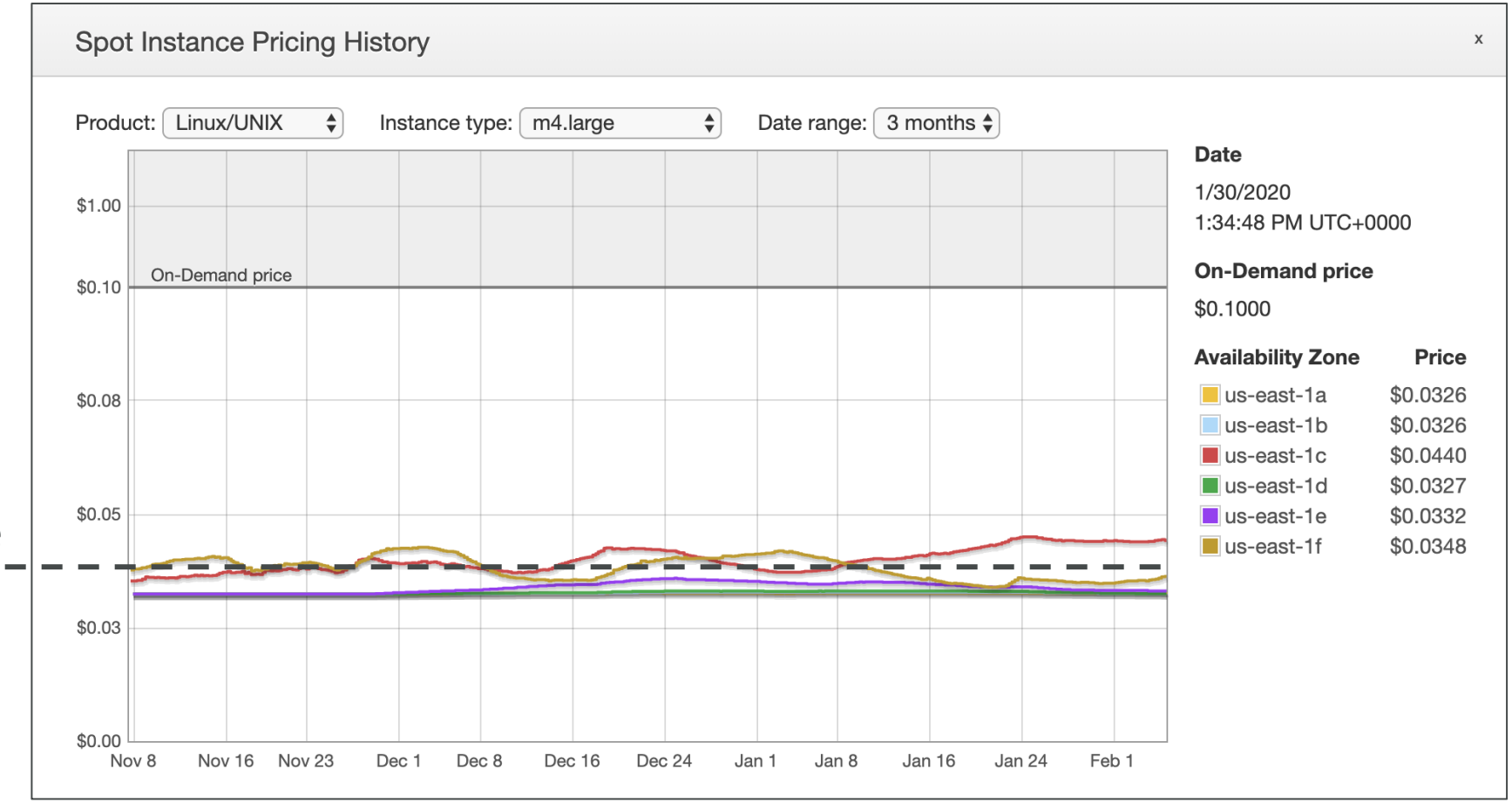
- Used for batch jobs, data analysis, or workloads that are resilient to failures.
- Not great for critical jobs or databases

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

EC2 Spot Instances

User-defined max price



https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#SpotInstances:

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

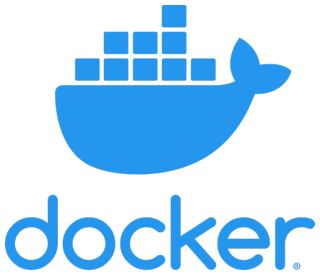
Spot Fleets

- Spot Fleets = set of Spot Instances + (optional) On-Demand Instances
- The Spot Fleet will try to meet the target capacity with price constraints
  - Define possible launch pools: instance type (m5.large), OS, Availability Zone
  - Can have multiple launch pools, so that the fleet can choose
  - Spot Fleet stops launching instances when reaching capacity or max cost
- Strategies to allocate Spot Instances:
  - **lowestPrice:** from the pool with the lowest price (cost optimization, short workload)
  - **diversified:** distributed across all pools (great for availability, long workloads)
  - **capacityOptimized:** pool with the optimal capacity for the number of instances
  - **priceCapacityOptimized (recommended):** pools with highest capacity available, then select the pool with the lowest price (best choice for most workloads)
- Spot Fleets allow us to automatically request Spot Instances with the lowest price

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

What is Docker?



- Docker is a software development platform to deploy apps
- Apps are packaged in **containers** that can be run on any OS
- Apps run the same, regardless of where they're run
  - Any machine (no compatibility issues, predictable behavior)
  - Less work
  - Easier to maintain and deploy
  - Works with any language, any OS, any technology
- Control how much memory / CPU is allocated to your container
- Scale containers up and down very quickly (seconds)
- More efficient than Virtual machines

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Docker Containers Management on AWS

- To manage containers, we need a container management platform

- **Amazon Elastic Container Service (Amazon ECS)**
  - Amazon's own container platform



Amazon ECS

- **Amazon Elastic Kubernetes Service (Amazon EKS)**
  - Amazon's managed Kubernetes (open source)



Amazon EKS

- **AWS Fargate**
  - Amazon's own Serverless container platform
  - Works with ECS and with EKS



AWS Fargate

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon ECS – Use cases

- Run Microservices
  - Run multiple Docker containers on the same machine
  - Easy Service Discovery features to enhance communication
  - Direct integration with Application Load Balancer and Network Load Balancer
  - Auto Scaling capability
- Run Batch Processing / Scheduled Tasks
  - Schedule ECS tasks to run on On-demand / Reserved / Spot instances
- Migrate Applications to the Cloud
  - Dockerize legacy applications running on-premises
  - Move Docker containers to run on Amazon ECS

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

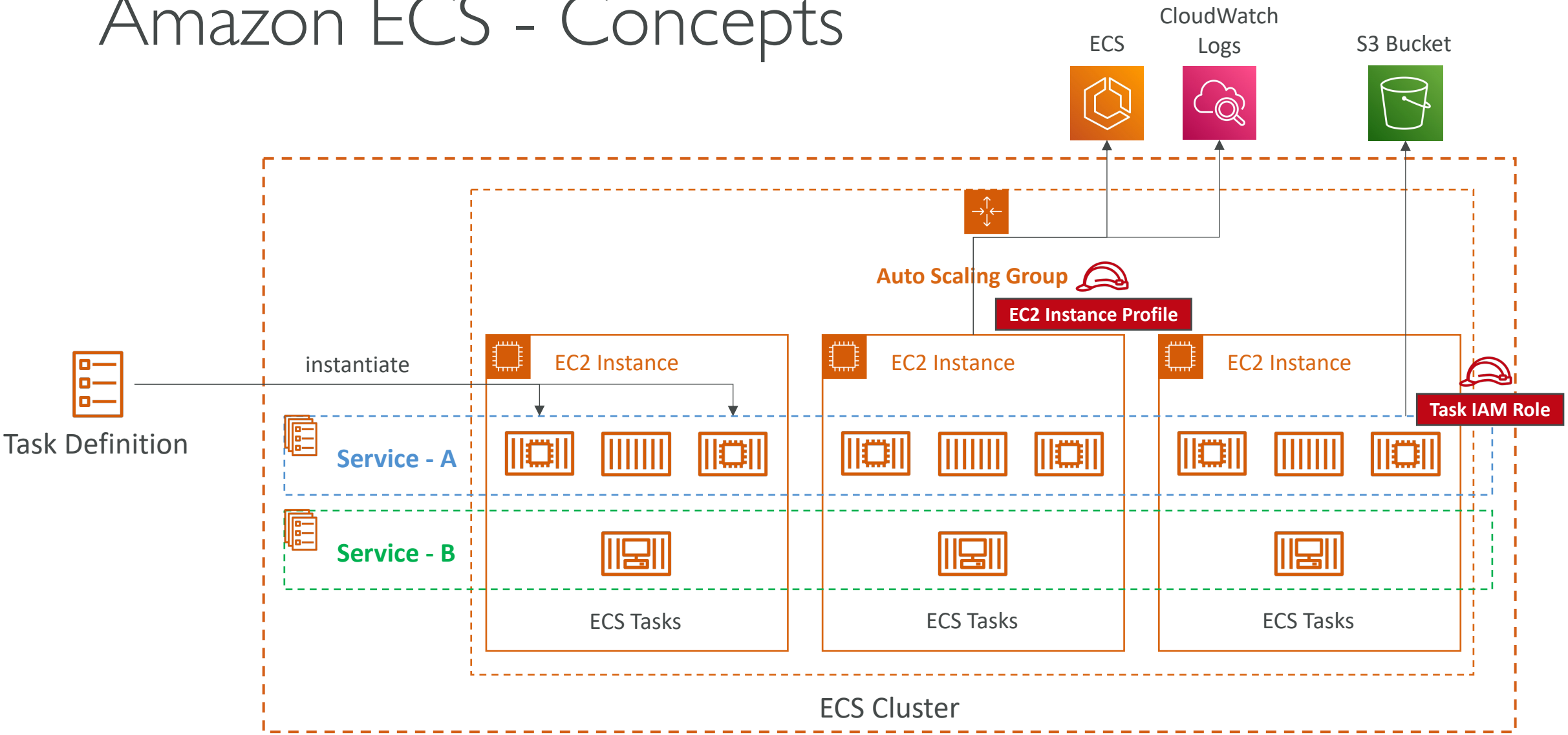
Amazon ECS – Concepts

- **ECS Cluster** – logical grouping of EC2 instances
- **ECS Service** – defines how many tasks should run and how they should be run
- **Task Definitions** – metadata in **JSON form** to tell ECS how to run a Docker container (image name, CPU, RAM, …)
- **ECS Task** – an instance of a Task Definition, a running Docker container(s)
- **ECS IAM Roles**
  - **EC2 Instance Profile** – used by the EC2 instance (e.g., make API calls to ECS, send logs, …)
  - **ECS Task IAM Role** – allow each task to have a specific role (e.g., make API calls to S3, DynamoDB, …)

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon ECS - Concepts

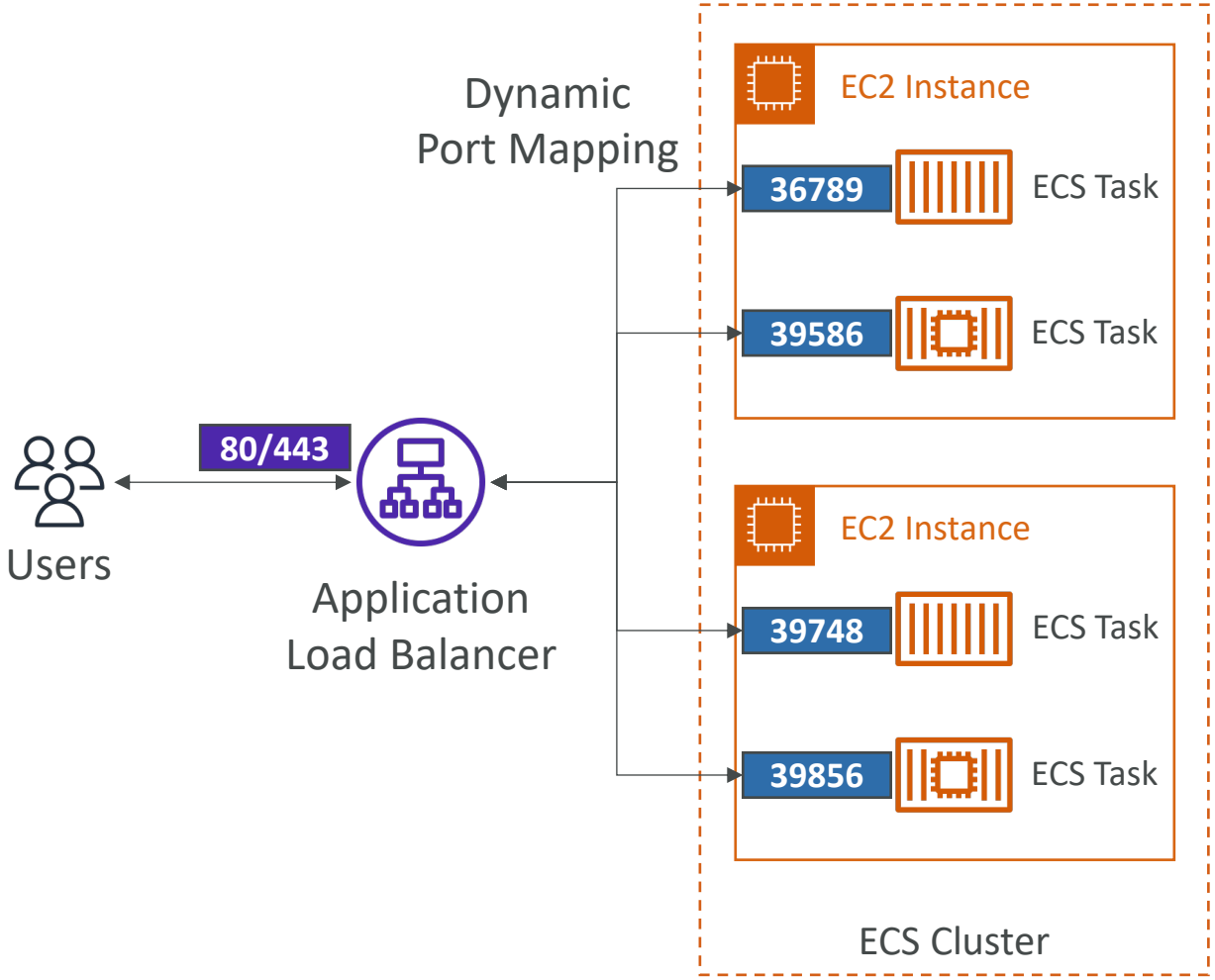


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon ECS – ALB Integration

- We get **Dynamic Port Mapping**
- Allows you to run multiple instances of the same application on the same EC2 instance
- The ALB finds the right port on your EC2 Instances
- Use cases:
  - Increased resiliency even if running on one EC2 instance
  - Maximize utilization of CPU / cores
  - Ability to perform rolling upgrades without impacting app uptime

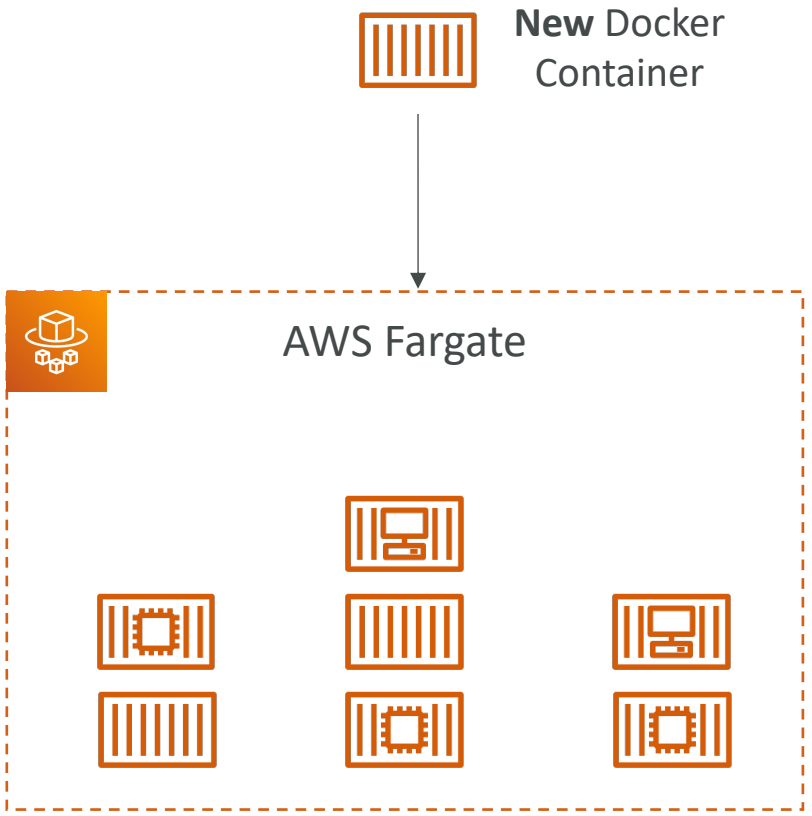


NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© Stephane Maarek

AWS Fargate

- Launch Docker containers on AWS
- **You do not provision the infrastructure (no EC2 instances to manage)**
- **It's all serverless!**
- You create task definitions
- AWS runs containers for you based on the CPU / RAM you need
- To scale, just increase the number of tasks. Simple! No more EC2 instances ☺



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© Stephane Maarek

Amazon ECS – Security & Networking

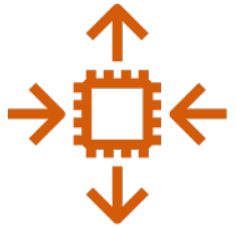
- You can inject secrets and configurations as Environment Variables into running Docker containers
  - Integration with **SSM Parameter Store** and **Secrets Manager**

- ECS Tasks Networking
  - **none** – no network connectivity, no port mappings
  - **bridge** – uses Docker's virtual container-based network
  - **host** – bypass Docker's network, uses the underlying host network interface
  - **awsvpc**
    - Every tasks launched on the instance gets its own ENI and a private IP address
    - Simplified networking, enhanced security, Security Groups, monitoring, VPC Flow Logs
    - Default mode for Fargate tasks

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon ECS – Service Auto Scaling



- Automatically increase/decrease the desired number of tasks
- Amazon ECS leverages **AWS Application Auto Scaling**
- CPU and RAM is tracked in CloudWatch at the ECS Service level

- **Target Tracking** – scale based on target value for a specific CloudWatch metric
- **Step Scaling** – scale based on a specified CloudWatch Alarm
- **Scheduled Scaling** – scale based on a specified date/time (predictable changes)

- ECS Service Auto Scaling (task level) ≠ EC2 Auto Scaling (EC2 instance level)
- Fargate Auto Scaling is much easier to setup (because **Serverless**)

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon ECS – Spot Instances



- ECS Classic (EC2 Launch Type)
  - Can have the underlying EC2 instances as Spot Instances (managed by an ASG)
  - Instances may go into draining mode to remove running tasks
  - Good for cost savings, but will impact reliability

- AWS Fargate
  - Specify minimum of tasks for on-demand baseline workload
  - Add tasks running on FARGATE_SPOT for cost-savings (can be reclaimed by AWS)
  - Regardless of On-demand or Spot, Fargate scales well based on load

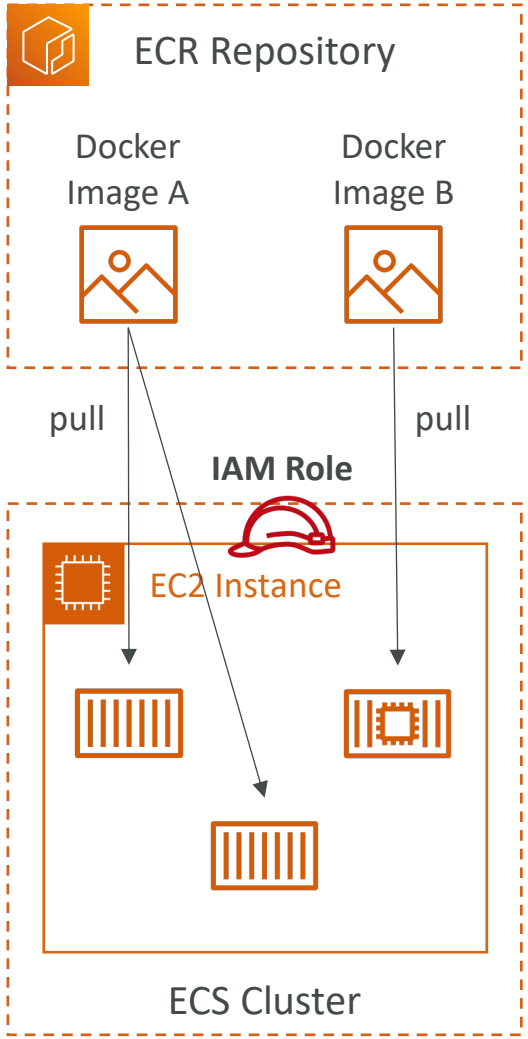
© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon ECR - Elastic Container Registry



- Store and manage Docker images on AWS
- **Private** and **Public** repository (**Amazon ECR Public Gallery** https://gallery.ecr.aws)
- Fully integrated with ECS
- Access is controlled through IAM (permission errors => check policy)
- Supports image vulnerability scanning, versioning, image tags, image lifecycle, …

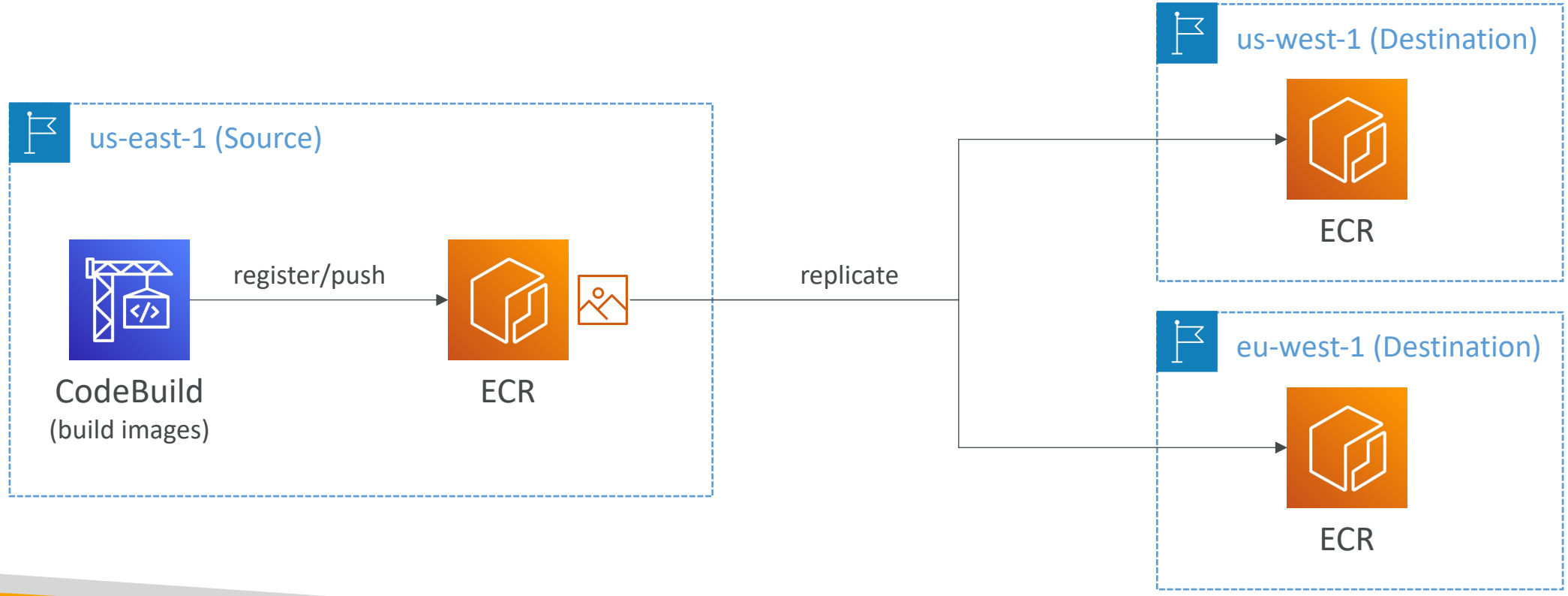


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon ECR – Cross Region Replication

- ECR private registry supports both **cross-Region** and **cross-account** replication

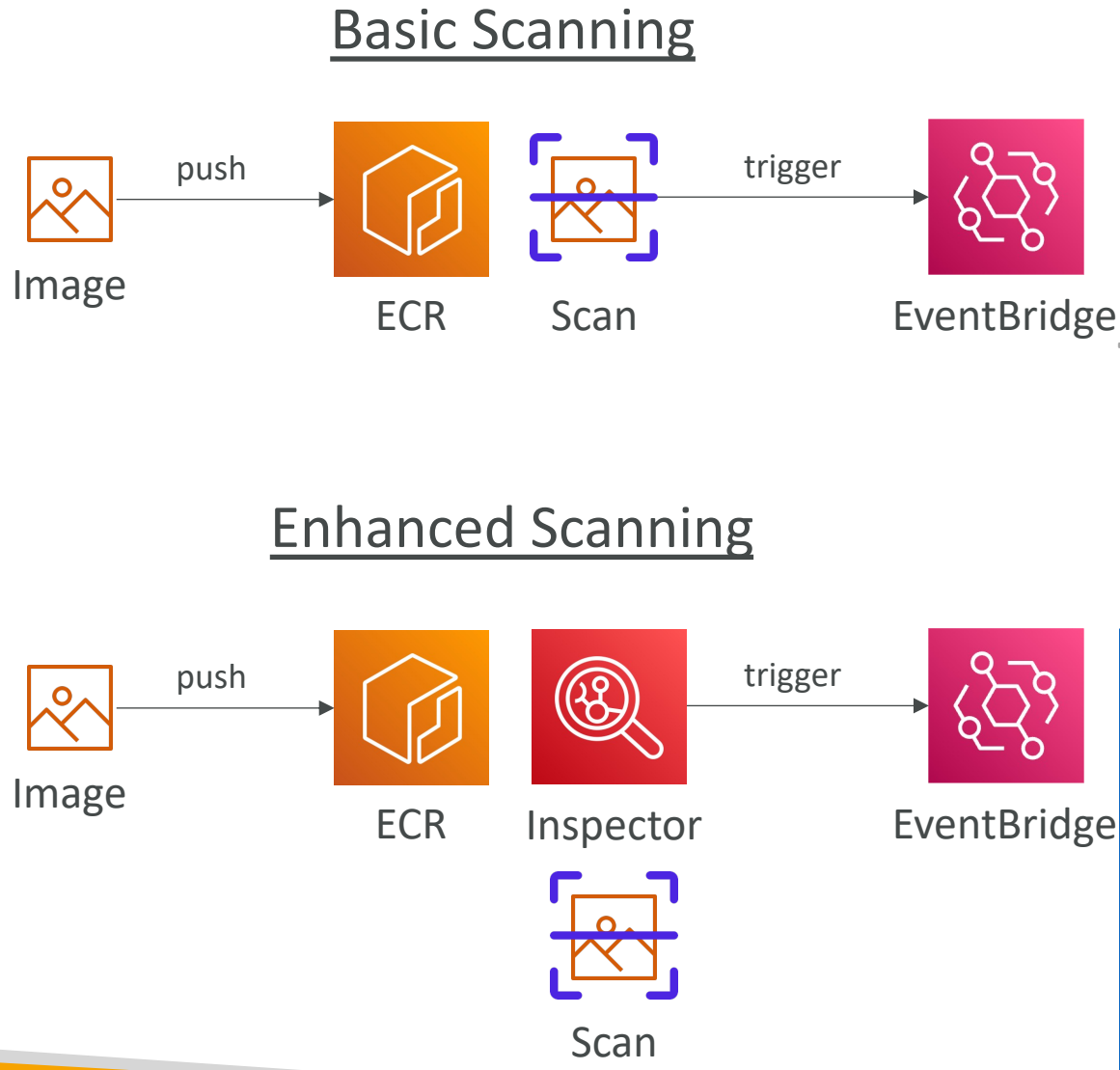


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon ECR – Image Scanning

- Manual Scan or Scan on Push
- Basic Scanning – Common CVE
- Enhanced Scanning – Leverages Amazon Inspector (OS & Programming Language vulnerabilities)
- Scan results can be retrieved from within the AWS console



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon EKS Overview



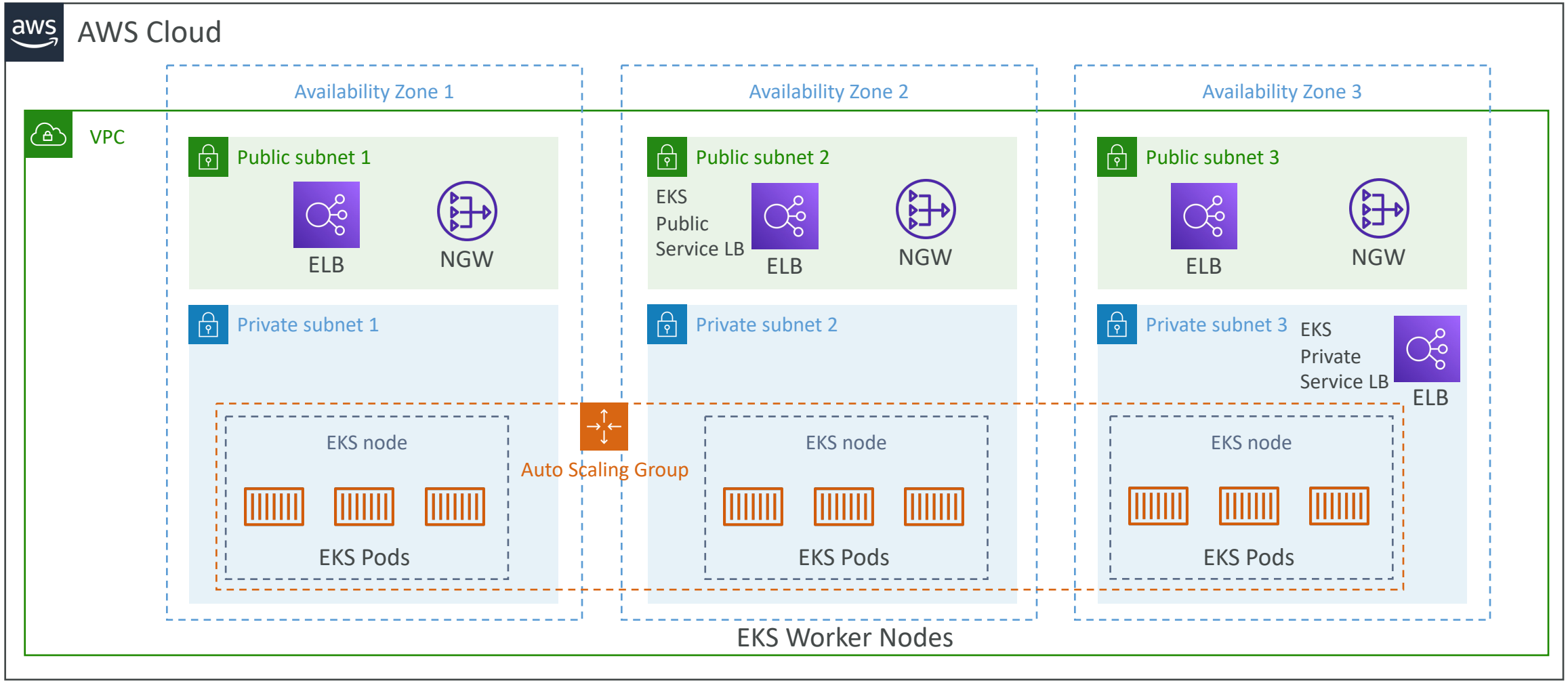


- Amazon EKS = Amazon Elastic **Kubernetes** Service
- It is a way to launch **managed Kubernetes clusters on AWS**
- Kubernetes is an **open-source system** for automatic deployment, scaling and management of containerized (usually Docker) application
- It's an alternative to ECS, similar goal but different API
- EKS supports **EC2** if you want to deploy worker nodes or **Fargate** to deploy serverless containers
- **Use case:** if your company is already using Kubernetes on-premises or in another cloud, and wants to migrate to AWS using Kubernetes
- **Kubernetes is cloud-agnostic** (can be used in any cloud – Azure, GCP…)
- For multiple regions, deploy one EKS cluster per region
- Collect logs and metrics using **CloudWatch Container Insights**

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon EKS - Diagram



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon EKS – Node Types

- Managed Node Groups
  - Creates and manages Nodes (EC2 instances) for you
  - Nodes are part of an ASG managed by EKS
  - Supports On-Demand or Spot Instances
- Self-Managed Nodes
  - Nodes created by you and registered to the EKS cluster and managed by an ASG
  - You can use prebuilt AMI - Amazon EKS Optimized AMI
  - Supports On-Demand or Spot Instances
- AWS Fargate
  - No maintenance required; no nodes managed

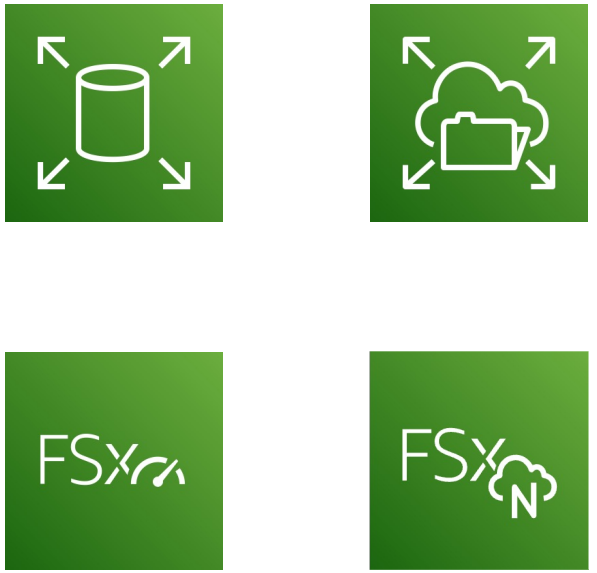
© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon EKS – Data Volumes

- Need to specify **StorageClass** manifest on your EKS cluster
- Leverages a **Container Storage Interface (CSI)** compliant driver

- Support for…
- Amazon EBS
- Amazon EFS (works with Fargate)
- Amazon FSx for Lustre
- Amazon FSx for NetApp ONTAP

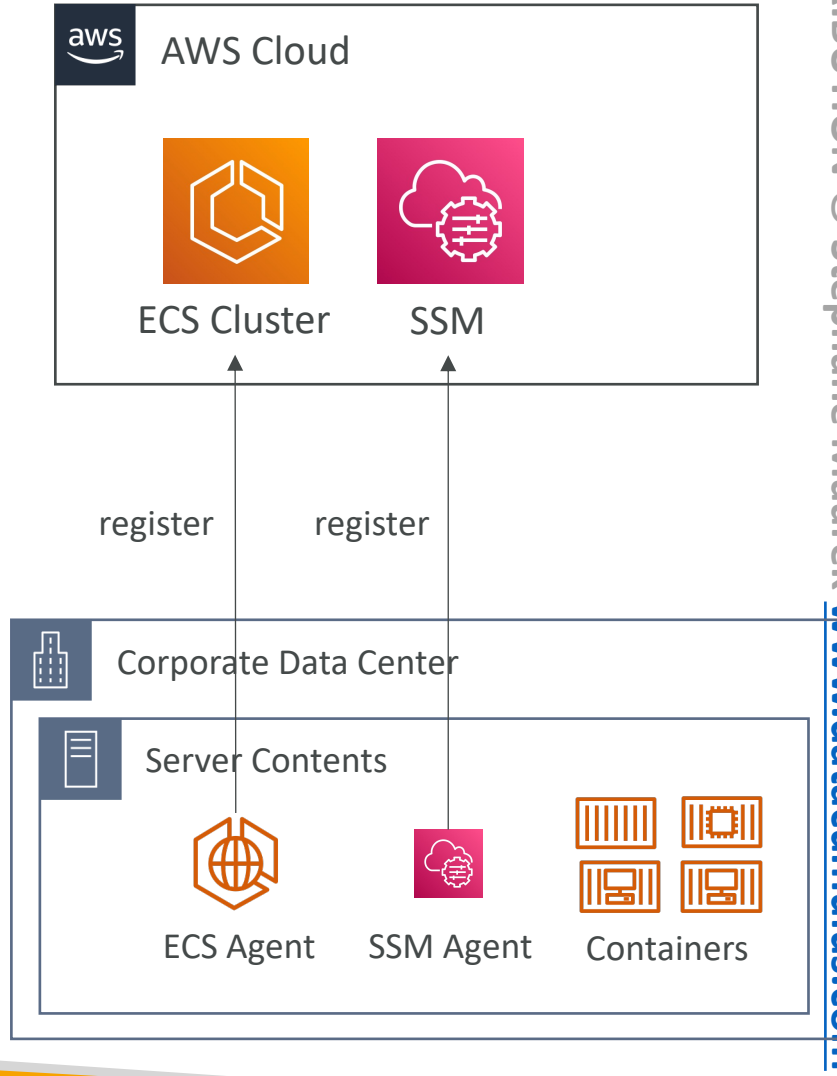


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Amazon ECS Anywhere

- Easily run containers on Customer-managed infrastructure (on-premises, VMs, …)
- Allows customers to deploy native Amazon ECS tasks in any environment
- Fully-managed Amazon ECS Control Plane
- **ECS Container Agent** and **SSM Agent** needs to be installed
- **"EXTERNAL"** Launch Type
- Must have a stable connection to the AWS Region
- Use cases:
  - Meet compliance, regulatory, and latency requirements
  - Run apps outside AWS Regions and closer to their other services
  - On-premises ML, video processing, data processing, …



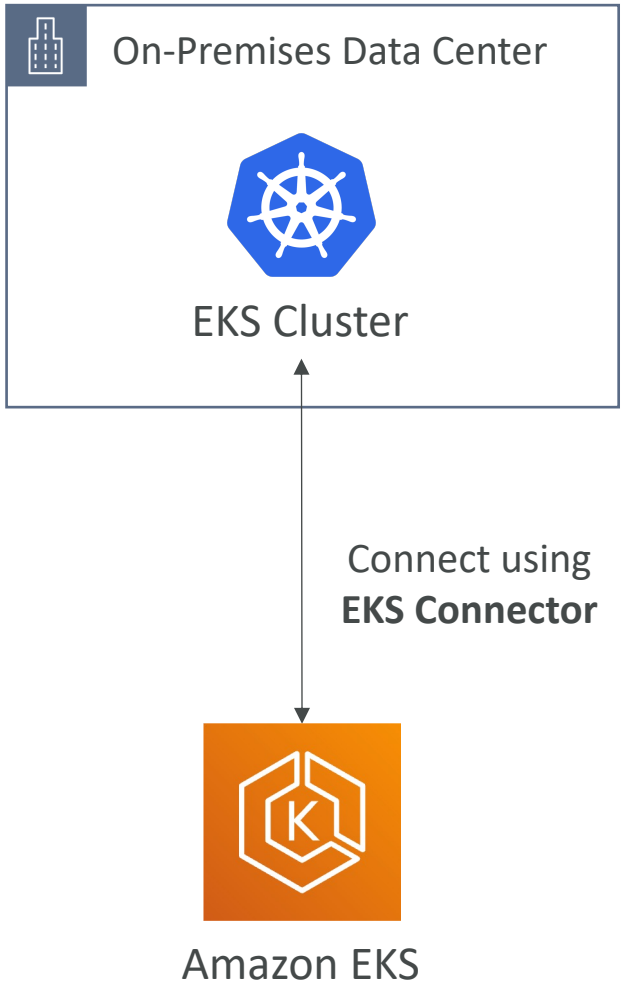
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© Stephane Maarek

Amazon EKS Anywhere

- Create and operate Kubernetes clusters created outside AWS
- Leverage the **Amazon EKS Distro** (AWS' bundled release of Kubernetes)
- Reduce support costs and avoid maintaing redundant 3rd party tools

- Install using the **EKS Anywhere Installer**
- Optionally use the **EKS Connector** to connect the EKS Anywhere clusters to AWS
  - Fully Connected & Partially Disconnected: you can connect to Amazon EKS Anywhere clusters to AWS, and leverage the EKS console
  - Fully Disconnected: must install the EKS Distro and leverage open-source tools to manage your clusters



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AWS Lambda Integrations
Main ones



API Gateway



Kinesis



DynamoDB



AWS S3 –
Simple Storage Service



AWS IoT
Internet of Things



Amazon EventBridge



CloudWatch Logs



AWS SNS



AWS Cognito



Amazon
SQS

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Example: Serverless Thumbnail creation



trigger



push



push





New image in S3



AWS Lambda Function
Creates a Thumbnail

New thumbnail in S3

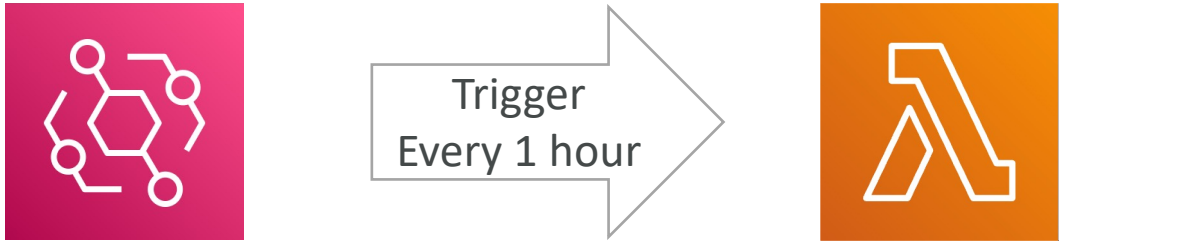
Image name
Image size
Creation date
etc…

Metadata in DynamoDB

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Example: Serverless CRON Job



Amazon EventBridge

AWS Lambda Function
Perform a task

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AWS Lambda Language Support (runtimes)

- Node.js (JavaScript)
- Python
- Java
- C# (.NET Core) / Powershell
- Ruby
- Custom Runtime API (community supported, example Rust or Golang)

- Lambda Container Image
  - The container image must implement the Lambda Runtime API
  - ECS / Fargate is preferred for running arbitrary Docker images

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Lambda – Limits to know

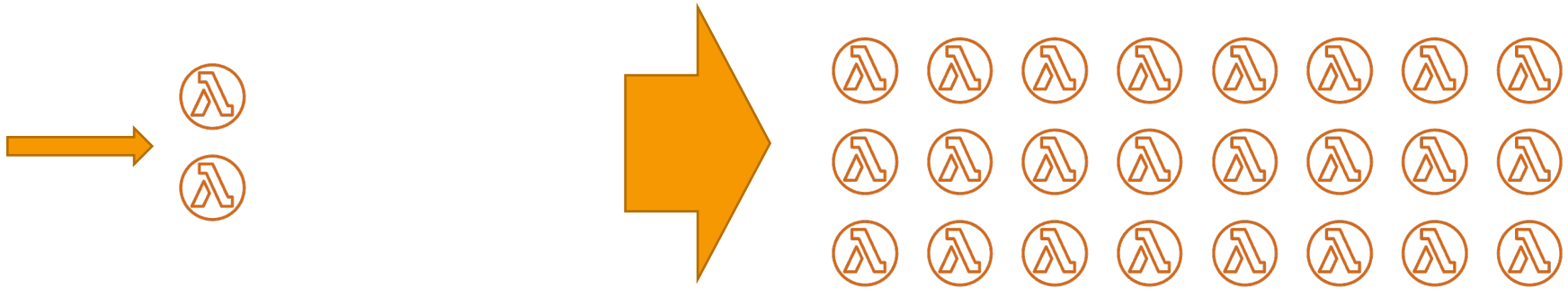
- **RAM** – 128 MB to 10,240 MB (10 GB)
- **CPU** – is linked to RAM (cannot be set manually)
  - 2 vCPUs are allocated at 1,769 MB of RAM
  - 6 vCPUs are allocated at 10,240 MB of RAM
- **Timeout** – up to 15 minutes
- **/tmp Storage** – 10,240 MB
- **Deployment Package** – 50 MB (zipped) , 250 MB (unzipped) including layers
- **Concurrent Executions** – 1000 (soft limit that can be increased)
- **Container Image Size** – 10 GB
- **Invocation Payload (request/response)** – 6 MB (sync), 256 KB (async)

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Lambda Concurrency and Throttling

- Concurrency limit: up to 1000 concurrent executions



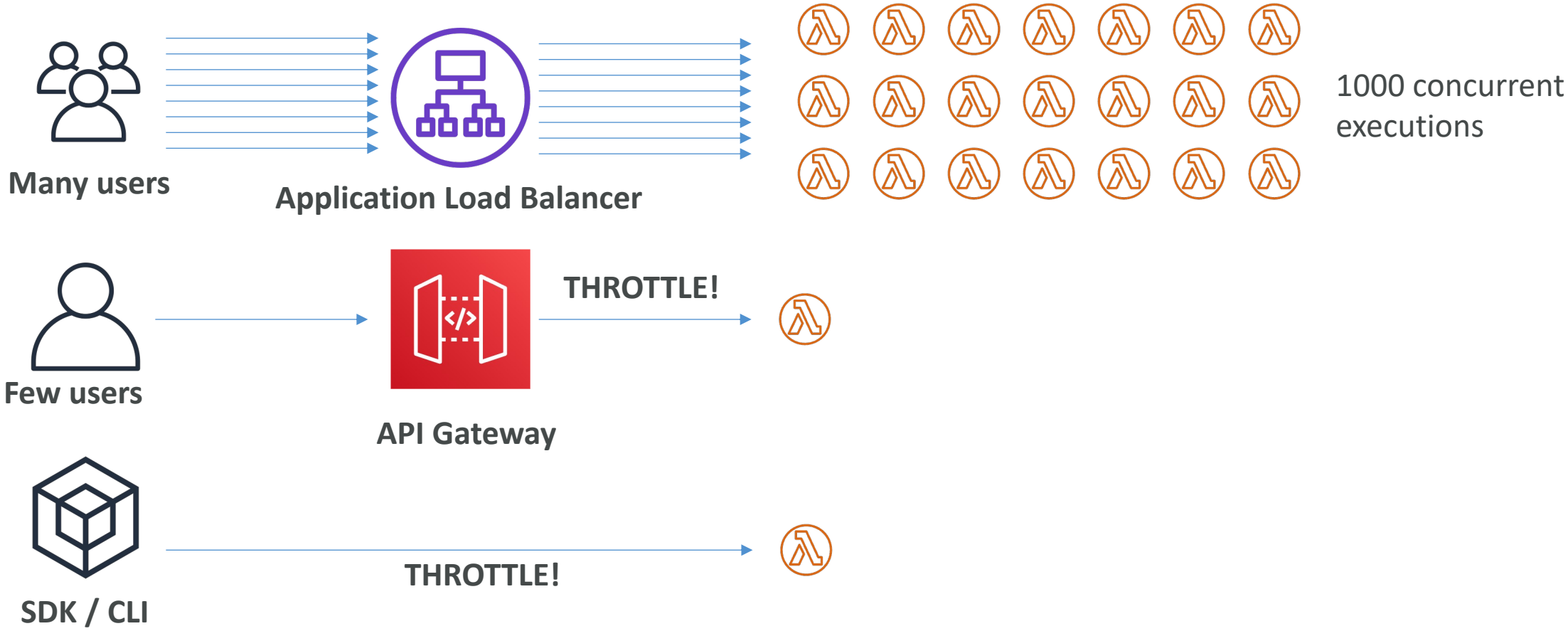
- Can set a "**reserved concurrency**" at the function level (=limit)
- Each invocation over the concurrency limit will trigger a "Throttle"
- Can request a quota increase in AWS Service Quotas

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Lambda Concurrency Issue

- If you don't reserve (=limit) concurrency, the following can happen:

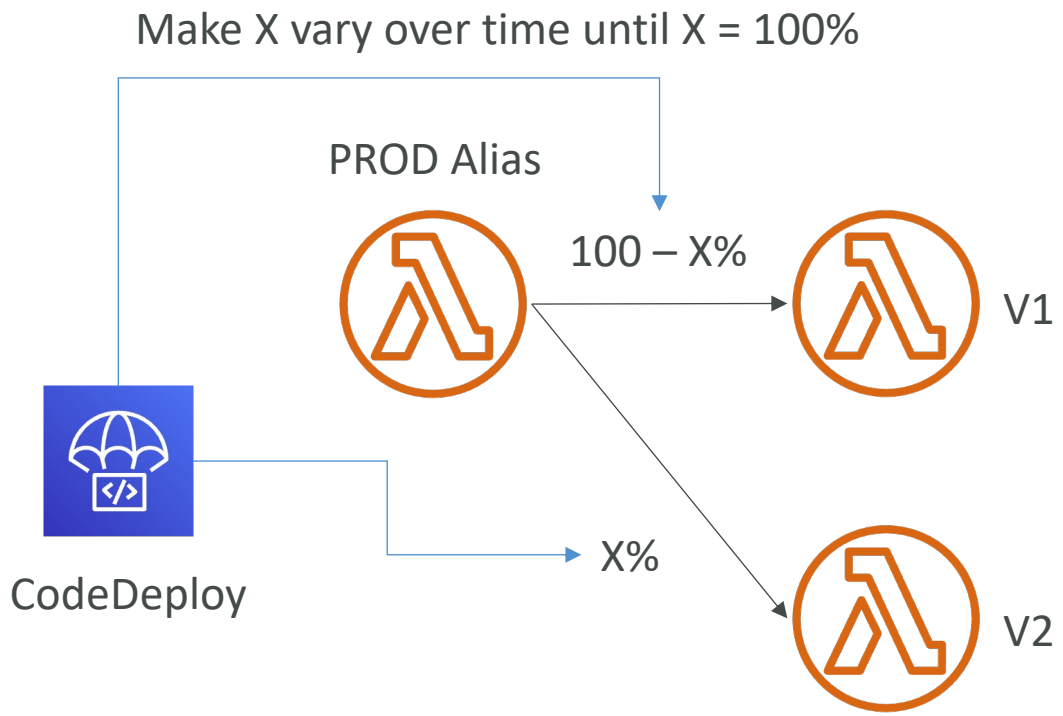


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Lambda & CodeDeploy

- **CodeDeploy** can help you automate traffic shift for Lambda aliases
- Feature is integrated within the SAM framework
- **Linear:** grow traffic every N minutes until 100%
  - Linear10PercentEvery3Minutes
  - Linear10PercentEvery10Minutes
- **Canary:** try X percent then 100%
  - Canary10Percent5Minutes
  - Canary10Percent30Minutes
- **AllAtOnce:** immediate
- Can create Pre & Post Traffic hooks to check the health of the Lambda function



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

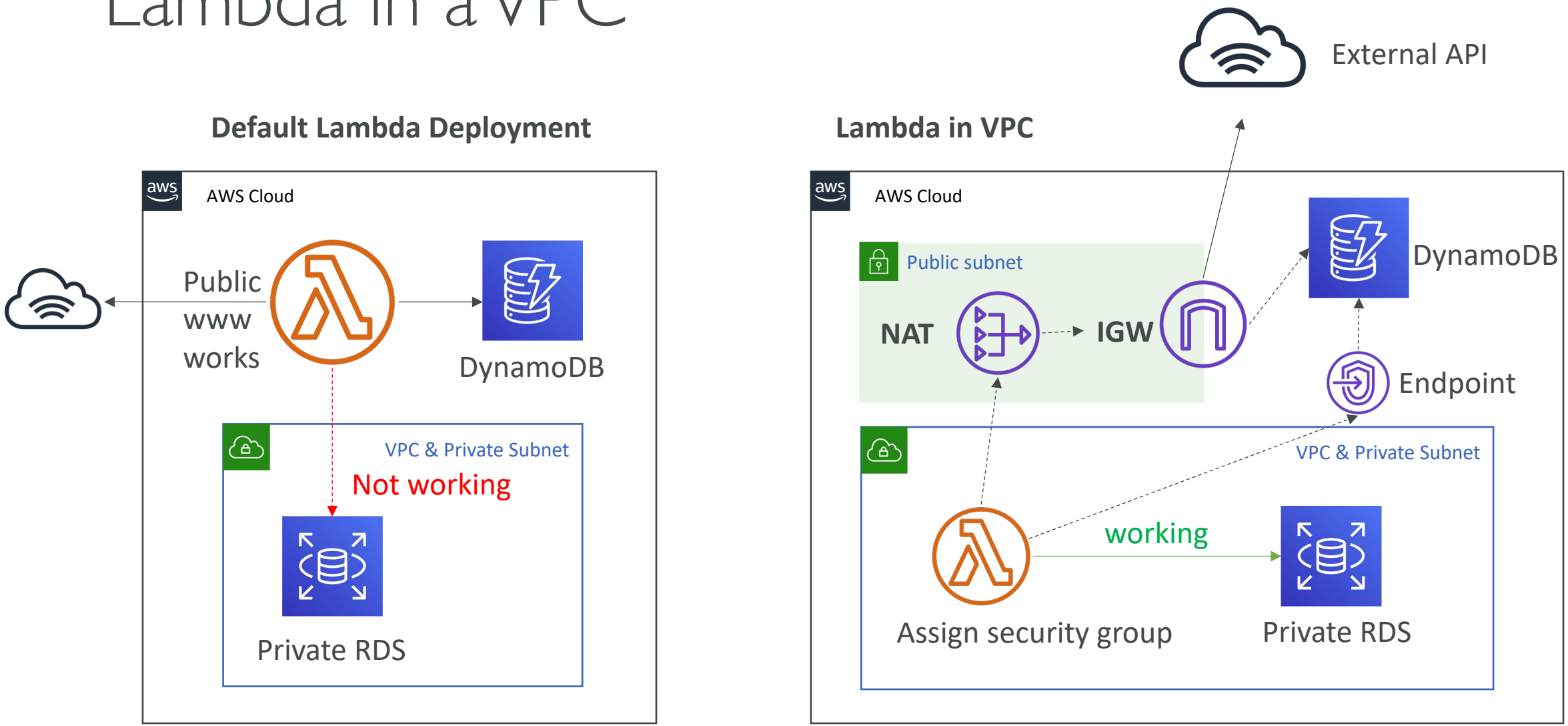
AWS Lambda Logging, Monitoring and Tracing

- CloudWatch:
  - AWS Lambda execution logs are stored in AWS CloudWatch Logs
  - AWS Lambda metrics are displayed in AWS CloudWatch Metrics (successful invocations, error rates, latency, timeouts, etc…)
  - **Make sure your AWS Lambda function has an execution role with an IAM policy that authorizes writes to CloudWatch Logs**
- X-Ray:
  - It's possible to trace Lambda with X-Ray
  - Enable in Lambda configuration (runs the X-Ray daemon for you)
  - Use AWS SDK in Code
  - **Ensure Lambda Function has correct IAM Execution Role**

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Lambda in a VPC

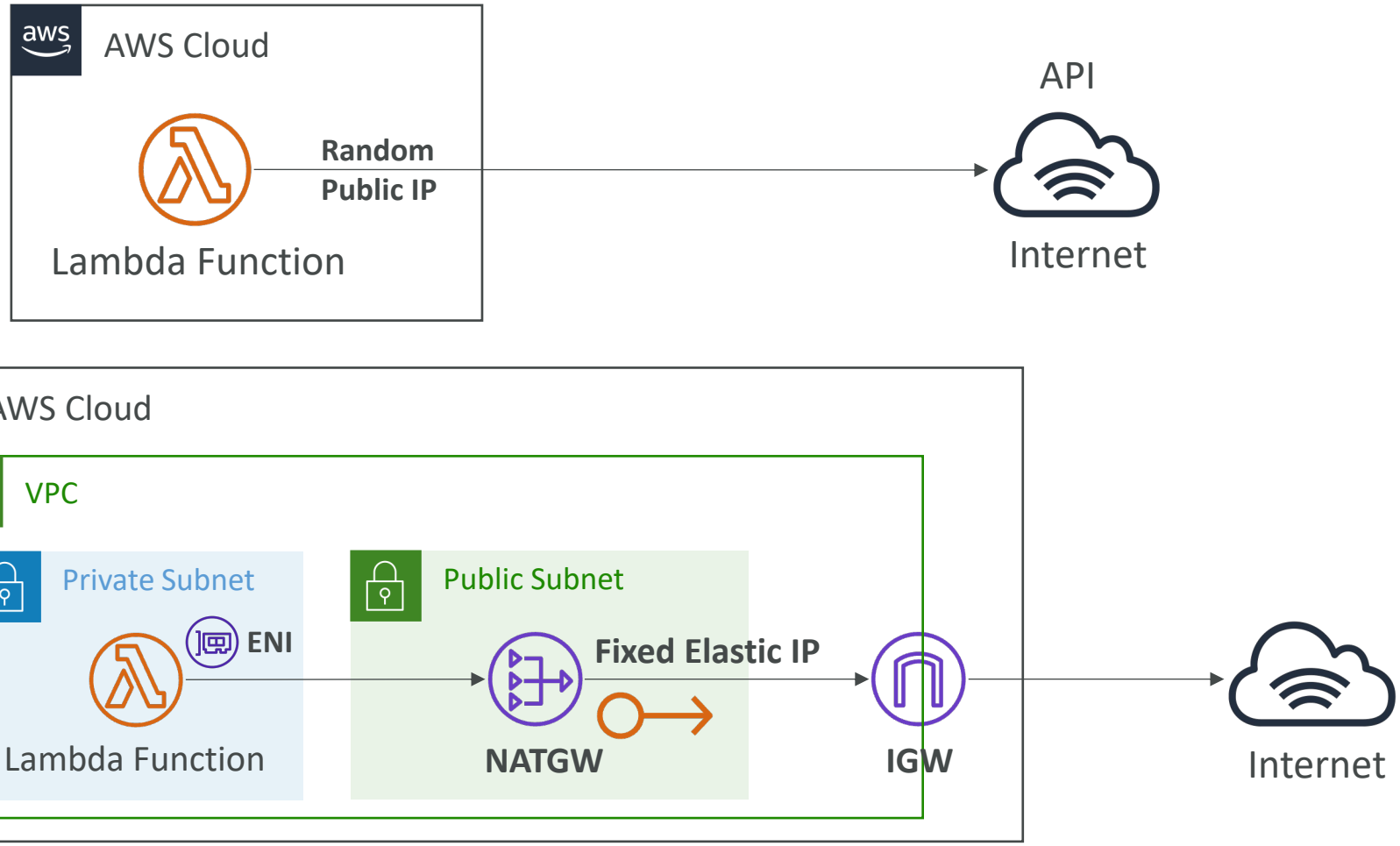


Note: Lambda - CloudWatch Logs works even without endpoint or NAT Gateway

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Lambda – Fixed Public IP for external comms

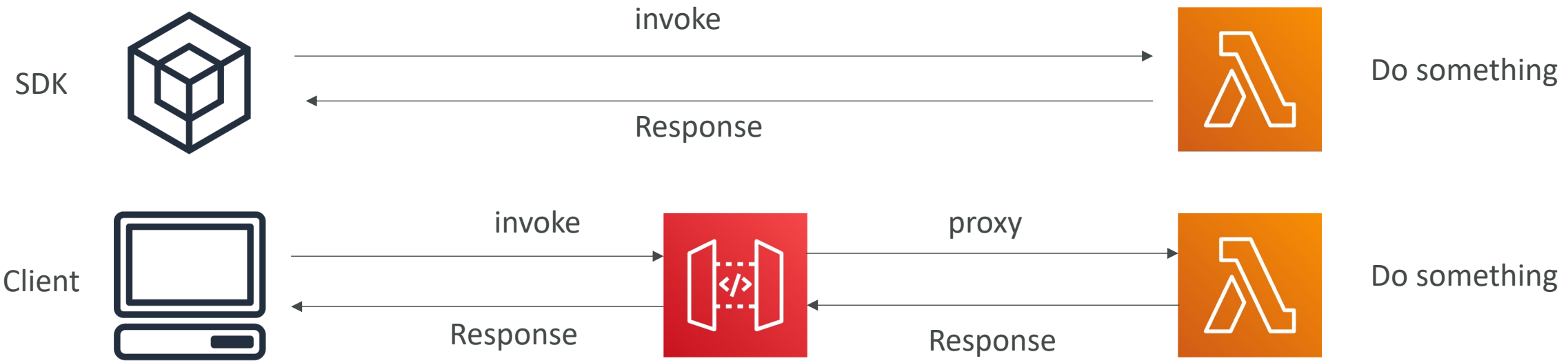


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Lambda – Synchronous Invocations

- Synchronous: CLI, SDK, API Gateway
  - Results is returned right away
  - Error handling must happen client side (retries, exponential backoff, etc…)



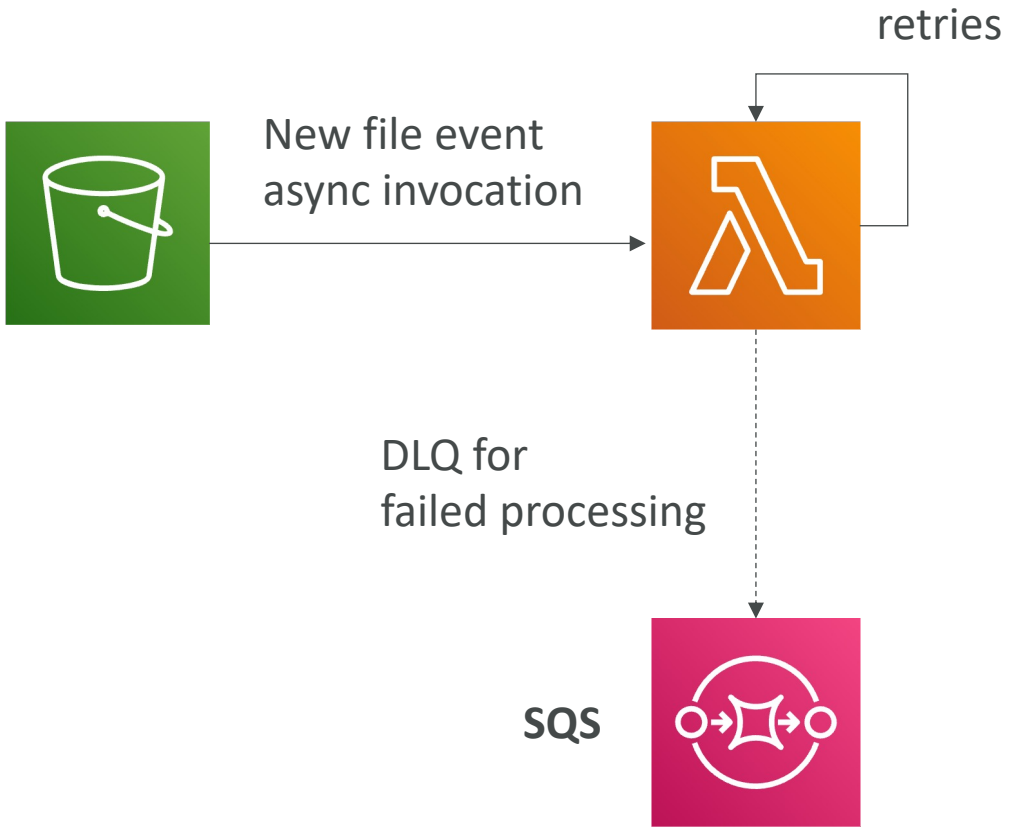
© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Lambda – Asynchronous Invocation

- S3, SNS, Amazon EventBridge…
- Lambda attempts to retry on errors (3 tries total)
- Make sure the processing is **idempotent** (in case of retries)

- Can define a DLQ (dead-letter queue) – **SNS or SQS** – for failed processing

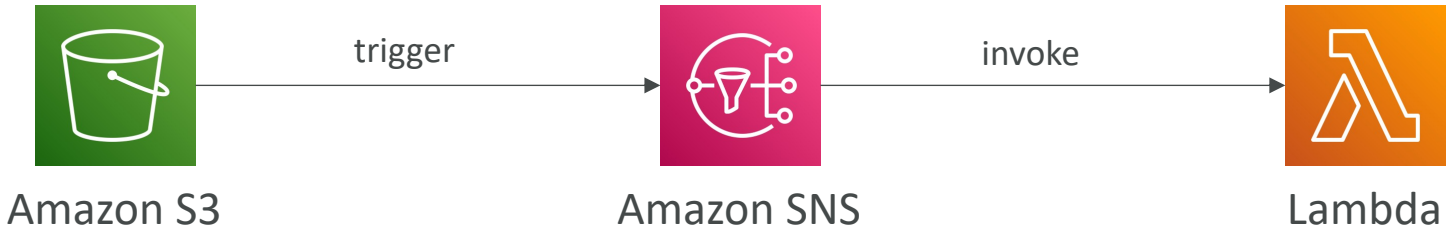


© Stephane Maarek

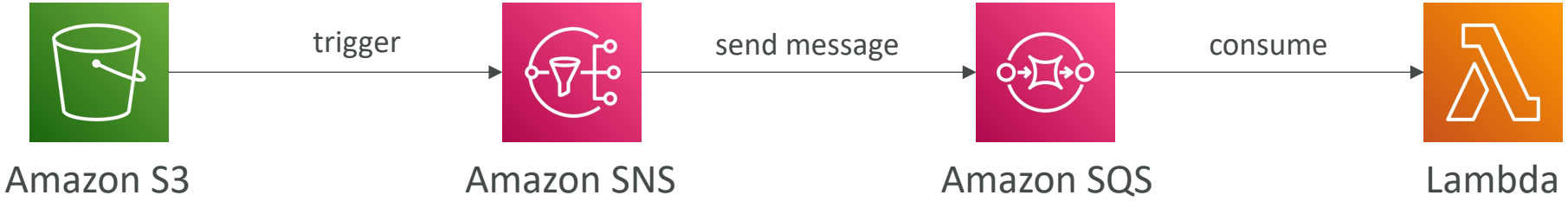
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Lambda – Architecture Discussion

Starts immediately
Parallel executions



Batched execution
Delay



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Types of load balancer on AWS



- AWS has 4 kinds of managed Load Balancers
- **Classic Load Balancer** (v1 - old generation) – 2009 – CLB
  - HTTP, HTTPS, TCP, SSL (secure TCP)
- **Application Load Balancer** (v2 - new generation) – 2016 – ALB
  - HTTP, HTTPS, WebSocket
- **Network Load Balancer** (v2 - new generation) – 2017 – NLB
  - TCP, TLS (secure TCP), UDP
- **Gateway Load Balancer** – 2020 – GWLB
  - Operates at layer 3 (Network layer) – IP Protocol

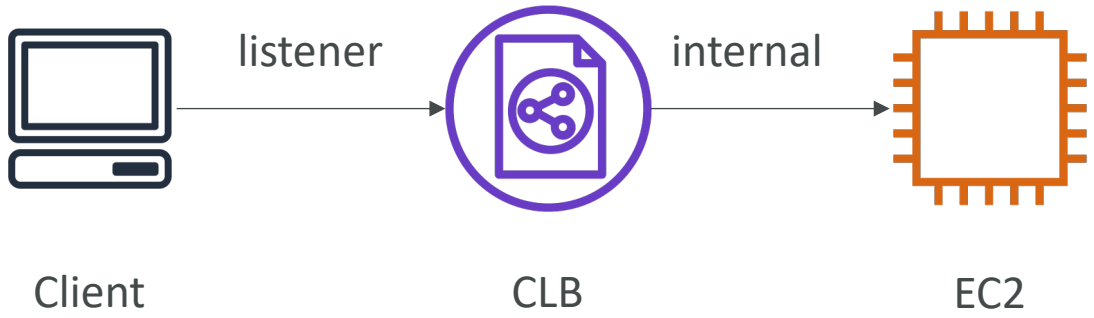
- Overall, it is recommended to use the newer generation load balancers as they provide more features
- Some load balancers can be setup as internal (private) or external (public) ELBs

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Classic Load Balancers (v1)

- Health Checks can be HTTP (L7) or TCP (L4) based including with SSL
- Supports only one SSL certificate
  - The SSL certificate can have many SAN (Subject Alternate Name), but the SSL certificate must be changed anytime a SAN is added / edited / removed
  - Better to use ALB with SNI (Server Name Indication) if possible
  - Can use multiple CLB if you want distinct SSL certificates
- TCP => TCP passes all the traffic to the EC2 instance
  - Only way to use 2-way SSL authentication



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Application Load Balancer (v2)



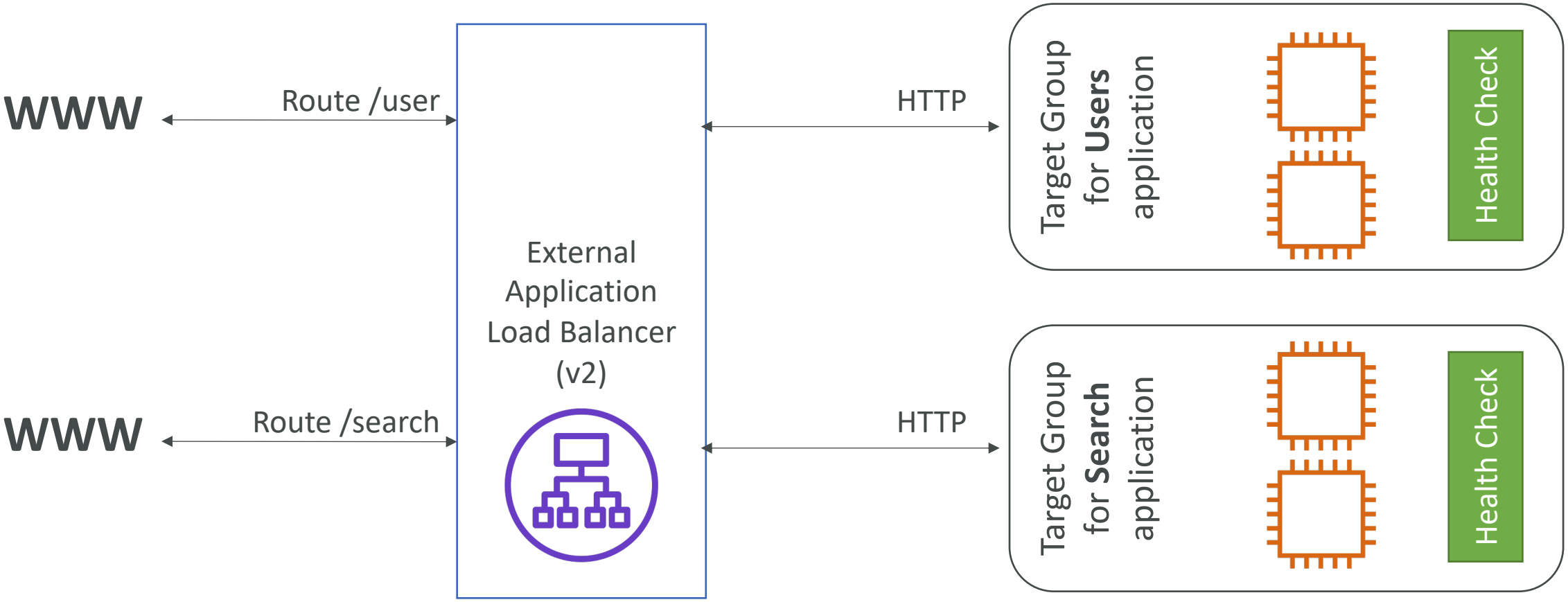
- Application load balancers is Layer 7 (HTTP)

- Load balancing to multiple HTTP applications across machines (target groups)
- Load balancing to multiple applications on the same machine (ex: containers) – great fit with ECS, has dynamic port mapping
- Support for HTTP/2 and WebSocket
- Support redirects (from HTTP to HTTPS for example)
- Routing Rules for path, headers, query string

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Application Load Balancer (v2)
HTTP Based Traffic



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Application Load Balancer (v2) Target Groups

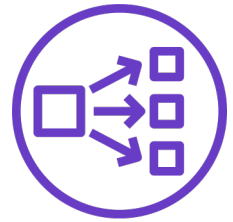
- **EC2 instances** (can be managed by an Auto Scaling Group) – HTTP
- **ECS tasks** (managed by ECS itself) – HTTP
- **Lambda functions** – HTTP request is translated into a JSON event
- **IP Addresses** – must be private IPs

- ALB can route to multiple target groups
- Health checks are at the target group level

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Network Load Balancer (v2)



- Network load balancers (Layer 4) allow to:
  - **Forward TCP & UDP traffic to your instances**
  - Handle millions of request per seconds
  - Less latency ~100 ms (vs 400 ms for ALB)

- **NLB has one static IP per AZ, and supports assigning Elastic IP** (helpful for whitelisting specific IP)

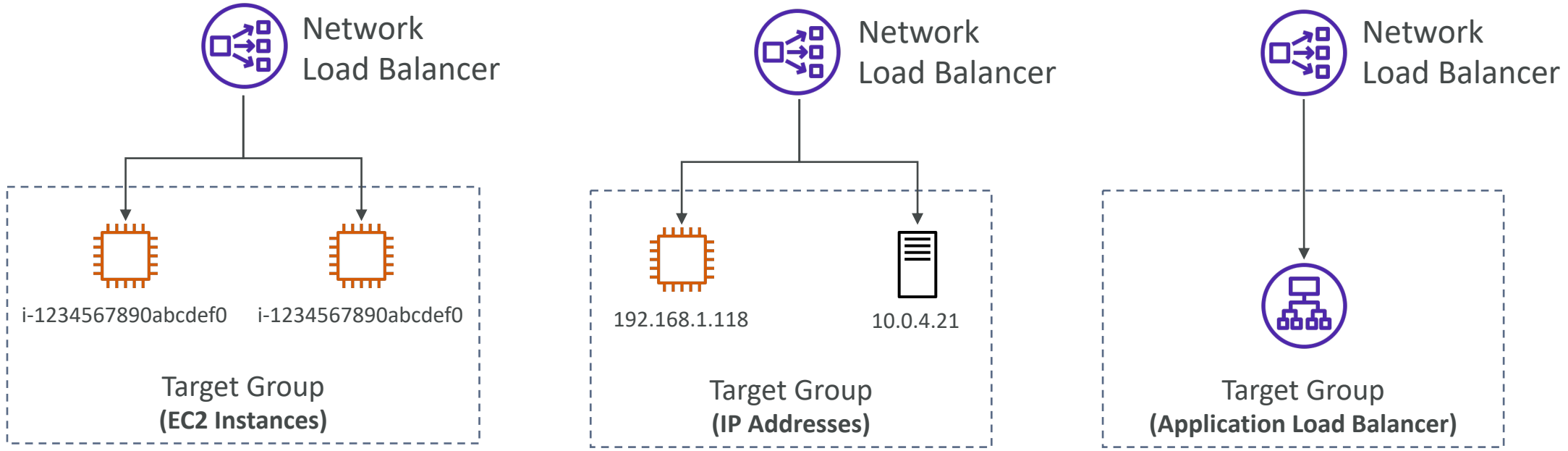
- NLB are used for extreme performance, TCP or UDP traffic
- Not included in the AWS free tier

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Network Load Balancer – Target Groups

- EC2 instances
- IP Addresses – must be private IPs
- Application Load Balancer

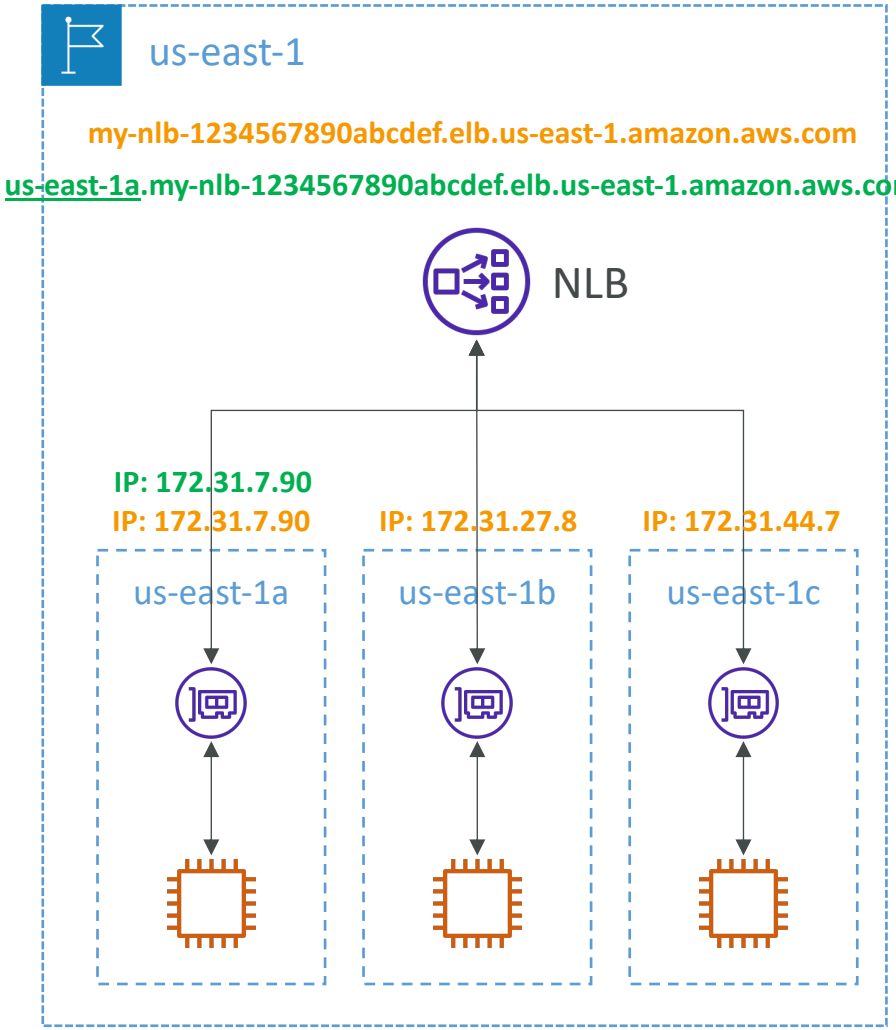


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Network Load Balancer – Zonal DNS Name

- Resolving **Regional NLB DNS** name returns the IP addresses for all NLB nodes in all enabled AZs
  - my-nlb-1234567890abcdef.elb.us-east-1.amazon.aws.com
- **Zonal DNS Name**
  - NLB has DNS names for each of its nodes
  - Use to determine the IP address of each node
  - us-east-1a.my-nlb-1234567890abcdef.elb.us-east-1.amazon.aws.com
  - Used to minimize latency and data transfer costs
  - You need to implement app specific logic



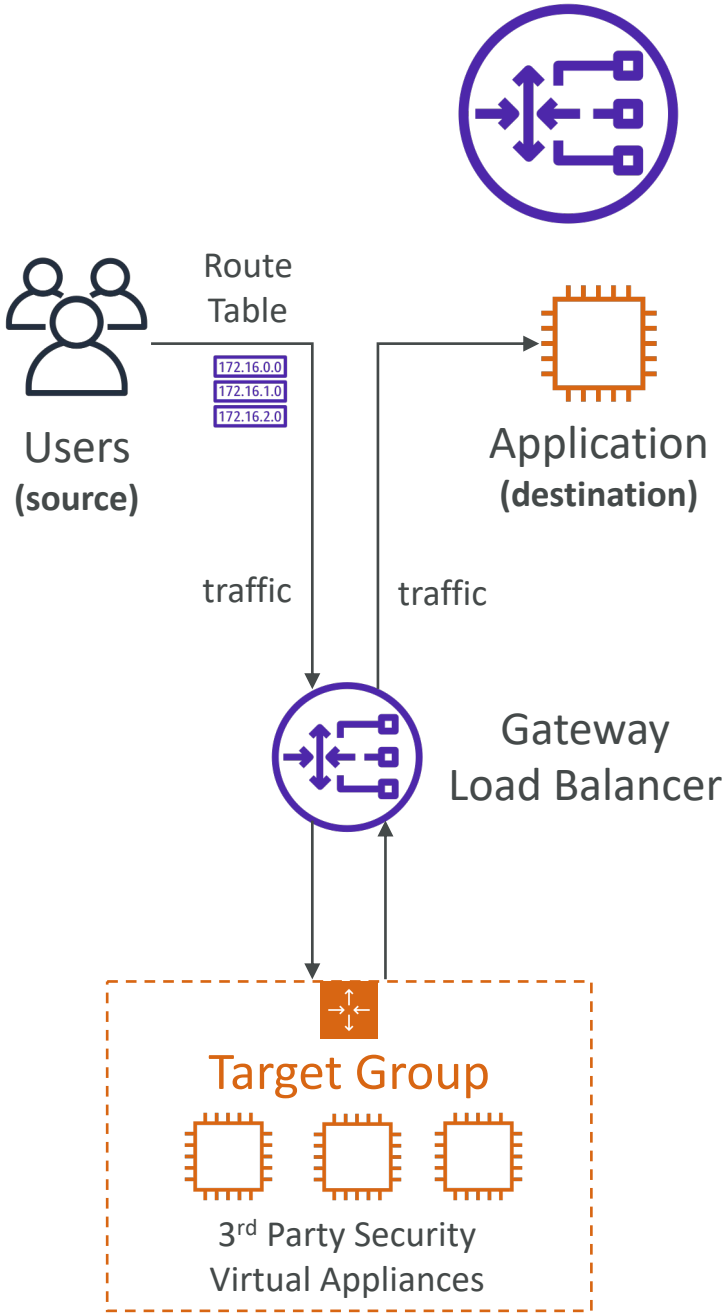
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© Stephane Maarek

Gateway Load Balancer

- Deploy, scale, and manage a fleet of 3rd party network virtual appliances in AWS
- Example: Firewalls, Intrusion Detection and Prevention Systems, Deep Packet Inspection Systems, payload manipulation, …

- Operates at Layer 3 (Network Layer) – IP Packets
- Combines the following functions:
  - **Transparent Network Gateway** – single entry/exit for all traffic
  - **Load Balancer** – distributes traffic to your virtual appliances
- Uses the **GENEVE** protocol on port **6081**

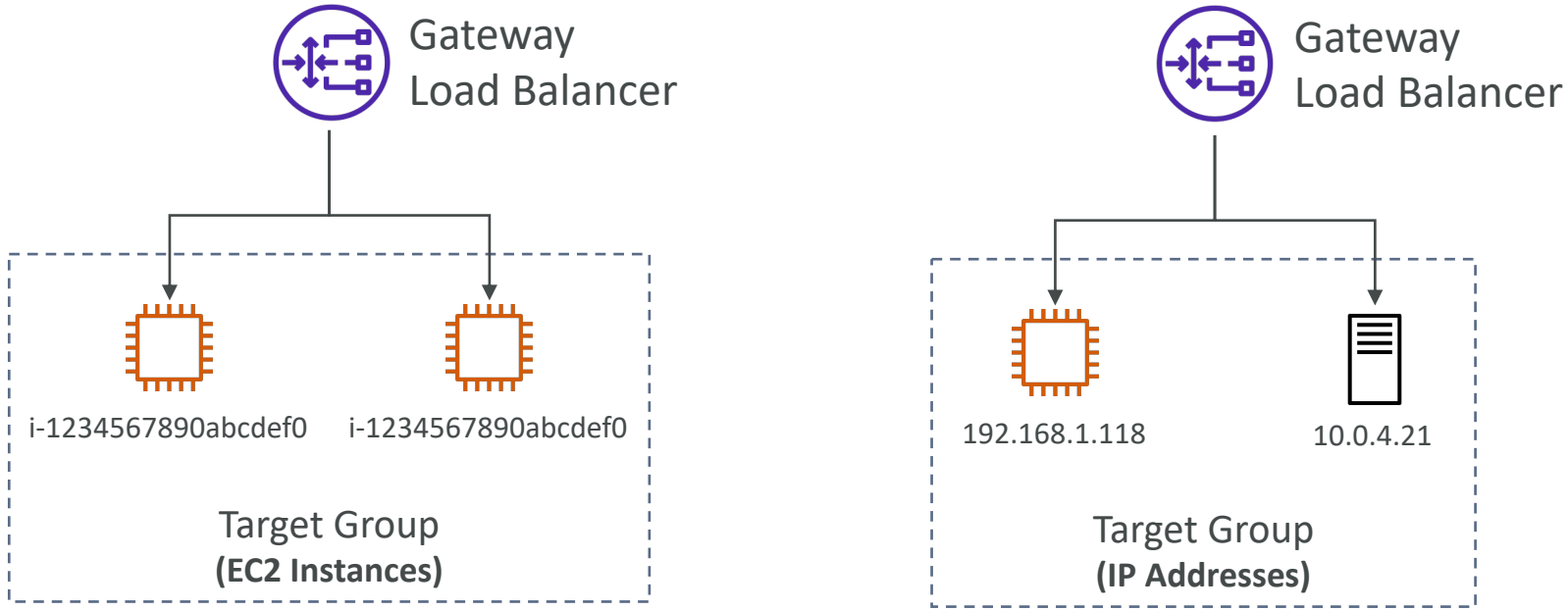


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Gateway Load Balancer – Target Groups

- EC2 instances
- IP Addresses – must be private IPs



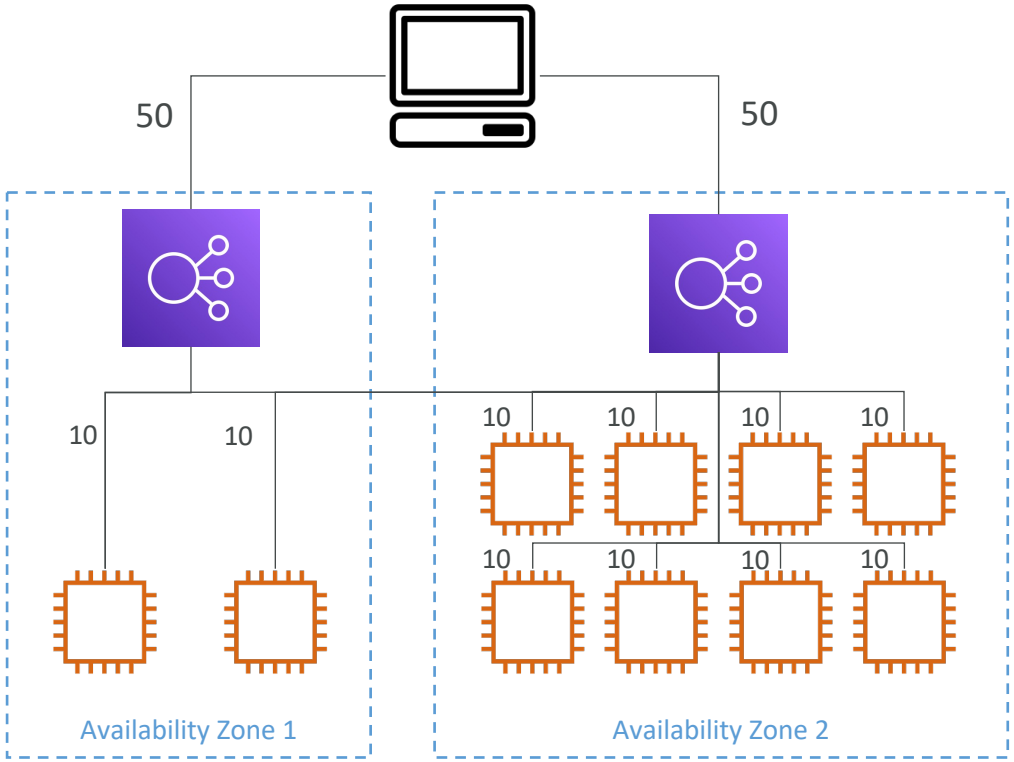
© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Cross-Zone Load Balancing

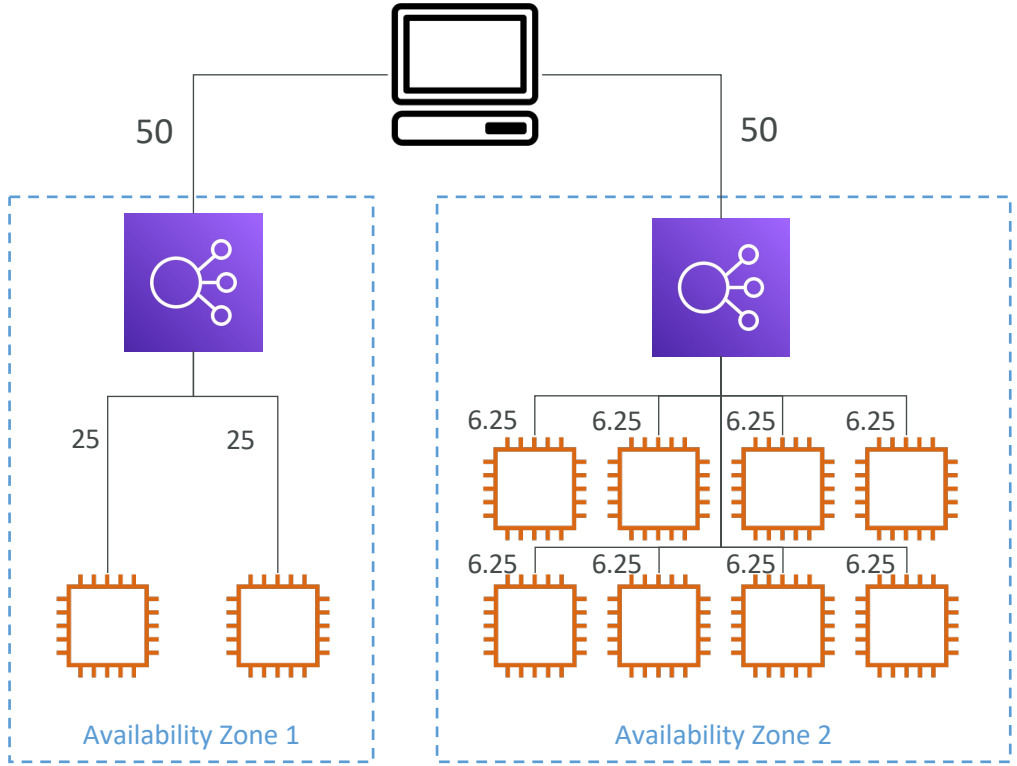
With Cross Zone Load Balancing:

each load balancer instance distributes evenly across all registered instances in all AZ



Without Cross Zone Load Balancing:

Requests are distributed in the instances of the node of the Elastic Load Balancer



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Cross-Zone Load Balancing

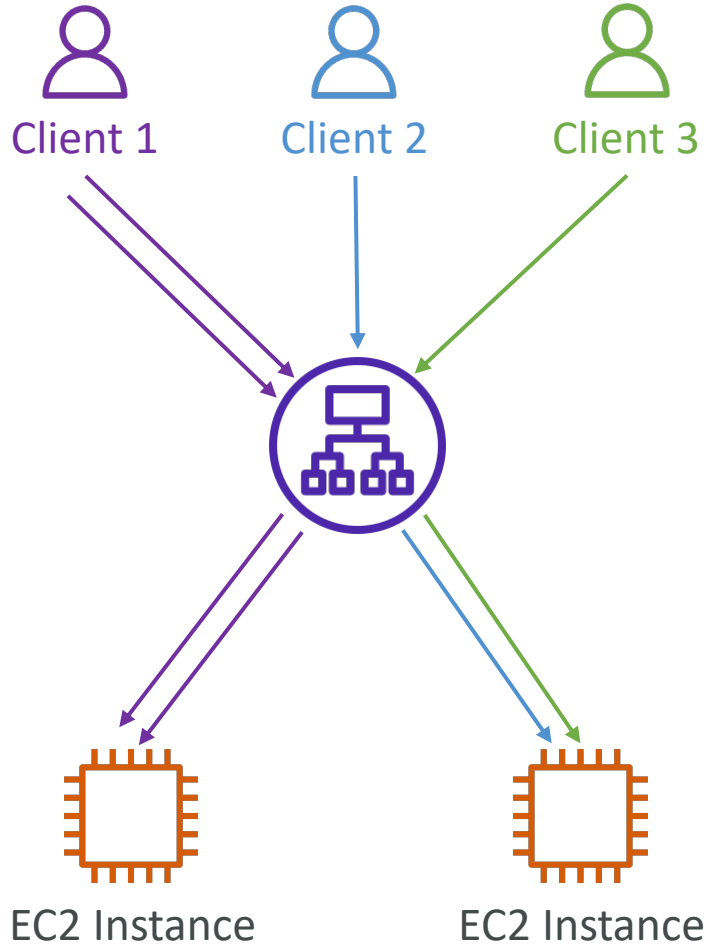
- Classic Load Balancer
  - Disabled by default
  - No charges for inter AZ data if enabled
- Application Load Balancer
  - Always on (can't be disabled)
  - No charges for inter AZ data
- Network Load Balancer
  - Disabled by default
  - You pay charges ($) for inter AZ data if enabled
- Gateway Load Balancer
  - Disabled by default
  - You pay charges ($) for inter AZ data if enabled

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Sticky Sessions (Session Affinity)

- It is possible to implement stickiness so that the same client is always redirected to the same instance behind a load balancer
- This works for Classic Load Balancers & Application Load Balancers
- The "cookie" used for stickiness has an expiration date you control
- Use case: make sure the user doesn't lose his session data
- Enabling stickiness may bring imbalance to the load over the backend EC2 instances

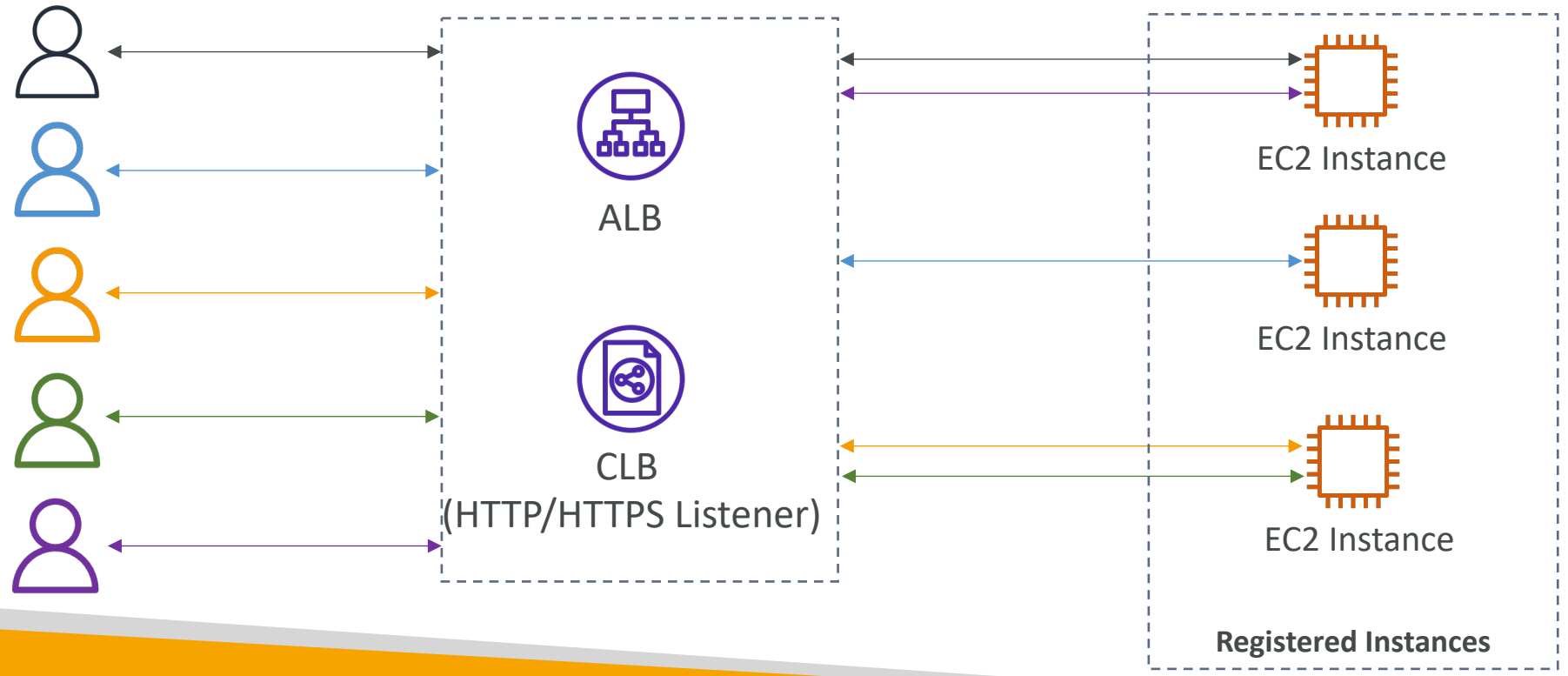


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Request Routing Algorithms – Least Outstanding Requests

- The next instance to receive the request is the instance that has the lowest number of pending/unfinished requests
- Works with **Application Load Balancer** and **Classic Load Balancer (HTTP/HTTPS)**

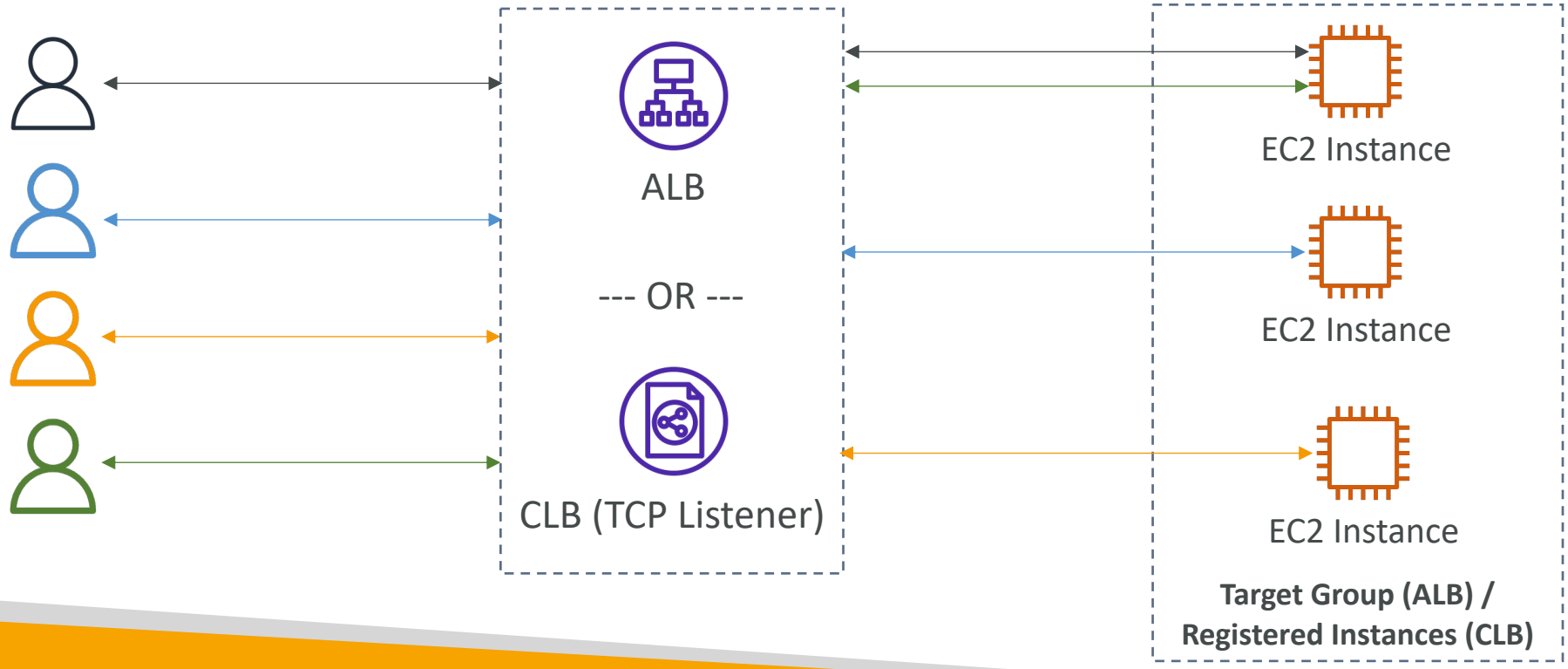


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Request Routing Algorithms – Round Robin

- Equally choose the targets from the target group
- Works with **Application Load Balancer** and **Classic Load Balancer (TCP)**

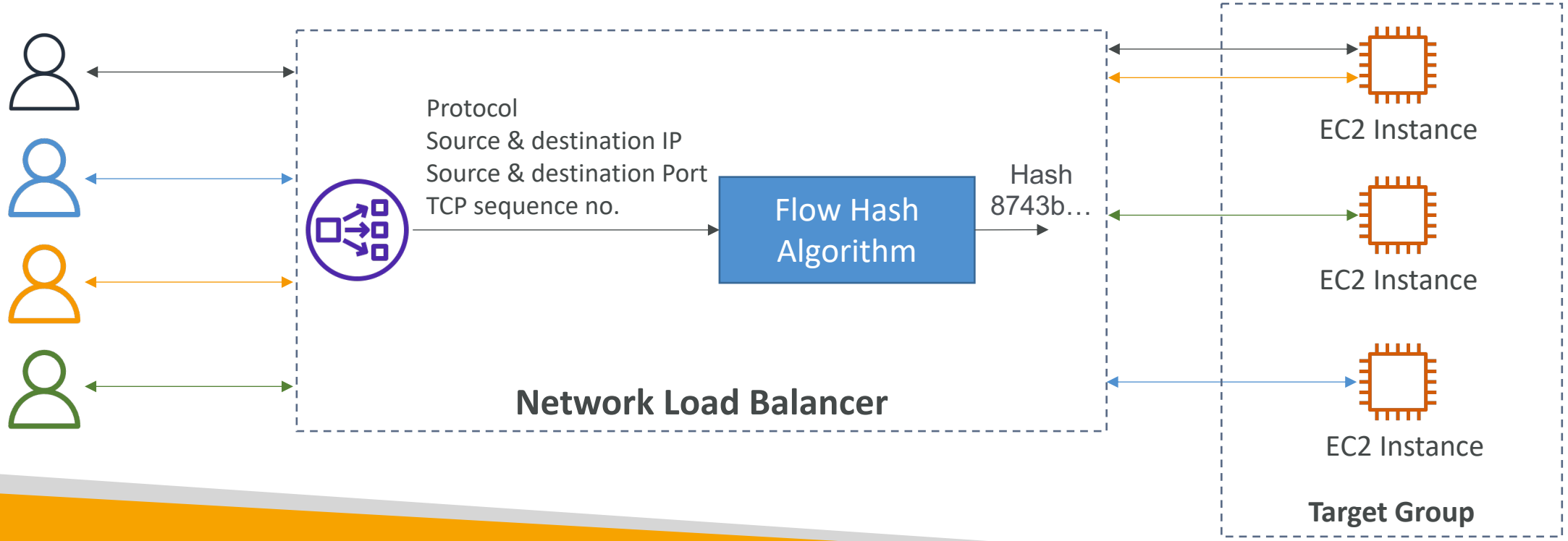


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Request Routing Algorithms – Flow Hash

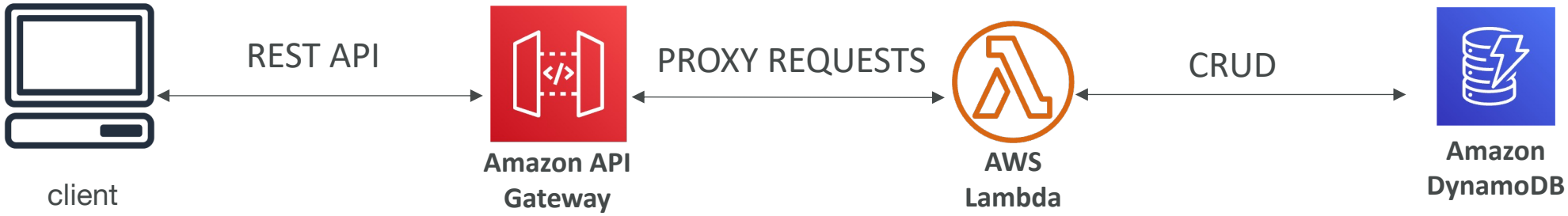
- Selects a target based on the protocol, source/destination IP address, source/destination port, and TCP sequence number
- Each TCP/UDP connection is routed to a single target for the life of the connection
- Works with **Network Load Balancer**



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway – Overview



- Helps expose Lambda, HTTP & AWS Services as an API
- API versioning, authorization, traffic management (API keys, throttles), huge scale, serverless, req/resp transformations, OpenAPI spec, CORS

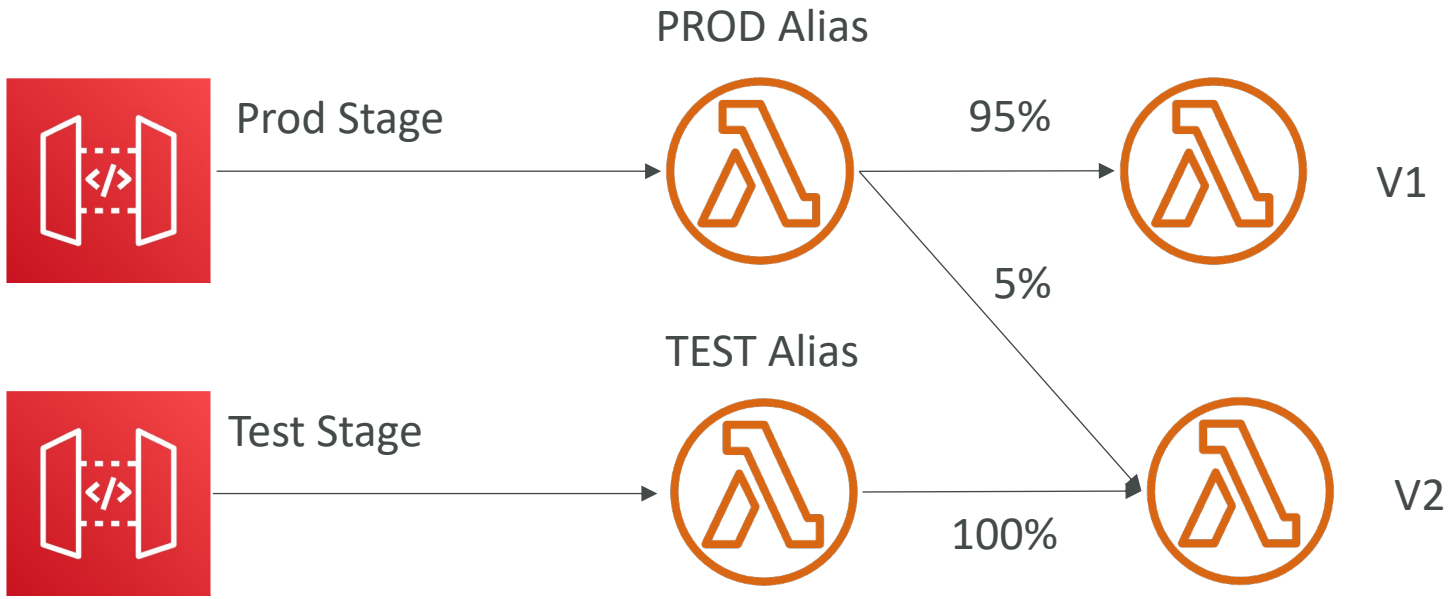
- Limits to know:
  - 29 seconds timeout
  - 10 MB max payload size

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway – Deployment Stages

- API changes are deployed to "Stages" (as many as you want)
- Use the naming you like for stages (dev, test, prod)
- Stages can be rolled back as a history of deployments is kept



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway – Integrations

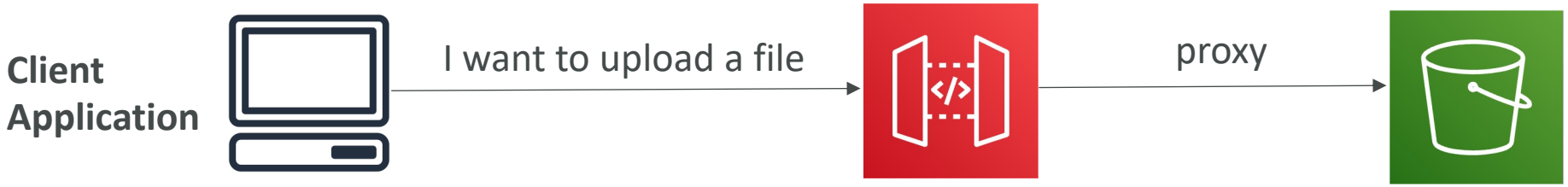
- HTTP
  - Expose HTTP endpoints in the backend
  - Example: internal HTTP API on premise, Application Load Balancer…
  - Why? Add rate limiting, caching, user authentications, API keys, etc…
- Lambda Function
  - Invoke Lambda function
  - Easy way to expose REST API backed by AWS Lambda
- AWS Service
  - Expose any AWS API through the API Gateway?
  - Example: start an AWS Step Function workflow, post a message to SQS
  - Why? Add authentication, deploy publicly, rate control…

© Stephane Maarek

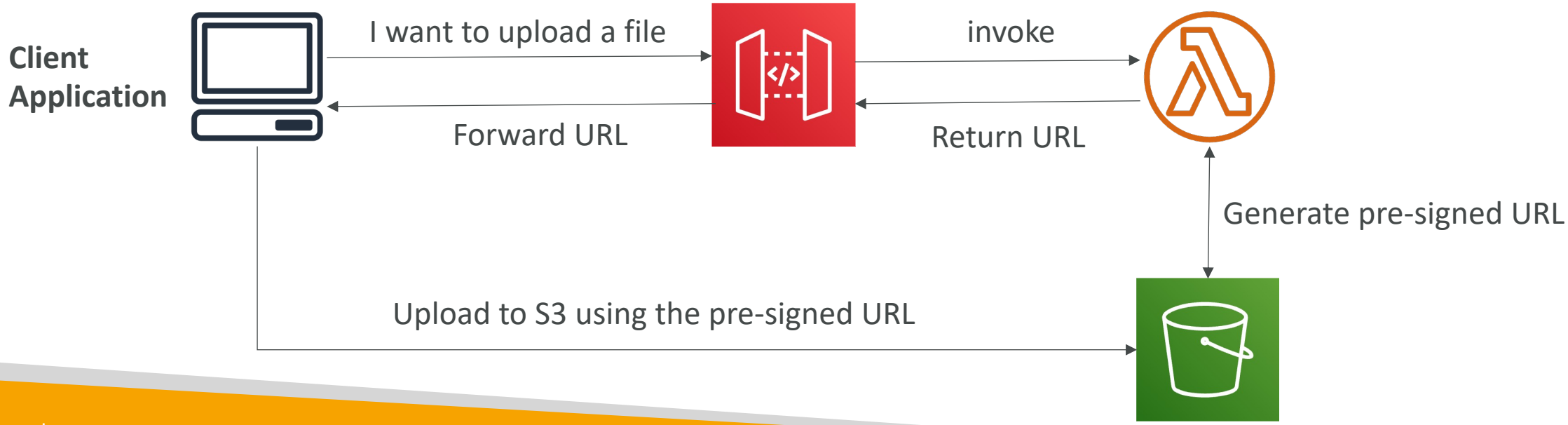
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Solution Architecture Discussion: API Gateway in front of S3

- You will be impacted by the **10 MB payload size limit**



- Better architecture:



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway - Endpoint Types

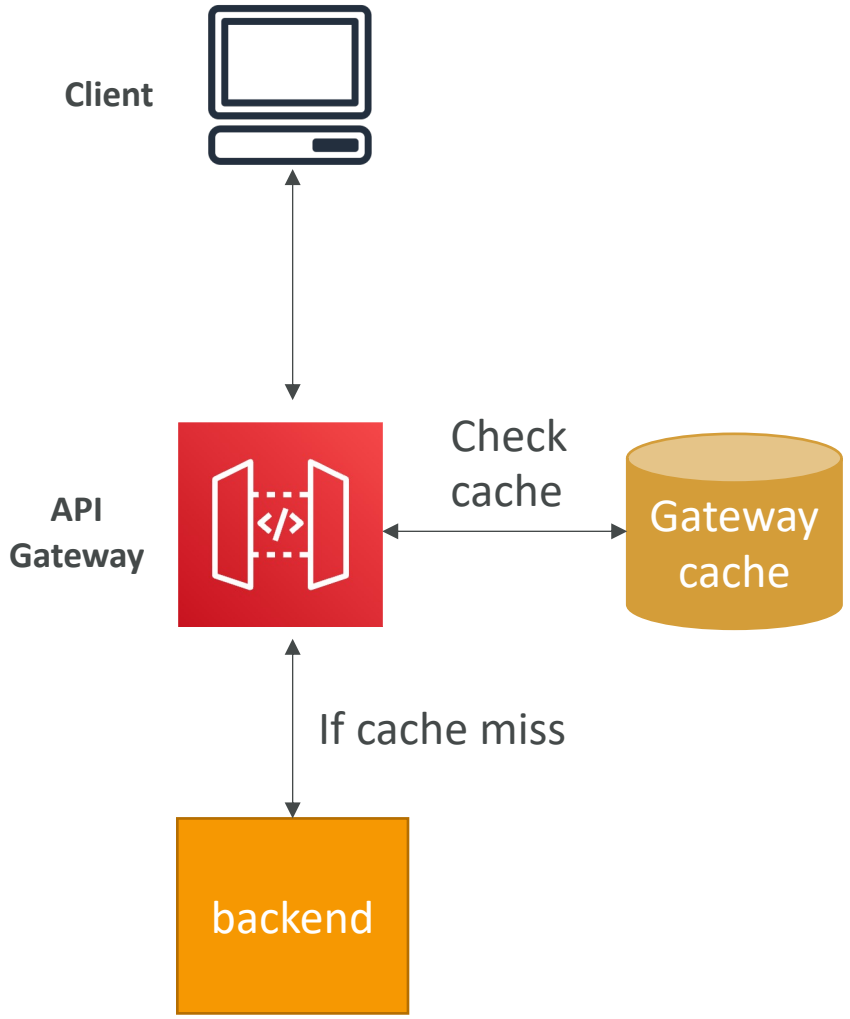
- **Edge-Optimized (default):** For global clients
  - Requests are routed through the CloudFront Edge locations (improves latency)
  - The API Gateway still lives in only one region
- **Regional:**
  - For clients within the same region
  - Could manually combine with CloudFront (more control over the caching strategies and the distribution)
- **Private:**
  - Can only be accessed from your VPC using an interface VPC endpoint (ENI)
  - Use a resource policy to define access

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Caching API responses

- Caching reduces the number of calls made to the backend
- Default TTL (time to live) is 300 seconds (min: 0s, max: 3600s)
- Caches are defined **per stage**
- Possible to override cache settings **per method**
- Clients can invalidate the cache with header: **Cache-Control: max-age=0** (with proper IAM authorization)
- Able to flush the entire cache (invalidate it) immediately
- Cache encryption option
- Cache capacity between 0.5GB to 237GB



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway - Errors

- 4xx means Client errors
  - 400: Bad Request
  - 403: Access Denied, WAF filtered
  - 429: Quota exceeded, Throttle

- 5xx means Server errors
  - 502: Bad Gateway Exception, usually for an incompatible output returned from a Lambda proxy integration backend and occasionally for out-of-order invocations due to heavy loads.
  - 503: Service Unavailable Exception
  - 504: Integration Failure – ex Endpoint Request Timed-out Exception
    API Gateway requests time out after 29 second maximum

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway – Security

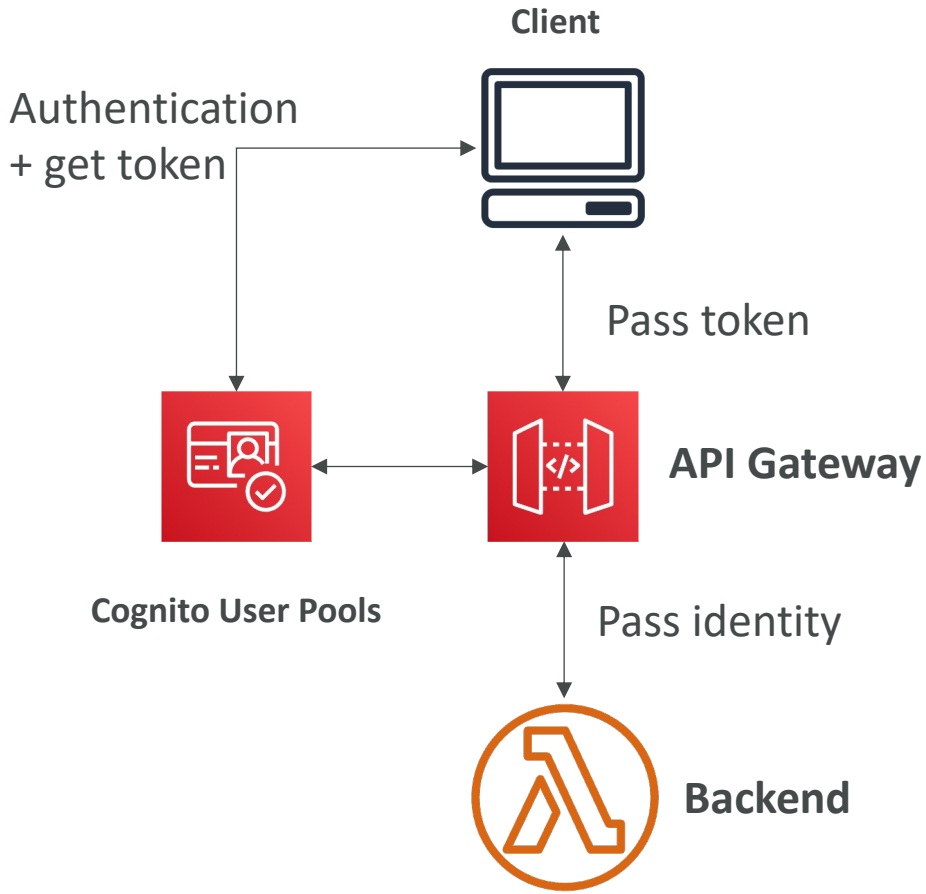
- Load SSL certificates and use Route53 to define a CNAME
- Resource Policy (~S3 Bucket Policy):
  - control who can access the API
  - Users from AWS accounts, IP or CIDR blocks, VPC or VPC Endpoints
- IAM Execution Roles for API Gateway at the API level
  - To invoke a Lambda Function, an AWS service…
- CORS (Cross-origin resource sharing):
  - Browser based security
  - Control which domains can call your API

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway – Authentication

- IAM based access (AWS_IAM)
  - Good for providing access within your infrastructure
  - Pass IAM credentials in headers through Sig V4
- Lambda Authorizer (formerly Custom Authorizer)
  - Use Lambda to verify a custom OAuth / SAML / 3rd party authentication
- Cognito User Pools
  - Client authenticates with Cognito
  - Client passes the token to API Gateway
  - API Gateway knows out-of-the-box how to verify to token



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway – Logging, Monitoring, Tracing

- CloudWatch Logs:
  - Enable CloudWatch logging at the Stage level (with Log Level – ERROR, INFO)
  - Can log full requests / responses data
  - Can send API Gateway Access Logs (customizable)
  - Can send logs directly into Kinesis Data Firehose (as an alternative to CW logs)
- CloudWatch Metrics:
  - Metrics are by stage, possibility to enable detailed metrics
  - *IntegrationLatency*, *Latency*, *CacheHitCount*, *CacheMissCount*
- X-Ray:
  - Enable tracing to get extra information about requests in API Gateway
  - X-Ray API Gateway + AWS Lambda gives you the full picture

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway – Usage Plans & API Keys

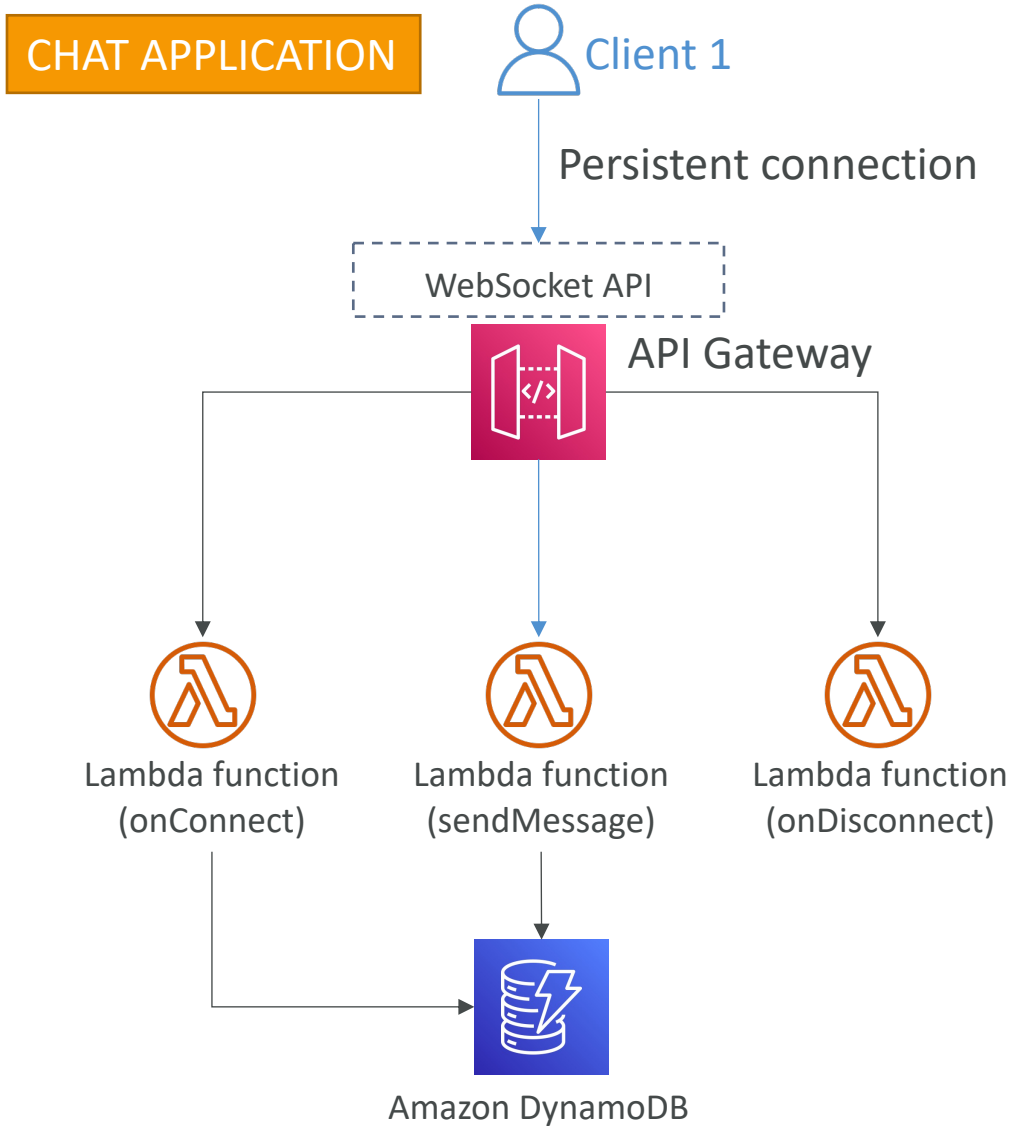
- If you want to make an API available as an offering ($) to your customers
- **Usage Plan:**
  - who can access one or more deployed API stages and methods
  - how much and how fast they can access them
  - uses API keys to identify API clients and meter access
  - configure throttling limits and quota limits that are enforced on individual client
- **API Keys:**
  - alphanumeric string values to distribute to your customers
  - Ex: WBjHxNtoAb4WPKBC7cGm64CBibIb24b4jt8jJHo9
  - Can use with usage plans to control access
  - Throttling limits are applied to the API keys
  - Quotas limits is the overall number of maximum requests
- **429 Too Many Requests:**
  - Account level throttling across all APIs in a region
  - Clients must implement retry mechanisms

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway – WebSocket API – Overview

- What's WebSocket?
  - Two-way interactive communication between a user's browser and a server
  - Server can push information to the client
  - This enables **stateful** application use cases
- WebSocket APIs are often used in **real-time applications** such as chat applications, collaboration platforms, multiplayer games, and financial trading platforms.
- Works with AWS Services (Lambda, DynamoDB) or HTTP endpoints

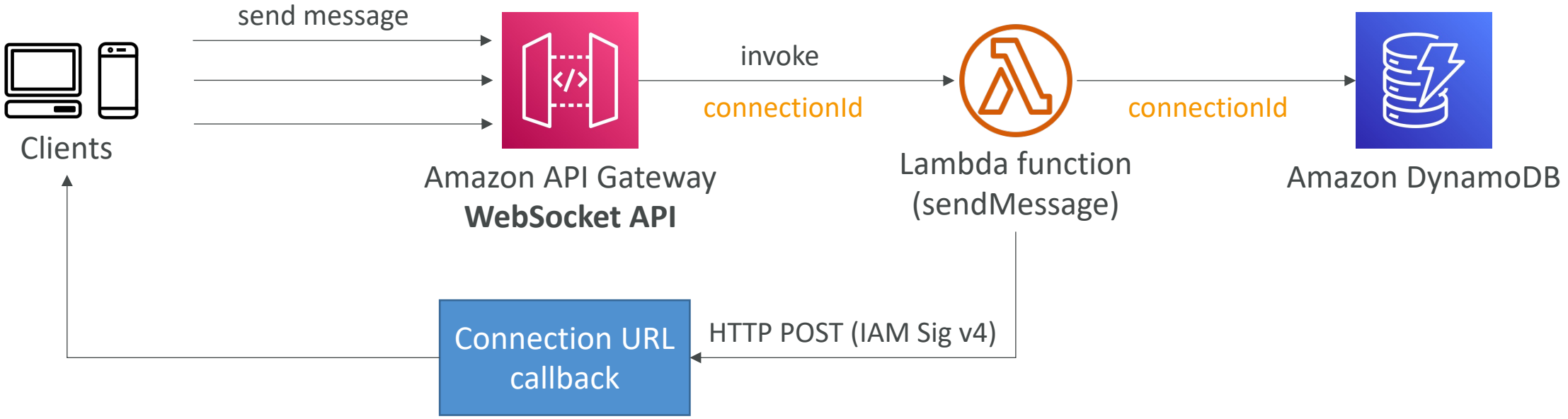


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Server to Client Messaging @connections used for replies to clients

WebSocket URL
wss://abcdef.execute-api.us-west-1.amazonaws.com/dev



Connection URL
wss://abcdef.execute-api.us-west-1.amazonaws.com/dev/@connections/connectionId

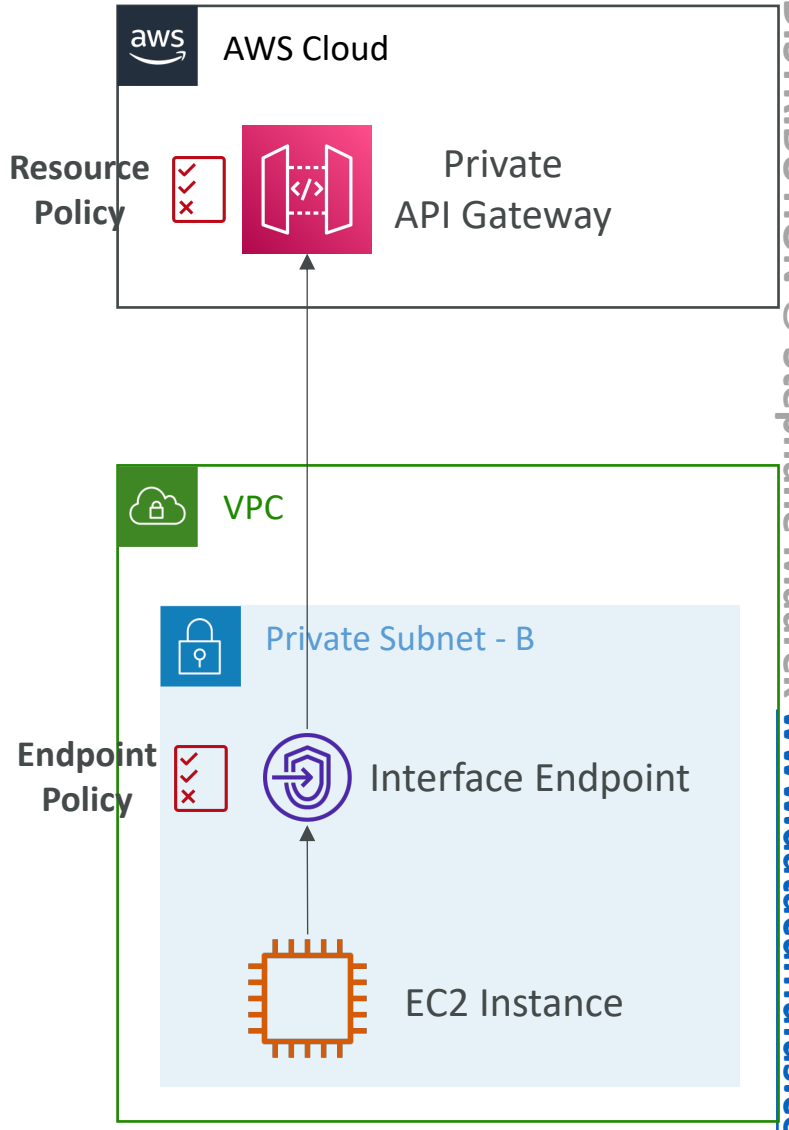
© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway – Private APIs

- Can only be accessed from your VPC by using an VPC Interface Endpoint
- Each VPC Interface Endpoint can be used to access multiple Private APIs

- API Gateway Resource Policy
  - Allow or deny access to API from selected VPCs and VPC Endpoints, including across AWS accounts
  - **aws:SourceVpc** and **aws:SourceVpce**



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© Stephane Maarek

AWS AppSync - Overview

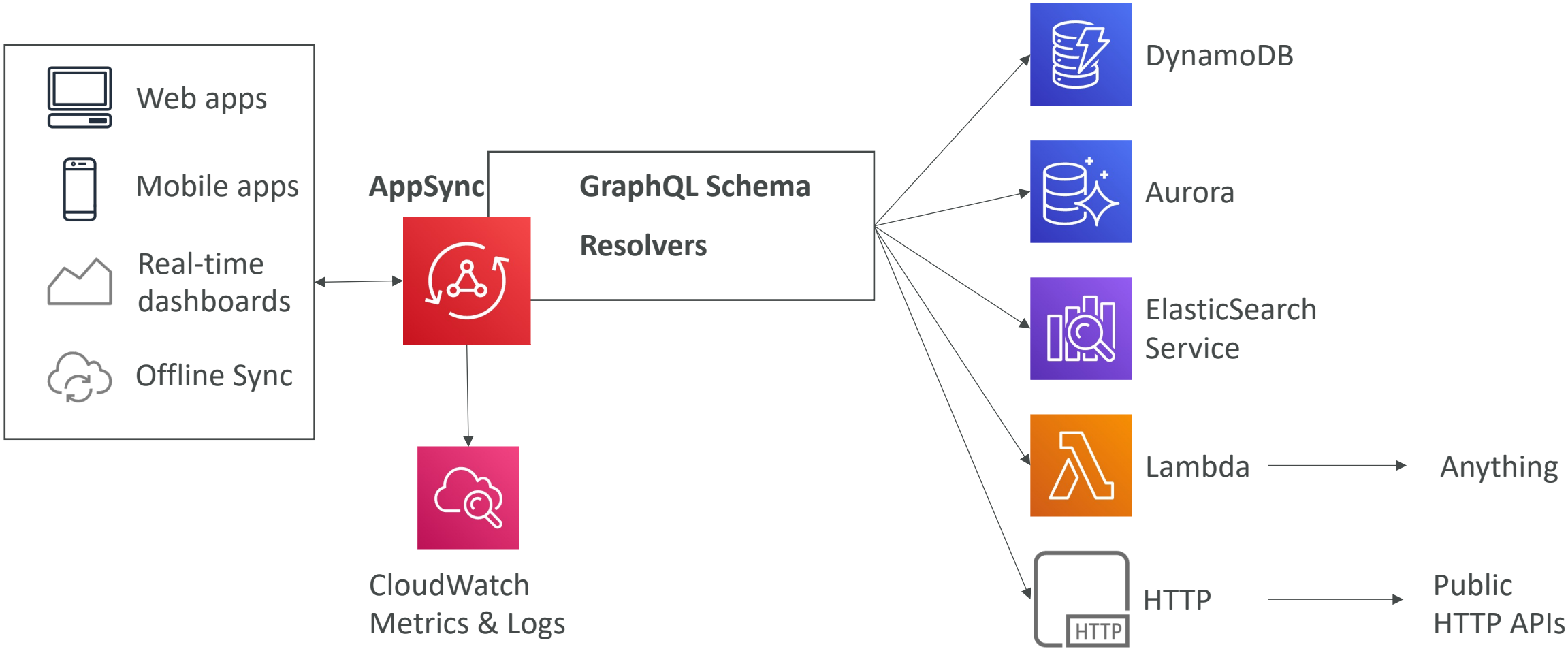


- **AppSync** is a managed service that uses **GraphQL**
- **GraphQL** makes it easy for applications to get exactly the data they need.
- This includes combining data from **one or more sources**
  - NoSQL data stores, Relational databases, HTTP APIs…
  - Integrates with DynamoDB, Aurora, Elasticsearch & others
  - Custom sources with AWS Lambda
- Retrieve data in **real-time with WebSocket or MQTT on WebSocket**
- For mobile apps: local data access & data synchronization
- It all starts with uploading one **GraphQL schema**

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AppSync Diagram



© Stephane Maarek

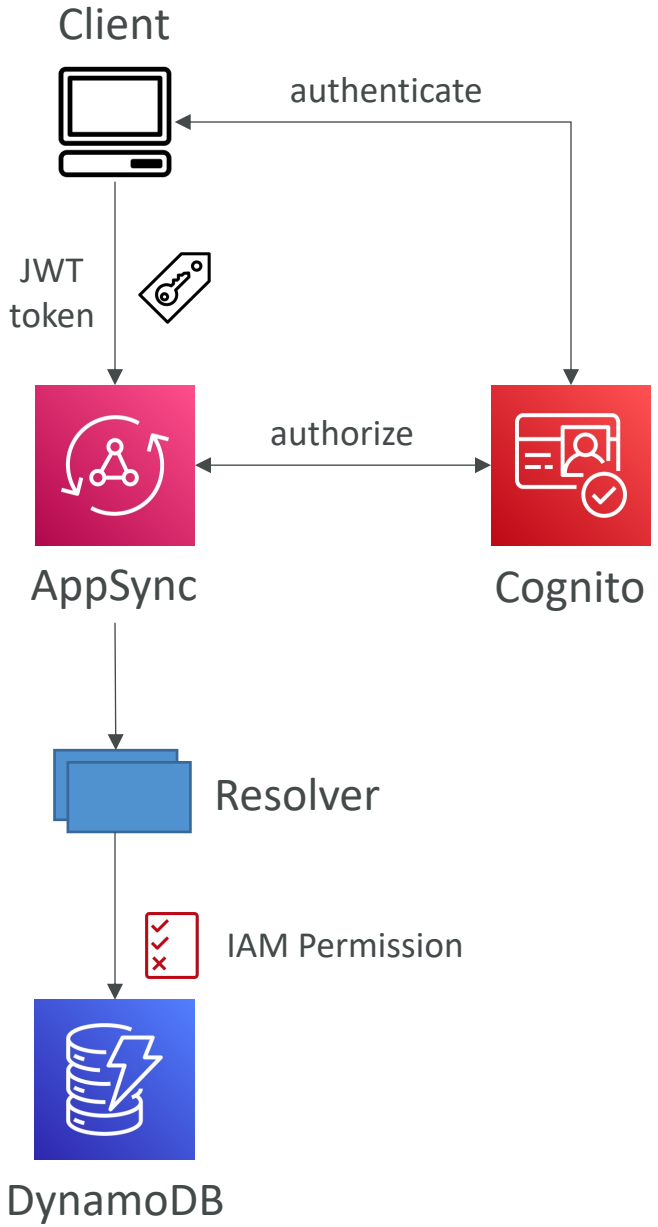
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AppSync – Cognito Integration

- Perform authorization on Cognito users based on the groups they belong to
- In the GraphQL schema, you can specify the security for Cognito groups

```
type Query {
    posts: [Post!]!
    @aws_auth(cognito_groups: ["Bloggers", "Readers"])
}

Type Mutation {
    addPost(id: ID!, title: String!): Post!
    @aws_auth(cognito_groups: ["Bloggers"])
}
```



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

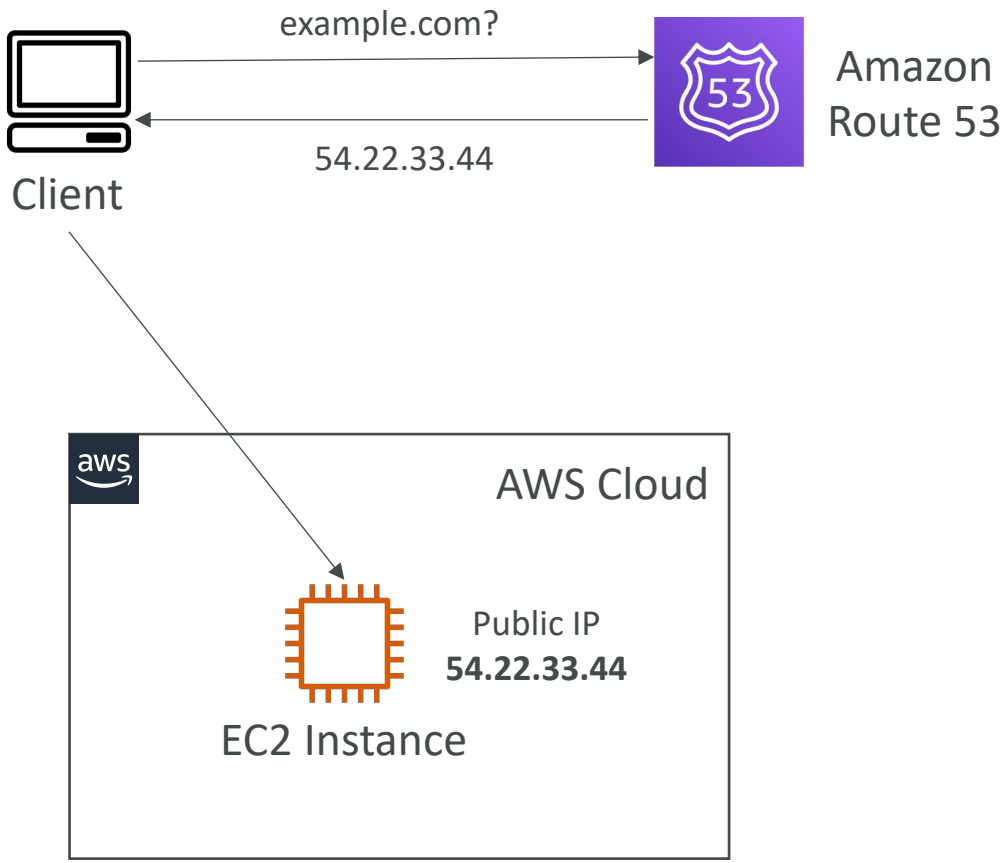
Route 53 – Record Types

- A – maps a hostname to IPv4
- AAAA – maps a hostname to IPv6
- CNAME – maps a hostname to another hostname
  - The target is a domain name which must have an A or AAAA record
  - Can't create a CNAME record for the top node of a DNS namespace (Zone Apex)
  - Example: you can't create for example.com, but you can create for www.example.com
- NS – Name Servers for the Hosted Zone
  - Control how traffic is routed for a domain

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Route 53 – Diagram for A record



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Route 53 – CNAME vs. Alias

- AWS Resources (Load Balancer, CloudFront...) expose an AWS hostname:
  - lb1-1234.us-east-2.elb.amazonaws.com and you want myapp.mydomain.com

- CNAME:
  - Points a hostname to any other hostname. (app.mydomain.com => blabla.anything.com)
  - ONLY FOR NON ROOT DOMAIN (aka. something.mydomain.com)
- Alias:
  - Points a hostname to an AWS Resource (app.mydomain.com => blabla.amazonaws.com)
  - Works for ROOT DOMAIN and NON ROOT DOMAIN (aka mydomain.com)
  - Free of charge
  - Native health check

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Route 53 – Alias Records Targets

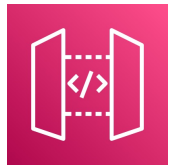
- Elastic Load Balancers
- CloudFront Distributions
- API Gateway
- Elastic Beanstalk environments
- S3 Websites
- VPC Interface Endpoints
- Global Accelerator accelerator
- Route 53 record in the same hosted zone



Elastic
Load Balancer



Amazon
CloudFront



Amazon
API Gateway



Elastic Beanstalk



S3 Websites



VPC Interface
Endpoints



Global Accelerator



Route 53 Record
(same Hosted Zone)

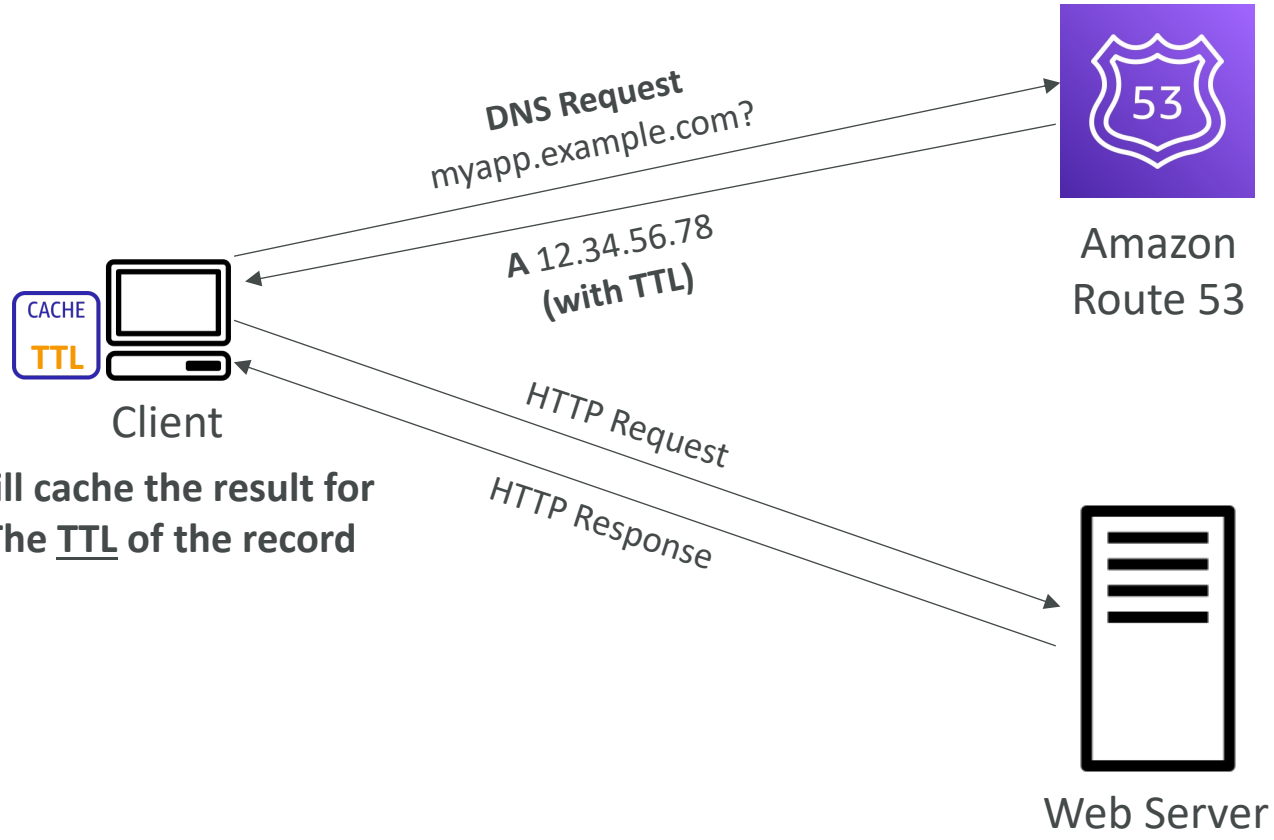
- You cannot set an ALIAS record for an EC2 DNS name

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Route 53 – Records TTL (Time To Live)

- High TTL – e.g., 24 hr
  - Less traffic on Route 53
  - Possibly outdated records
- Low TTL – e.g., 60 sec.
  - More traffic on Route 53 ($$)
  - Records are outdated for less time
  - Easy to change records
- Except for Alias records, TTL is mandatory for each DNS record



© Stephane Maarek

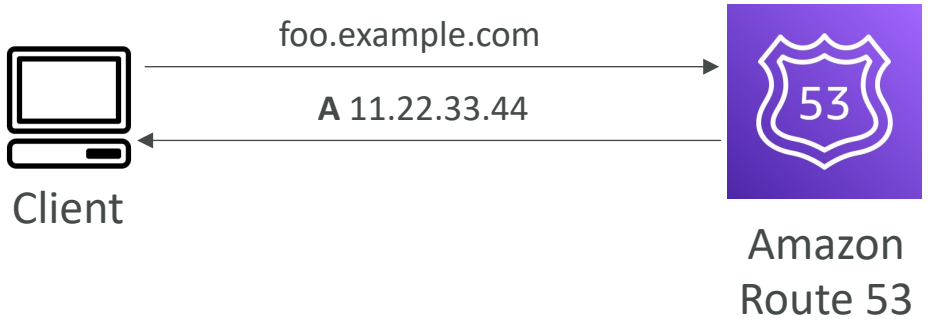
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Routing Policies – Simple

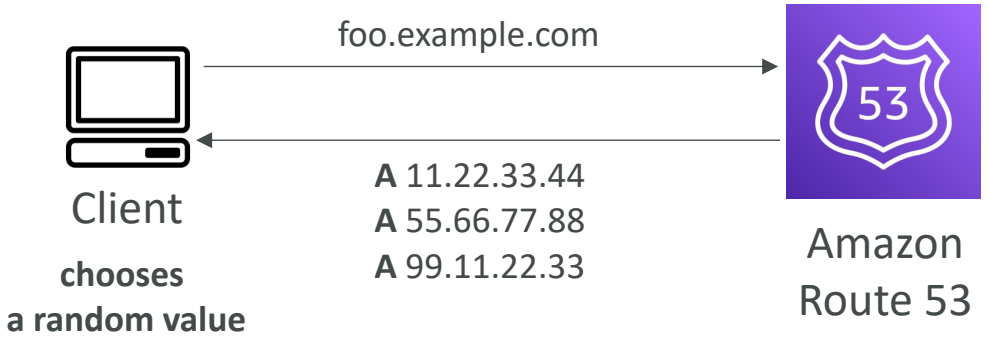
- Typically, route traffic to a single resource
- Can't be associated with Health Checks

- Can specify multiple values in the same record
- If multiple values are returned, a random one is chosen by the client

Single Value



Multiple Value

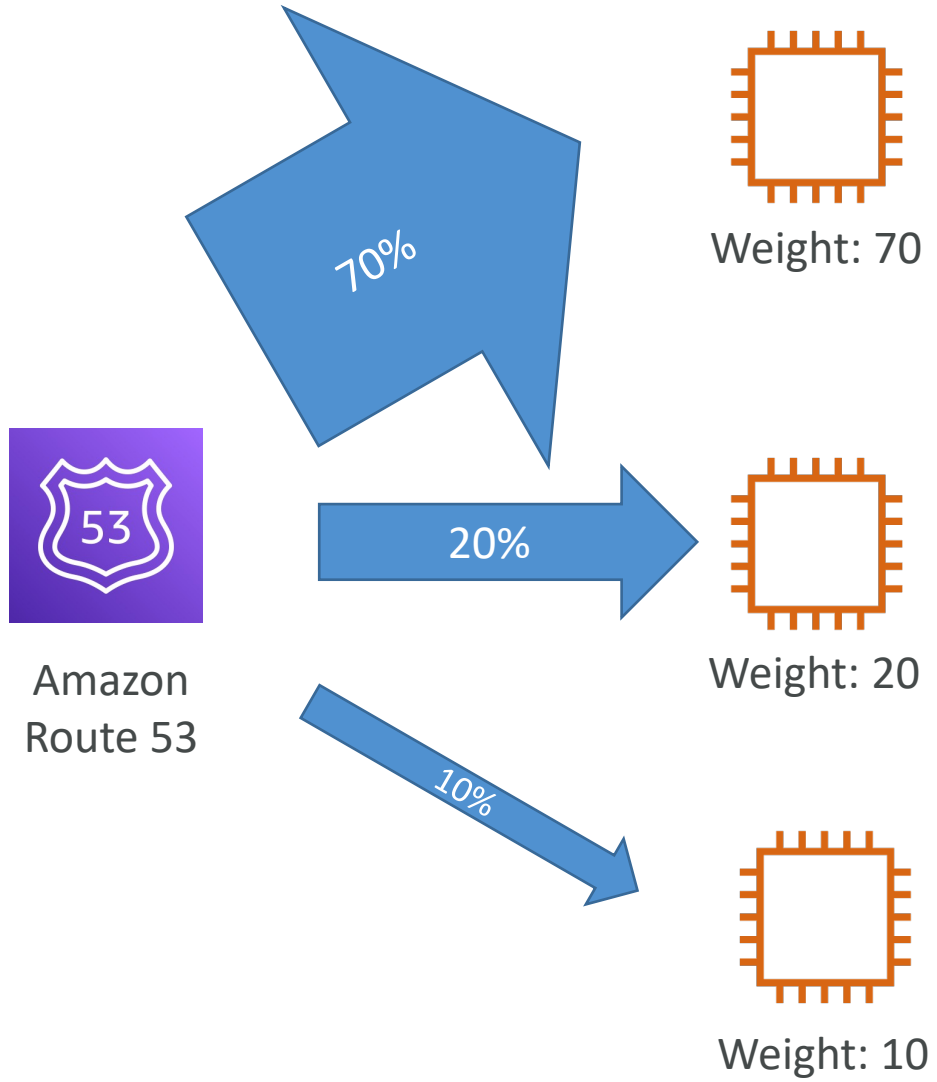


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Routing Policies – Weighted

- Control the % of the requests that go to each specific resource
- Can be associated with Health Checks
- Use cases: load balancing between regions, testing new application versions…

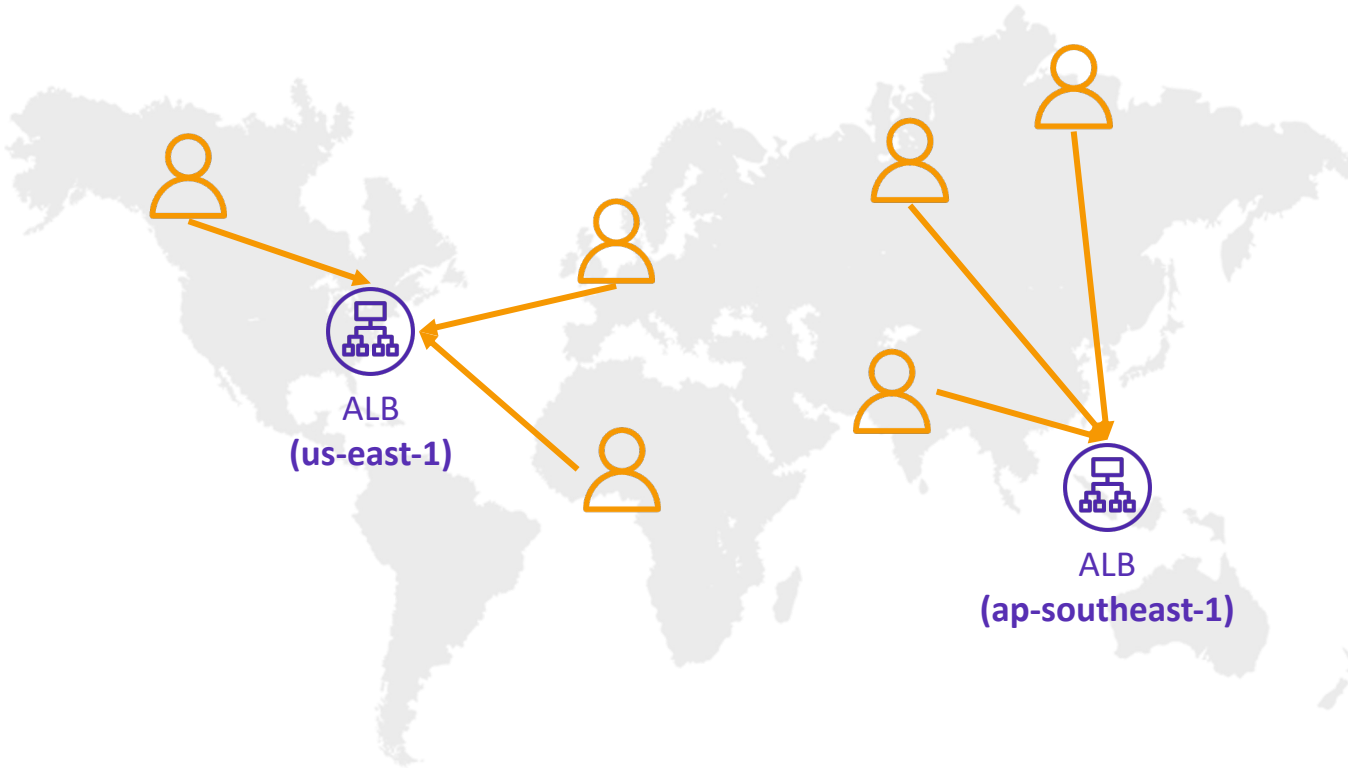


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Routing Policies – Latency-based

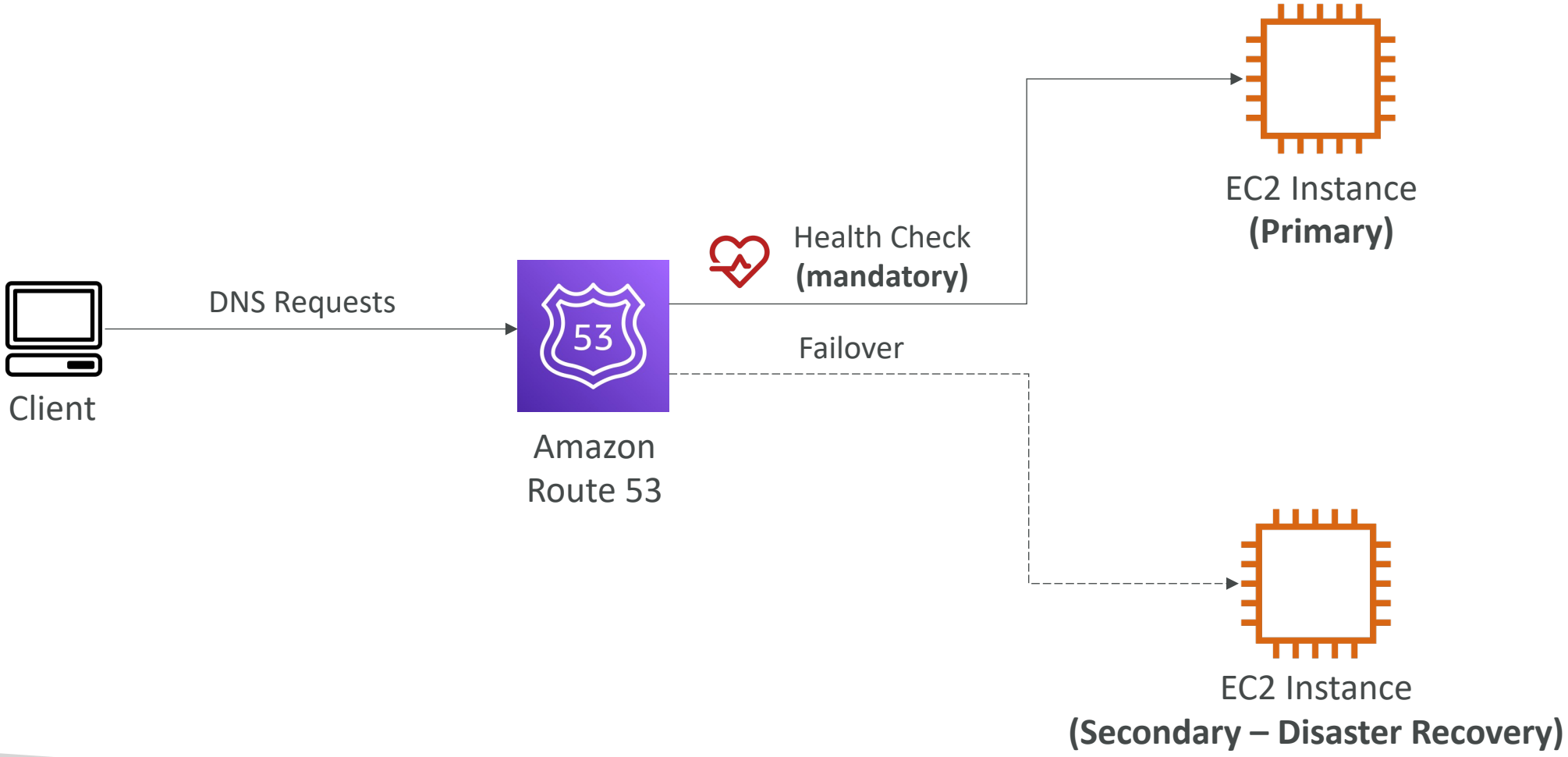
- Redirect to the resource that has the least latency close to us
- Super helpful when latency for users is a priority
- **Latency is based on traffic between users and AWS Regions**
- Germany users may be directed to the US (if that's the lowest latency)
- Can be associated with Health Checks (has a failover capability)



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Routing Policies – Failover (Active-Passive)

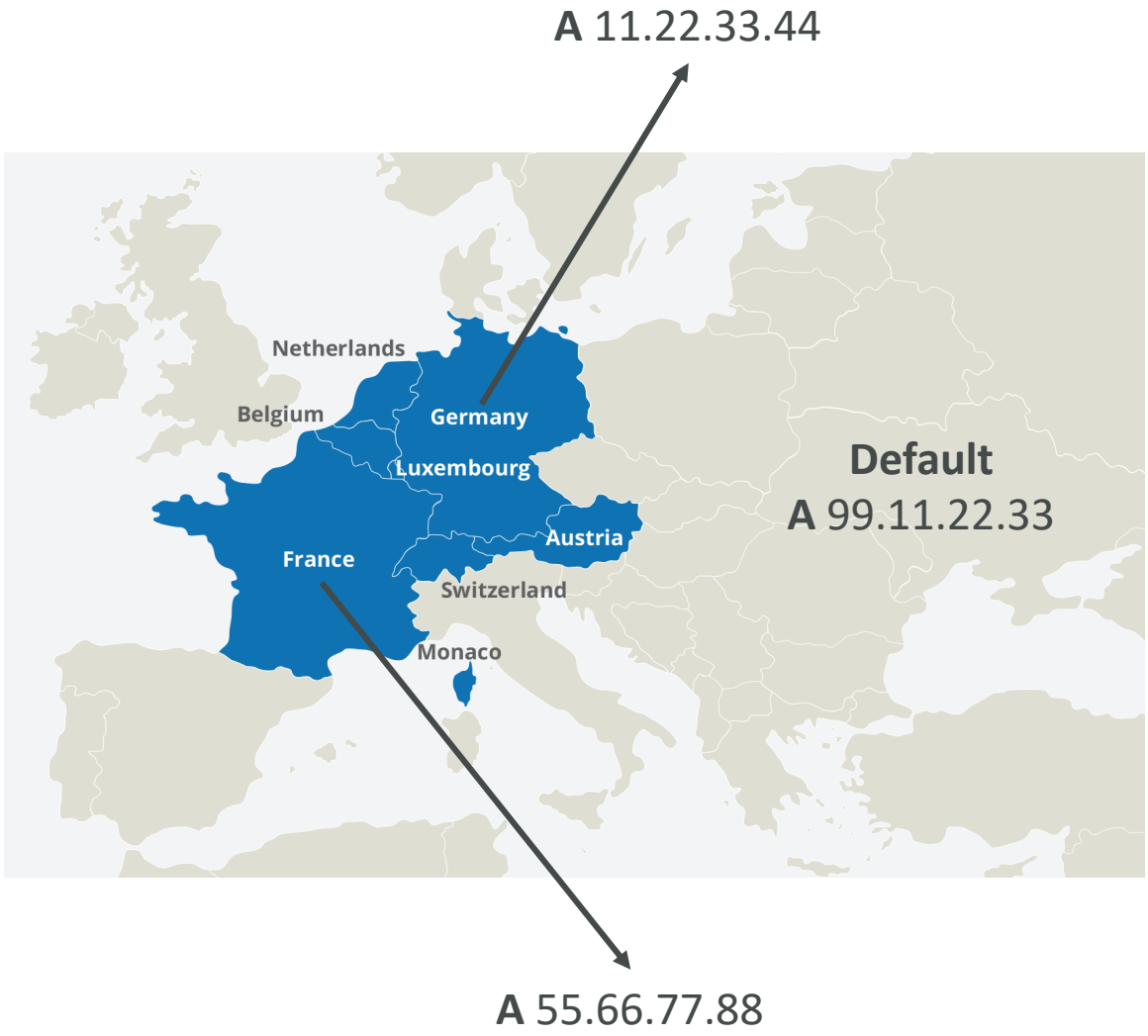


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Routing Policies – Geolocation

- Different from Latency-based!
- **This routing is based on user location**
- Specify location by Continent, Country or by US State (if there's overlapping, most precise location selected)
- Should create a **"Default"** record (in case there's no match on location)
- Use cases: website localization, restrict content distribution, load balancing, …
- Can be associated with Health Checks



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Routing Policies – Geoproximity

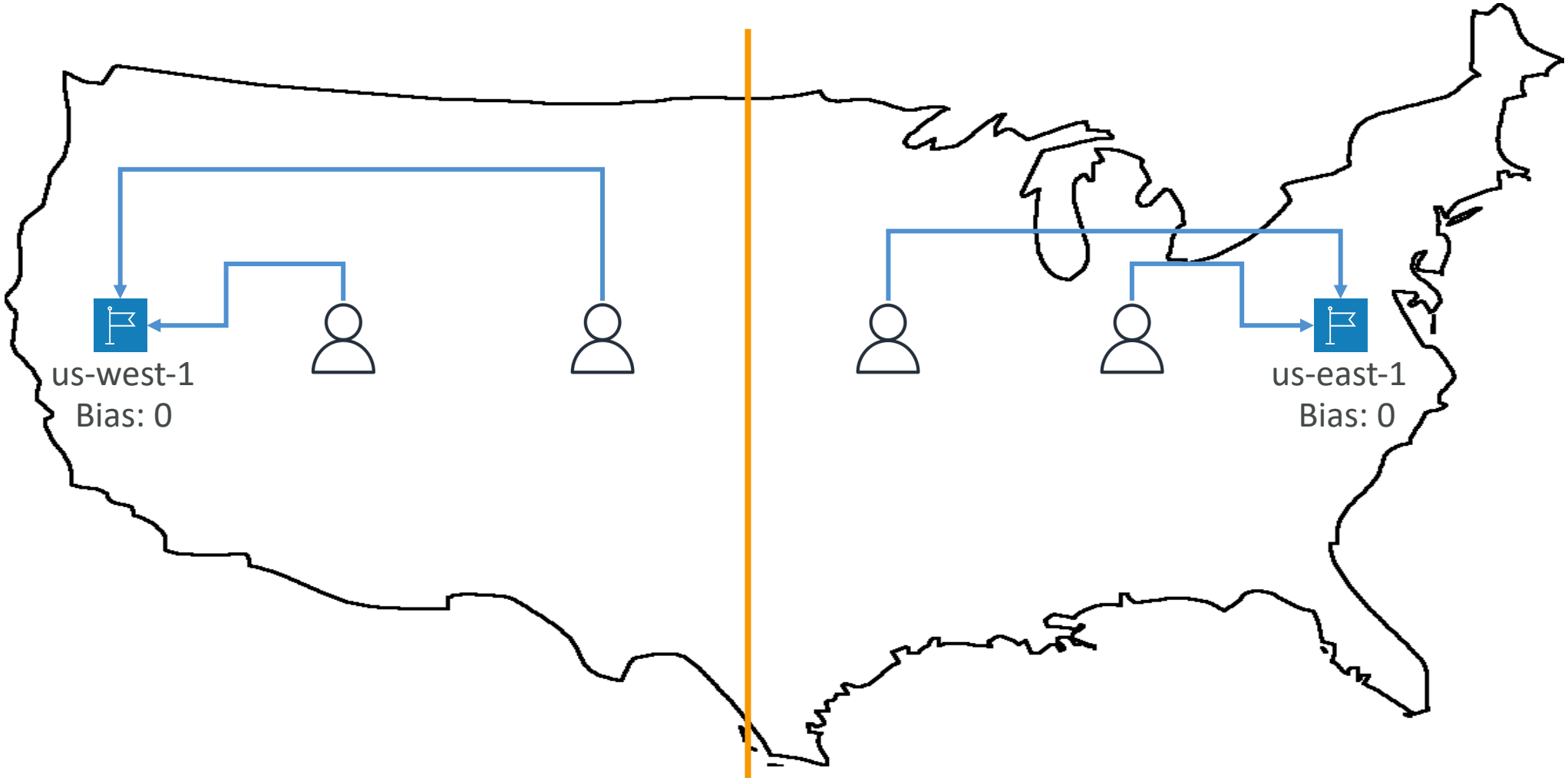
- Route traffic to your resources based on the geographic location of users and resources
- Ability **to shift more traffic to resources based** on the defined **bias**
- To change the size of the geographic region, specify **bias** values:
  - To expand (1 to 99) – more traffic to the resource
  - To shrink (-1 to -99) – less traffic to the resource

- Resources can be:
  - AWS resources (specify AWS region)
  - Non-AWS resources (specify Latitude and Longitude)
- You must use Route 53 **Traffic Flow** to use this feature

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

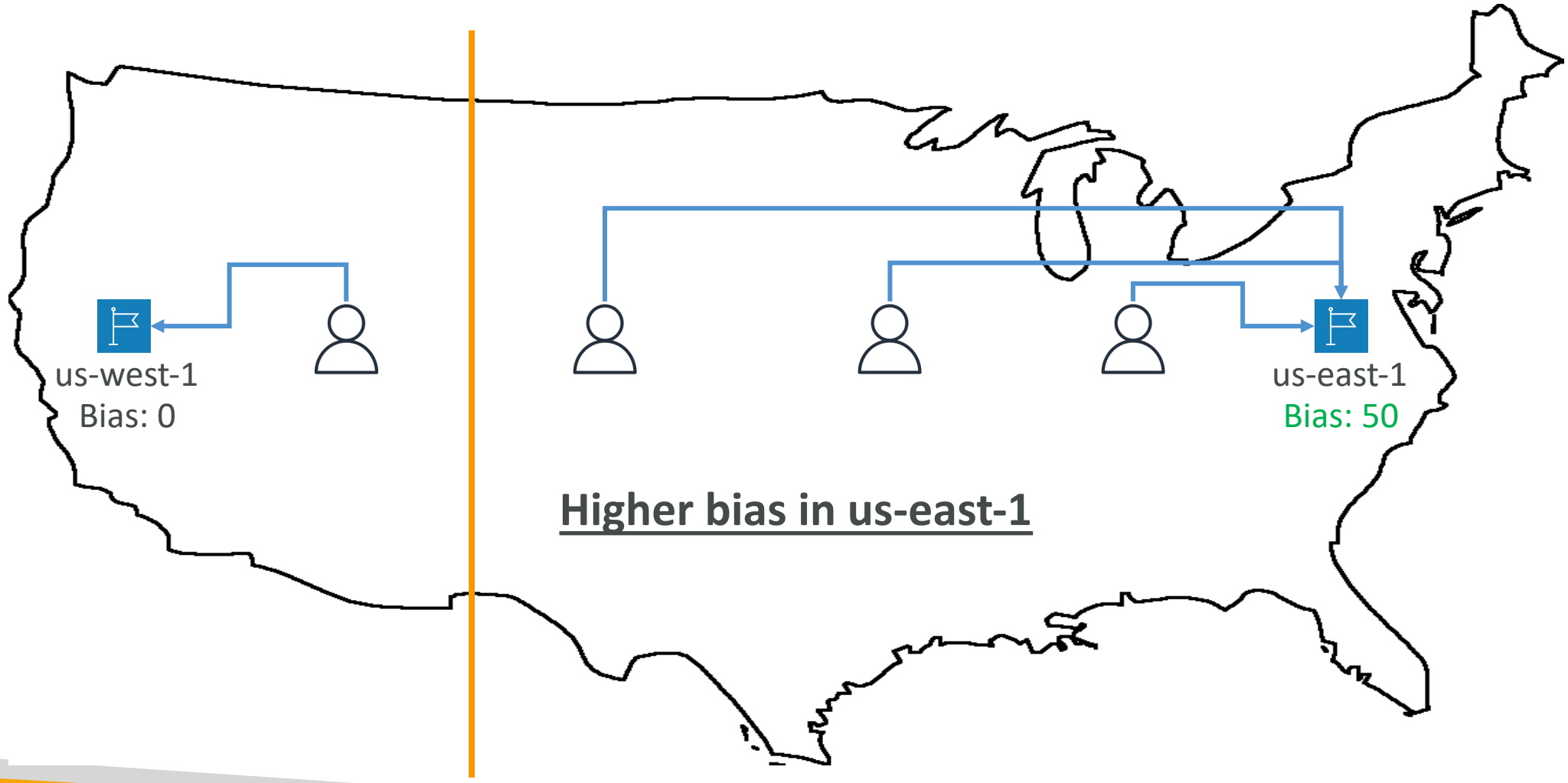
Routing Policies – Geoproximity



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Routing Policies – Geoproximity

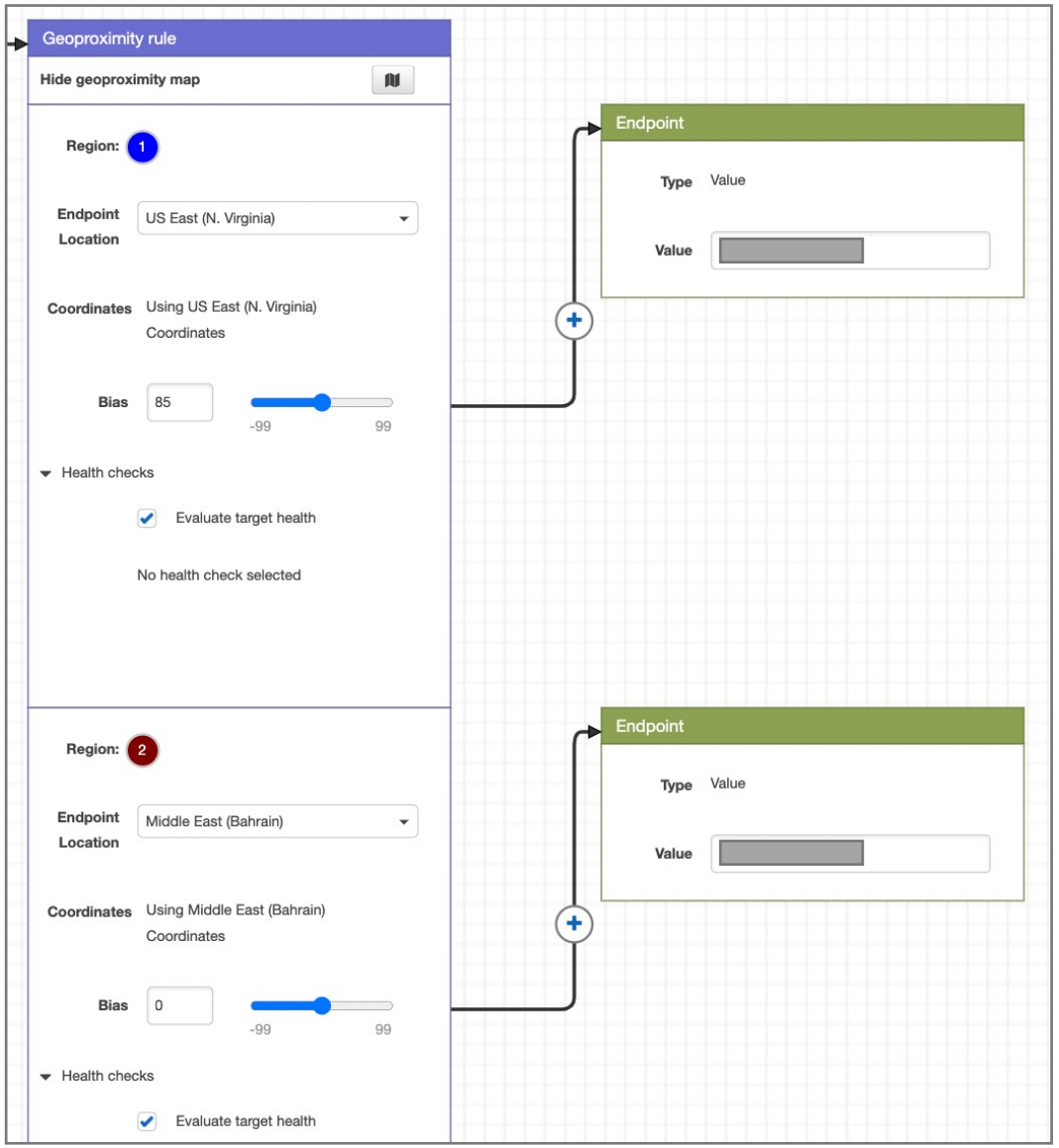


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Route 53 – Traffic flow

- Simplify the process of creating and maintaining records in large and complex configurations
- Visual editor to manage complex routing decision trees
- Configurations can be saved as **Traffic Flow Policy**
  - Can be applied to different Route 53 Hosted Zones (different domain names)
  - Supports versioning



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Routing Policies – Multi-Value

- Use when routing traffic to multiple resources
- Route 53 return multiple values/resources
- Can be associated with Health Checks (return only values for healthy resources)
- Up to 8 healthy records are returned for each Multi-Value query
- Multi-Value is not a substitute for having an ELB

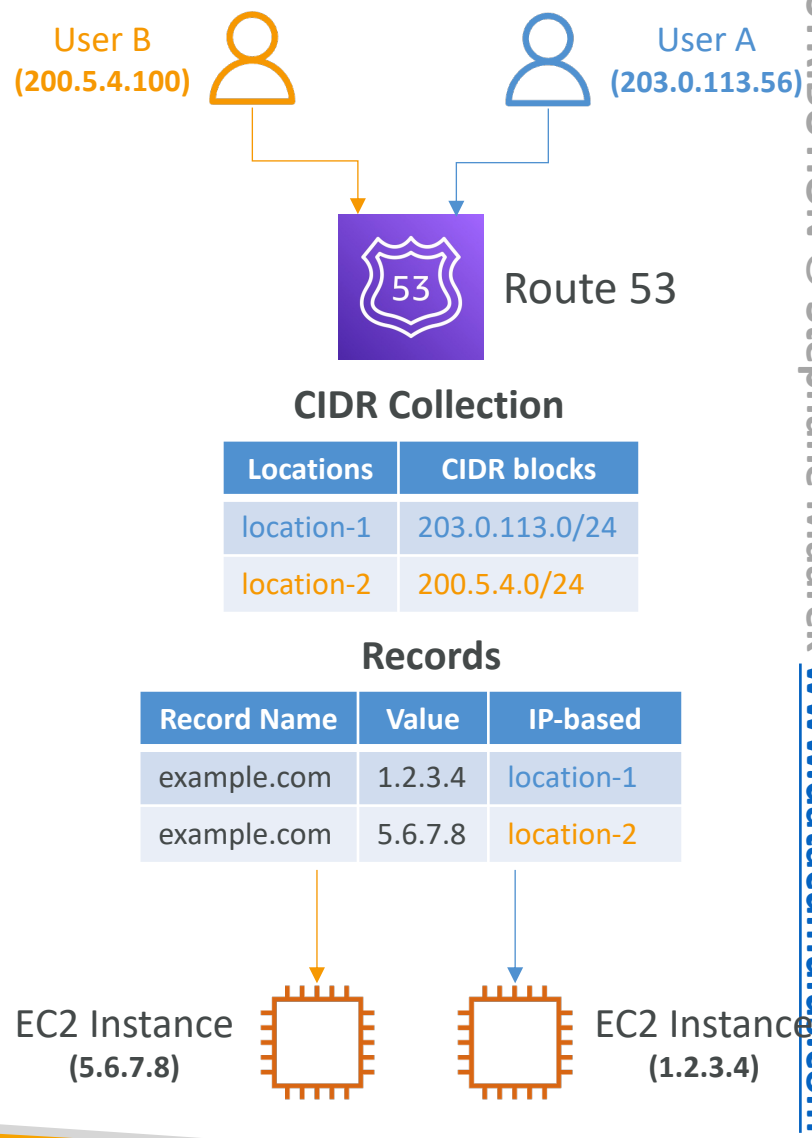
Name	Type	Value	TTL	Set ID	Health Check
www.example.com	A Record	192.0.2.2	60	Web1	A
www.example.com	A Record	198.51.100.2	60	Web2	B
www.example.com	A Record	203.0.113.2	60	Web3	C

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Routing Policies – IP-based Routing

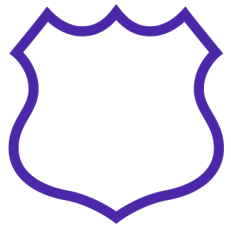
- Routing is based on clients' IP addresses
- You provide a list of CIDRs for your clients and the corresponding endpoints/locations (user-IP-to-endpoint mappings)
- Use cases: Optimize performance, reduce network costs…
- Example: route end users from a particular ISP to a specific endpoint



NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© Stephane Maarek

Route 53 – Hosted Zones



- A container for records that define how to route traffic to a domain and its subdomains

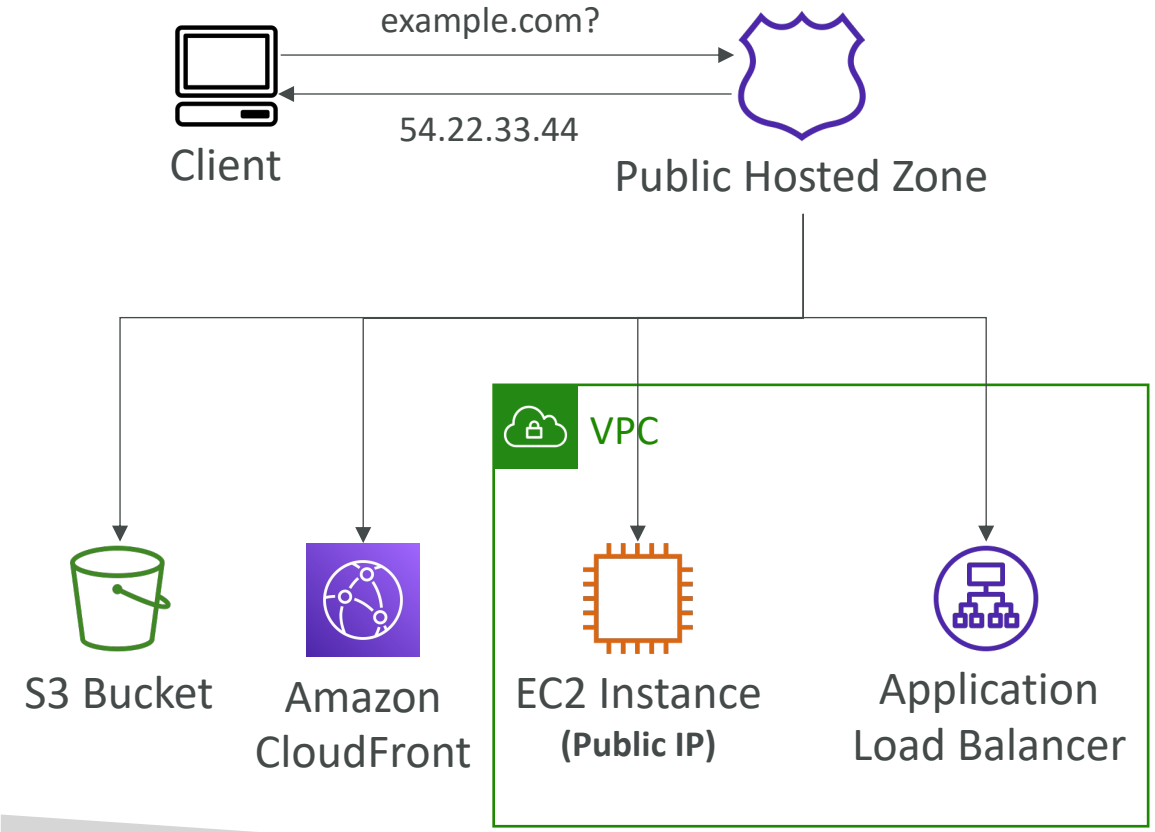
- **Public Hosted Zones** – contains records that specify how to route traffic on the Internet (public domain names)
  application1.mypublicdomain.com
- **Private Hosted Zones** – contain records that specify how you route traffic within one or more VPCs (private domain names)
  application1.company.internal

© Stephane Maarek

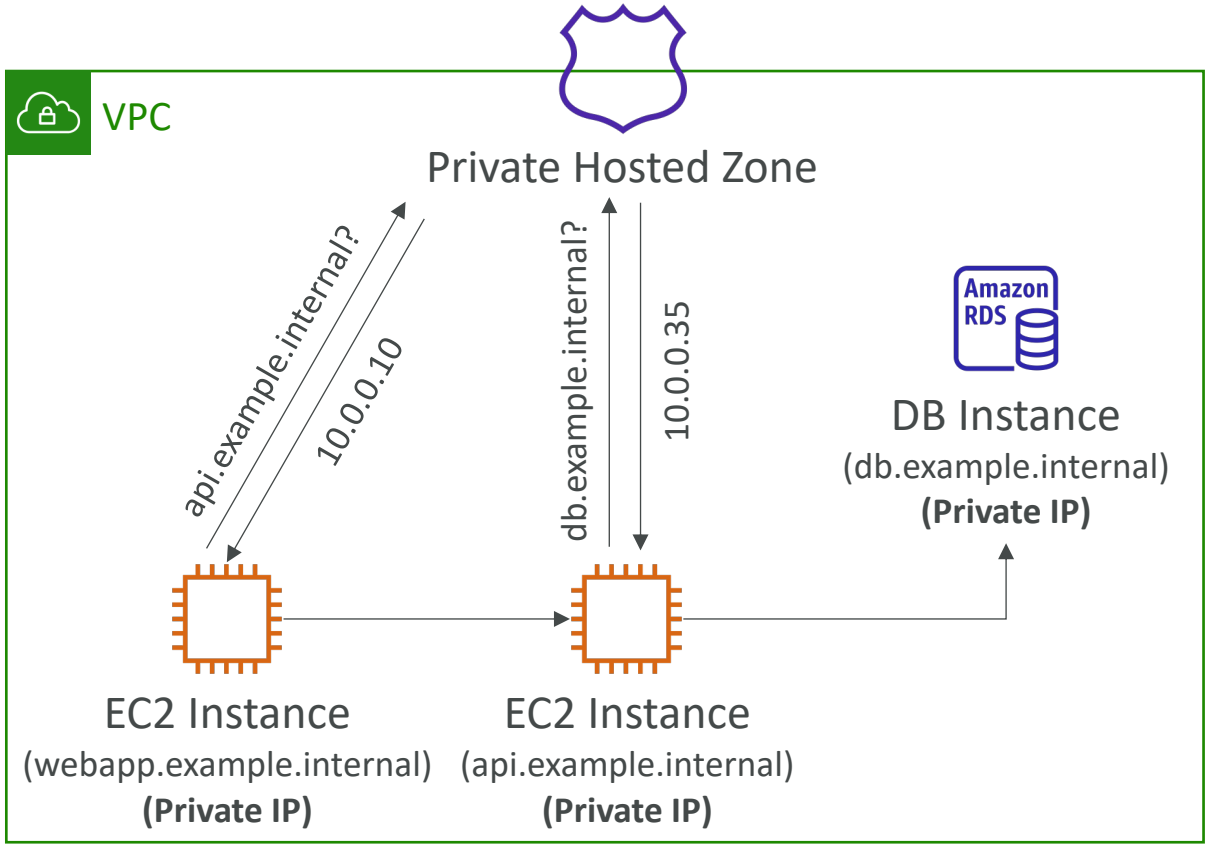
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Route 53 – Public vs. Private Hosted Zones

Public Hosted Zone



Private Hosted Zone



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Route 53 – Good to Know

- For internal private DNS (Private Hosted Zone), you must enable the VPC settings **enableDnsHostnames** and **enableDnsSupport**

- **DNS Security Extensions (DNSSEC)**
  - A protocol for securing DNS traffic, verifies DNS data integrity and origin
  - Protects against Man in the Middle (MITM) attacks
  - Route 53 supports both DNSSEC for Domain Registeration and DNSSEC Signing
  - Works only with **Public Hosted Zones**

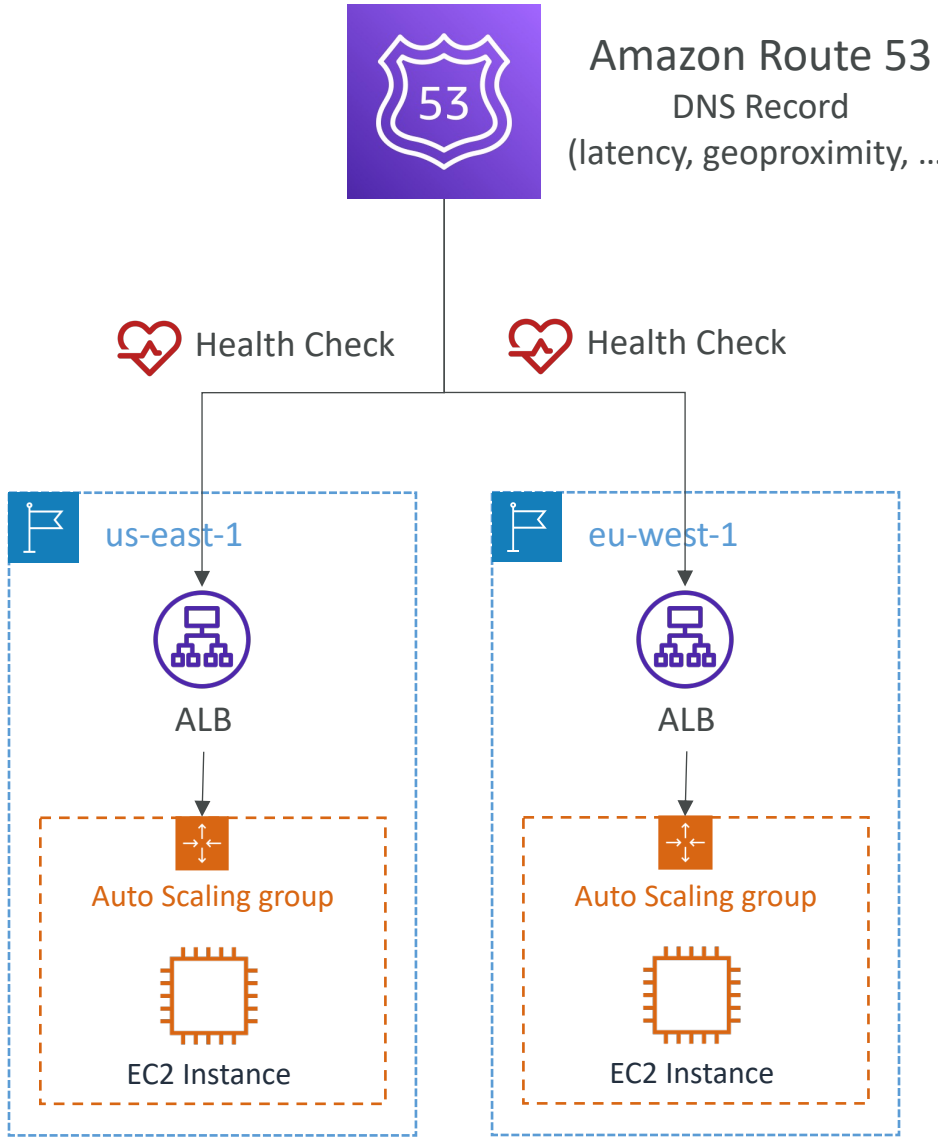
- **Route 53 with 3rd Registrar**
  - You can buy the domain out of AWS and use Route 53 as the DNS provider
  - Update the NS records on the 3rd party Registrar

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Route 53 – Health Checks

- HTTP Health Checks are only for **public resources**
- Health Check => Automated DNS Failover:
  1. Health checks that monitor an endpoint (application, server, other AWS resource)
  2. Health checks that monitor other health checks (Calculated Health Checks)
  3. Health checks that monitor CloudWatch Alarms (full control !!) – e.g., throttles of DynamoDB, alarms on RDS, custom metrics, … (helpful for private resources)
- Health Checks are integrated with CW metrics

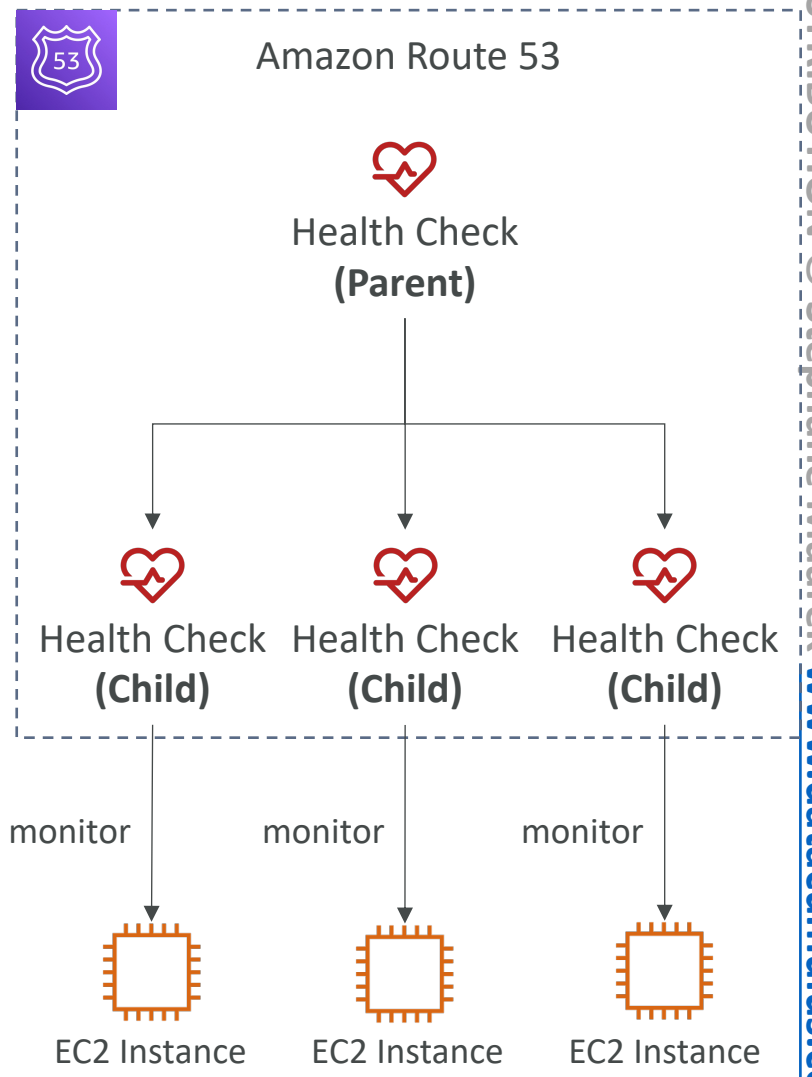


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Route 53 – Calculated Health Checks

- Combine the results of multiple Health Checks into a single Health Check
- You can use **OR**, **AND**, or **NOT**
- Can monitor up to 256 Child Health Checks
- Specify how many of the health checks need to pass to make the parent pass
- Usage: perform maintenance to your website without causing all health checks to fail

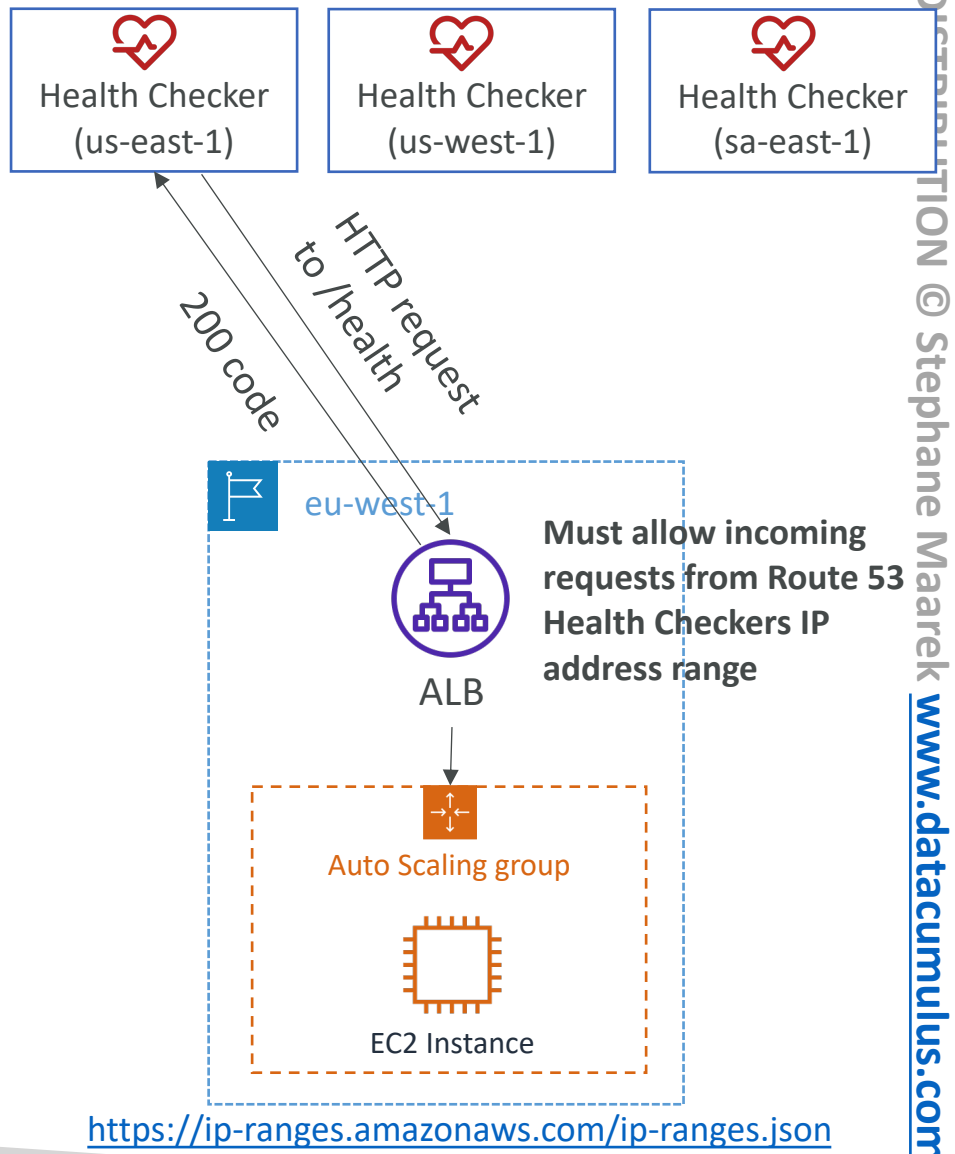


NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© Stephane Maarek

Health Checks – Monitor an Endpoint

- About 15 global health checkers will check the endpoint health
- Health Checks pass only when the endpoint responds with the 2xx and 3xx status codes
- Health Checks can be setup to pass / fail based on the text in the first 5120 bytes of the response

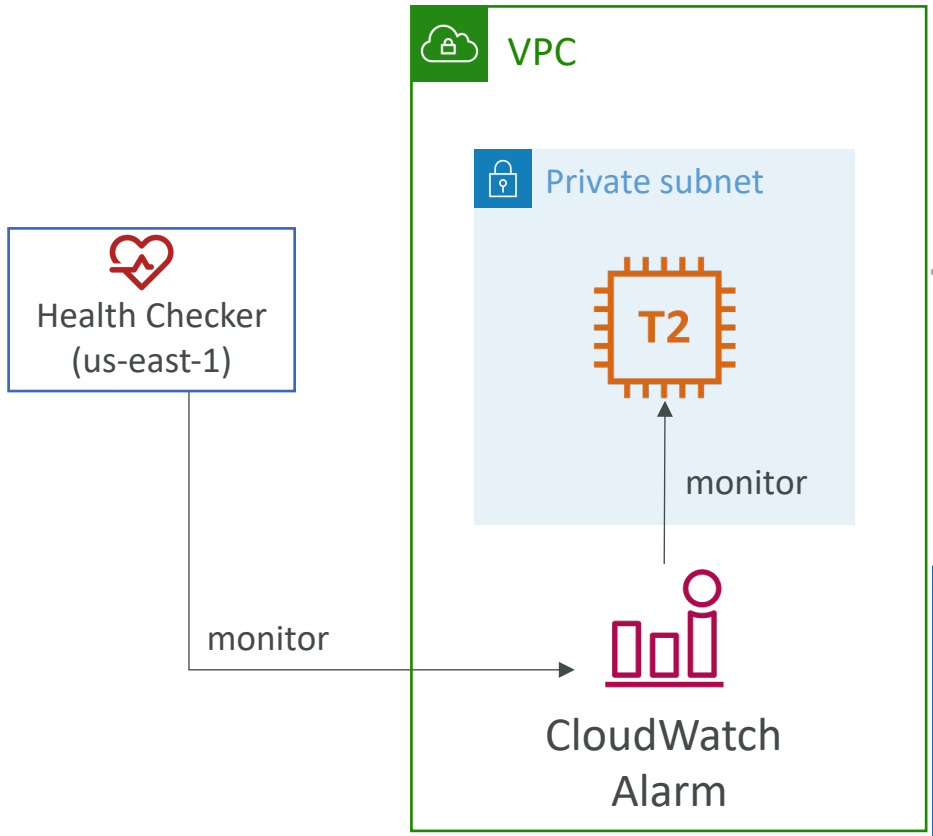


NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© Stephane Maarek

Health Checks – Private Hosted Zones

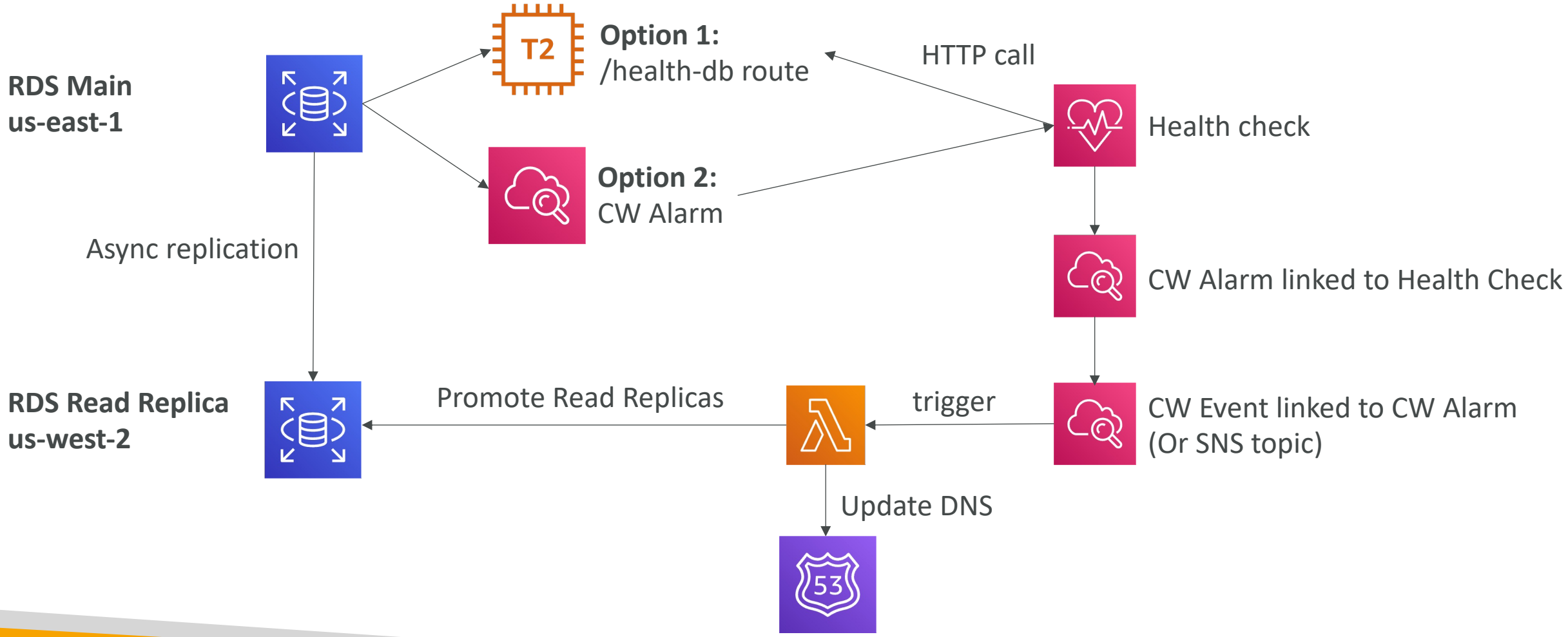
- Route 53 health checkers are outside the VPC
- They can't access **private** endpoints (private VPC or on-premises resource)
- You can create a **CloudWatch Metric** and associate a **CloudWatch Alarm**, then create a Health Check that checks the alarm itself



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Health Checks Solution Architecture
RDS multi-region failover

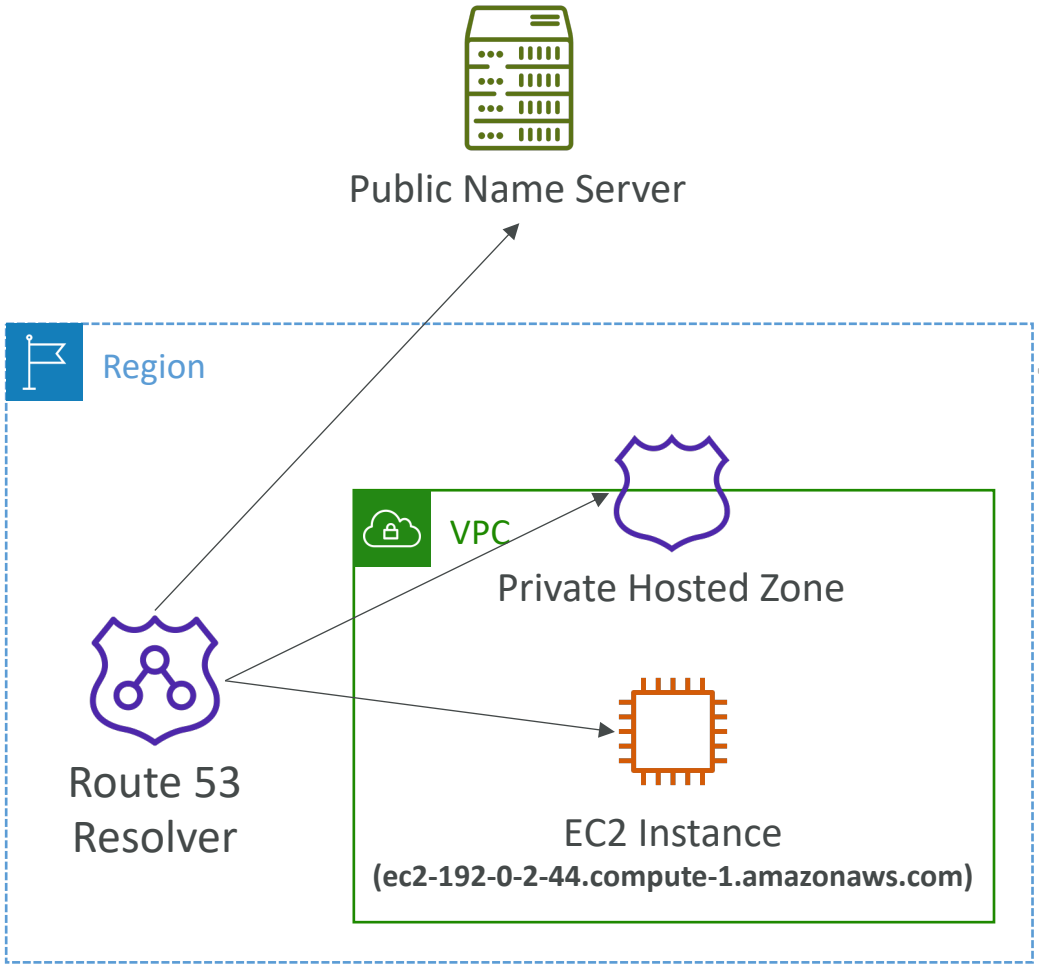


NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

© Stephane Maarek

Route 53 – Hybrid DNS

- By default, Route 53 Resolver automatically answers DNS queries for:
  - Local domain names for EC2 instances
  - Records in Private Hosted Zones
  - Records in public Name Servers
- **Hybrid DNS** – resolving DNS queries between VPC (Route 53 Resolver) and your networks (other DNS Resolvers)
- Networks can be:
  - VPC itself / Peered VPC
  - On-premises Network (connected through Direct Connect or AWS VPN)



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

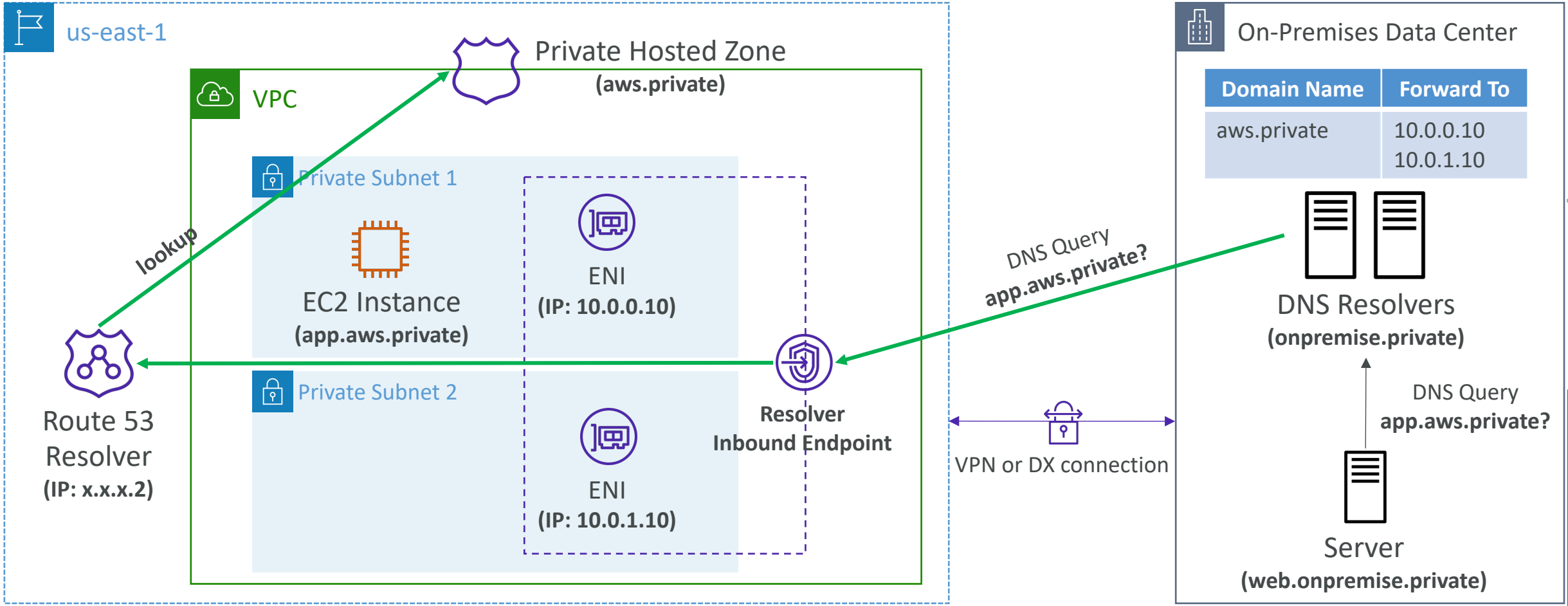
Route 53 – Resolver Endpoints

- Inbound Endpoint
  - DNS Resolvers on your network can forward DNS queries to Route 53 Resolver
  - Allows your DNS Resolvers to resolve domain names for AWS resources (e.g., EC2 instances) and records in Route 53 Private Hosted Zones
- Outbound Endpoint
  - Route 53 Resolver conditionally forwards DNS queries to your DNS Resolvers
  - Use **Resolver Rules** to forward DNS queries to your DNS Resolvers
- Associated with one or more VPCs in the same AWS Region
- Create in two AZs for high availability
- Each Endpoint supports 10,000 queries per second per IP address

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

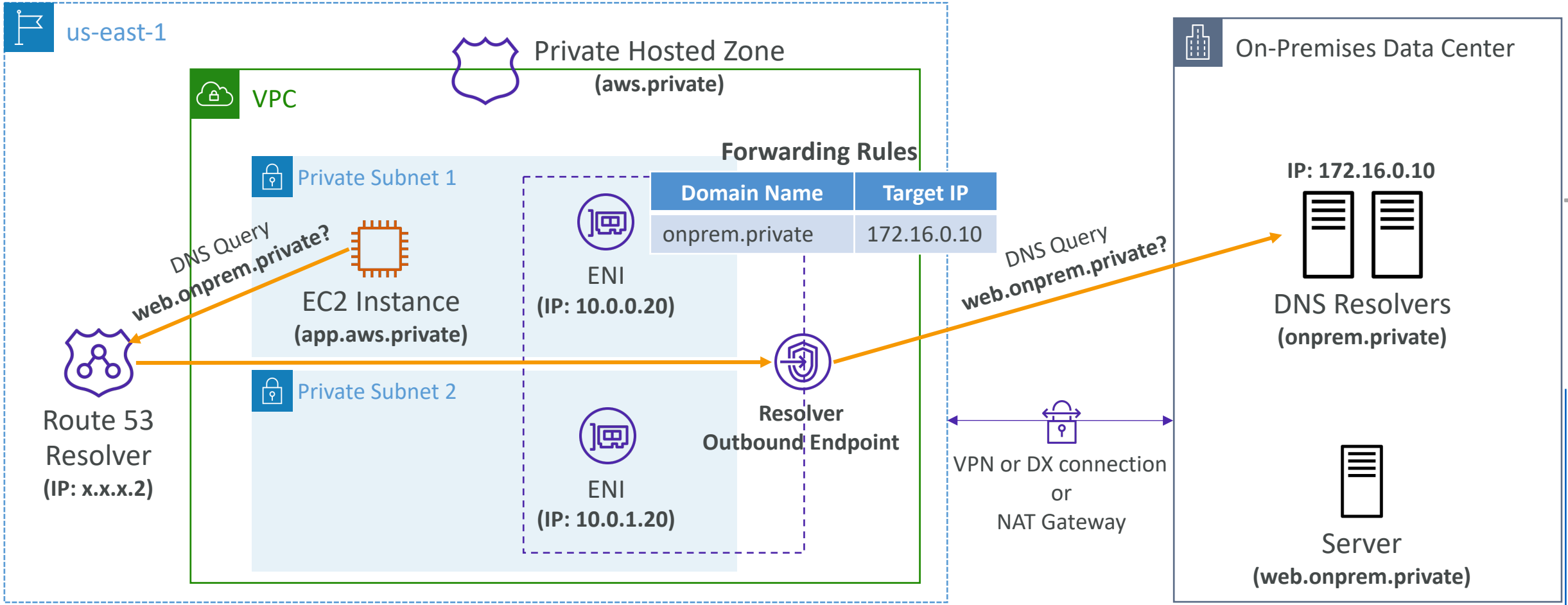
Route 53 – Resolver Inbound Endpoints



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Route 53 – Resolver Outbound Endpoints

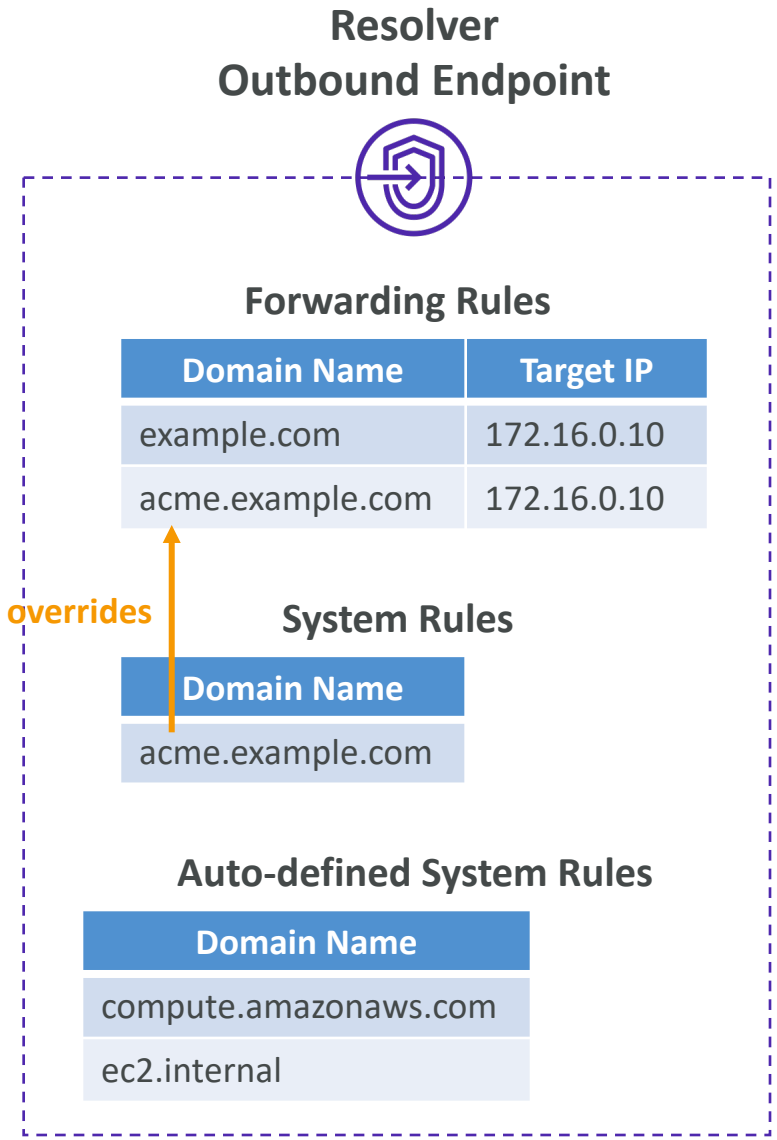


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Route 53 – Resolver Rules

- Control which DNS queries are forwarded to DNS Resolvers on your network
- Conditional Forwarding Rules (Forwarding Rules)
  - Forward DNS queries for a specified domain and all its subdomains **to target IP addresses**
- System Rules
  - Selectively overriding the behavior defined in Forwarding Rules (e.g., don't forward DNS queries for a subdomain acme.example.com)
- Auto-defined System Rules
  - Defines how DNS queries for selected domains are resolved (e.g., AWS internal domain names, Privated Hosted Zones)
- If multiple rules matched, Route 53 Resolver chooses the most specific match
- Resolver Rules can be shared across accounts using AWS RAM
  - Manage them centrally in one account
  - Send DNS queries from multiple VPC to the target IP defined in the rule

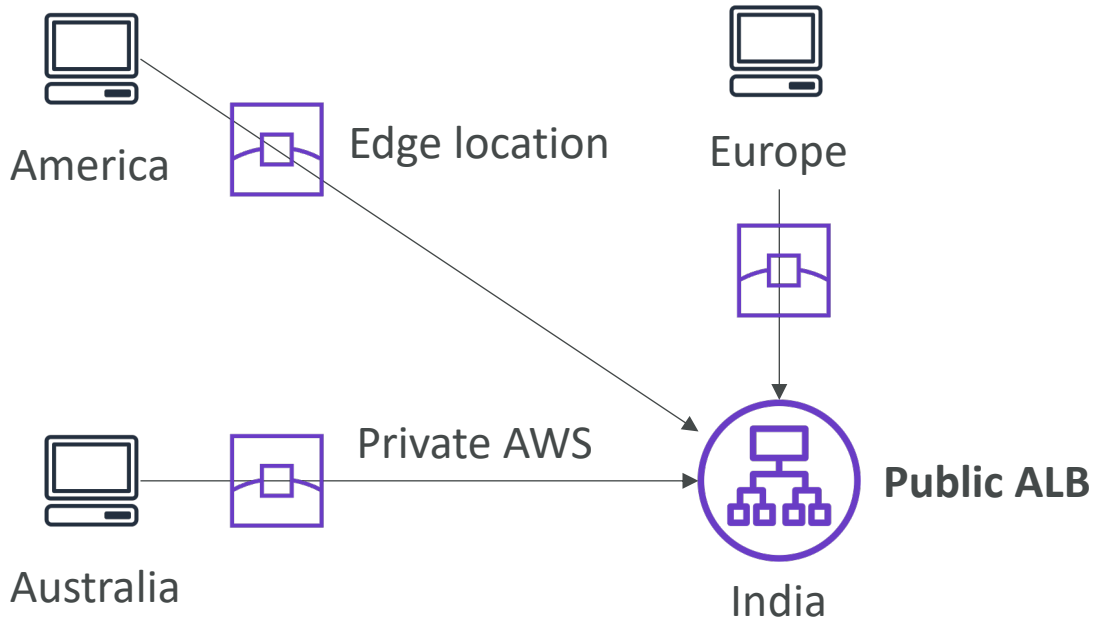


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AWS Global Accelerator

- Leverage the AWS internal network to route to your application
- 2 Anycast IP are created for your application
- The Anycast IP send traffic directly to Edge Locations
- The Edge locations send the traffic to your application



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AWS Global Accelerator

- Works with Elastic IP, EC2 instances, ALB, NLB, public or private
- Supports Client IP Address Preservation **except EIPs endpoints**
- Consistent Performance
  - Intelligent routing to lowest latency and fast regional failover
  - No issue with client cache (because the IP doesn't change)
  - Internal AWS network
- Health Checks
  - Global Accelerator performs a health check of your applications
  - Helps make your application global (failover less than 1 minute for unhealthy)
  - Great for disaster recovery (thanks to the health checks)
- Security
  - only 2 external IP need to be whitelisted
  - DDoS protection thanks to AWS Shield

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AWS Global Accelerator vs CloudFront

- They both use the AWS global network and its edge locations around the world
- Both services integrate with AWS Shield for DDoS protection.

- CloudFront
  - Improves performance for both cacheable content (such as images and videos)
  - Dynamic content (such as API acceleration and dynamic site delivery)
  - Content is served at the edge

- Global Accelerator
  - Improves performance for a wide range of applications over TCP or UDP
  - Proxying packets at the edge to applications running in one or more AWS Regions.
  - Good fit for non-HTTP use cases, such as gaming (UDP), IoT (MQTT), or Voice over IP
  - Good for HTTP use cases that require static IP addresses
  - Good for HTTP use cases that required deterministic, fast regional failover

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

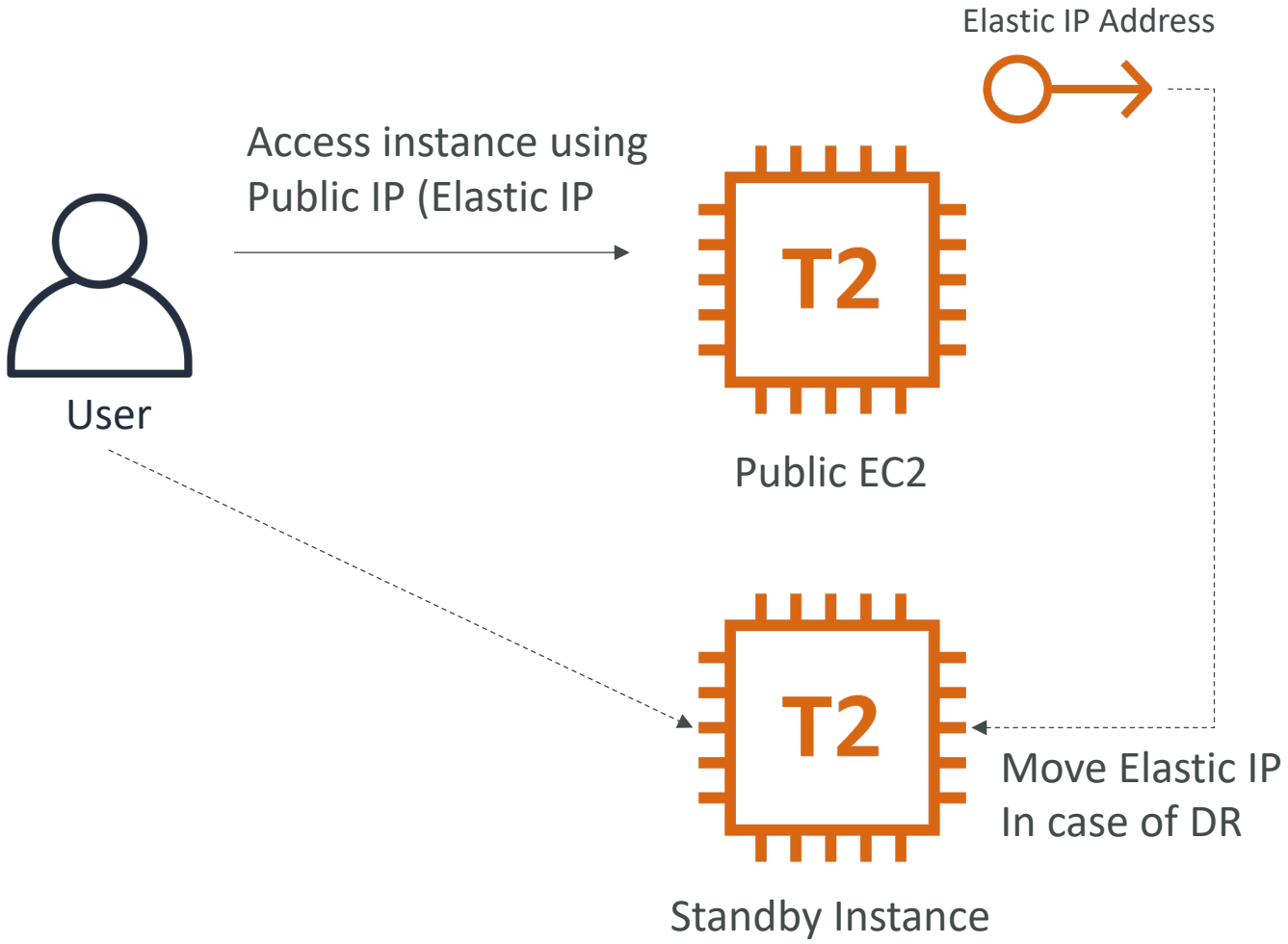
Solution Architecture Comparisons

- EC2 on its own with Elastic IP
- EC2 with Route53
- ALB + ASG
- ALB + ECS on EC2
- ALB + ECS on Fargate
- ALB + Lambda
- API Gateway + Lambda
- API Gateway + AWS Service
- API Gateway + HTTP backend (ex: ALB)

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

EC2 with Elastic IP

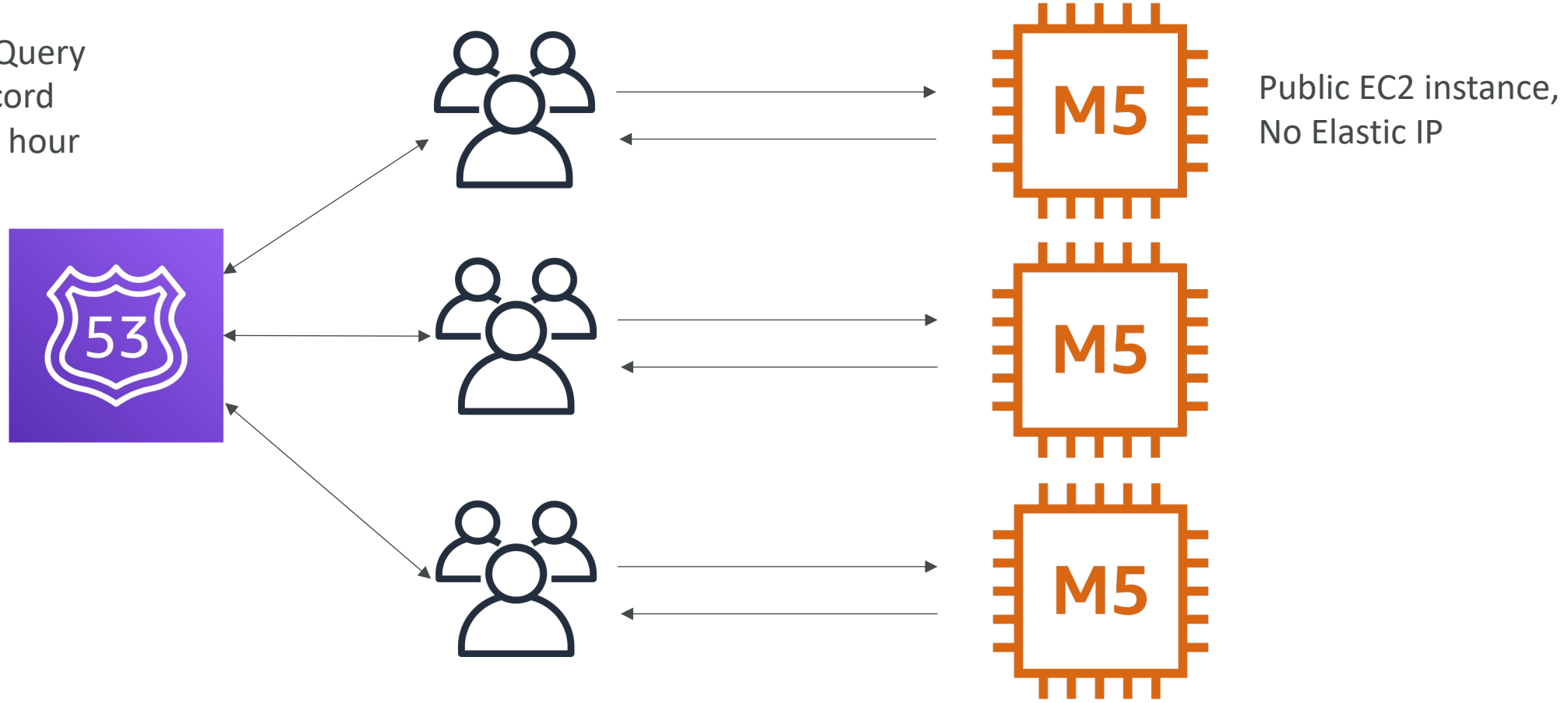


- Quick failover
- The client should not see the change happen
- Helpful if the client needs to resolve by static Public IP address
- Does not scale
- Cheap

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

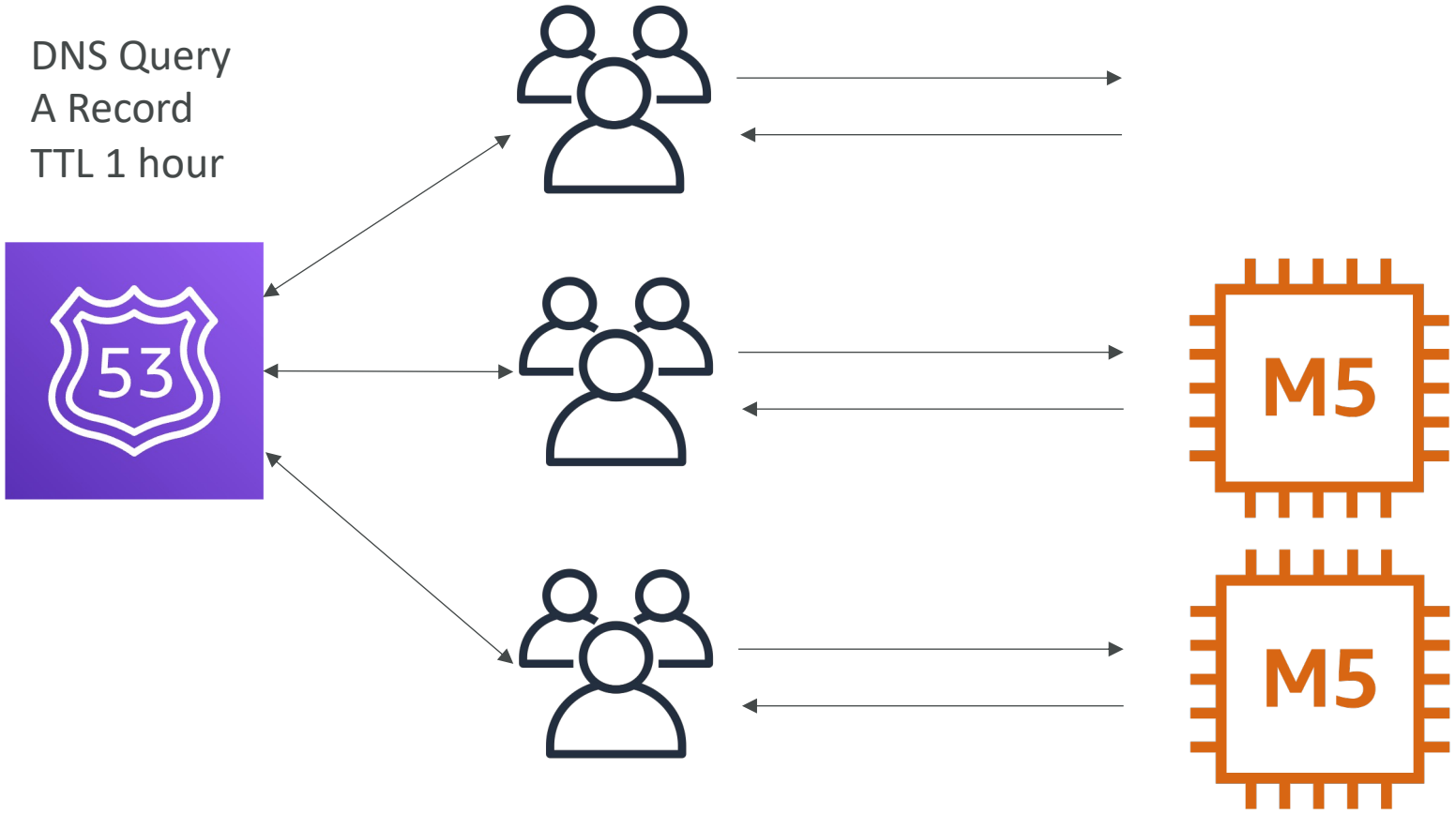
Stateless web app - scaling horizontally



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

Stateless web app - scaling horizontally

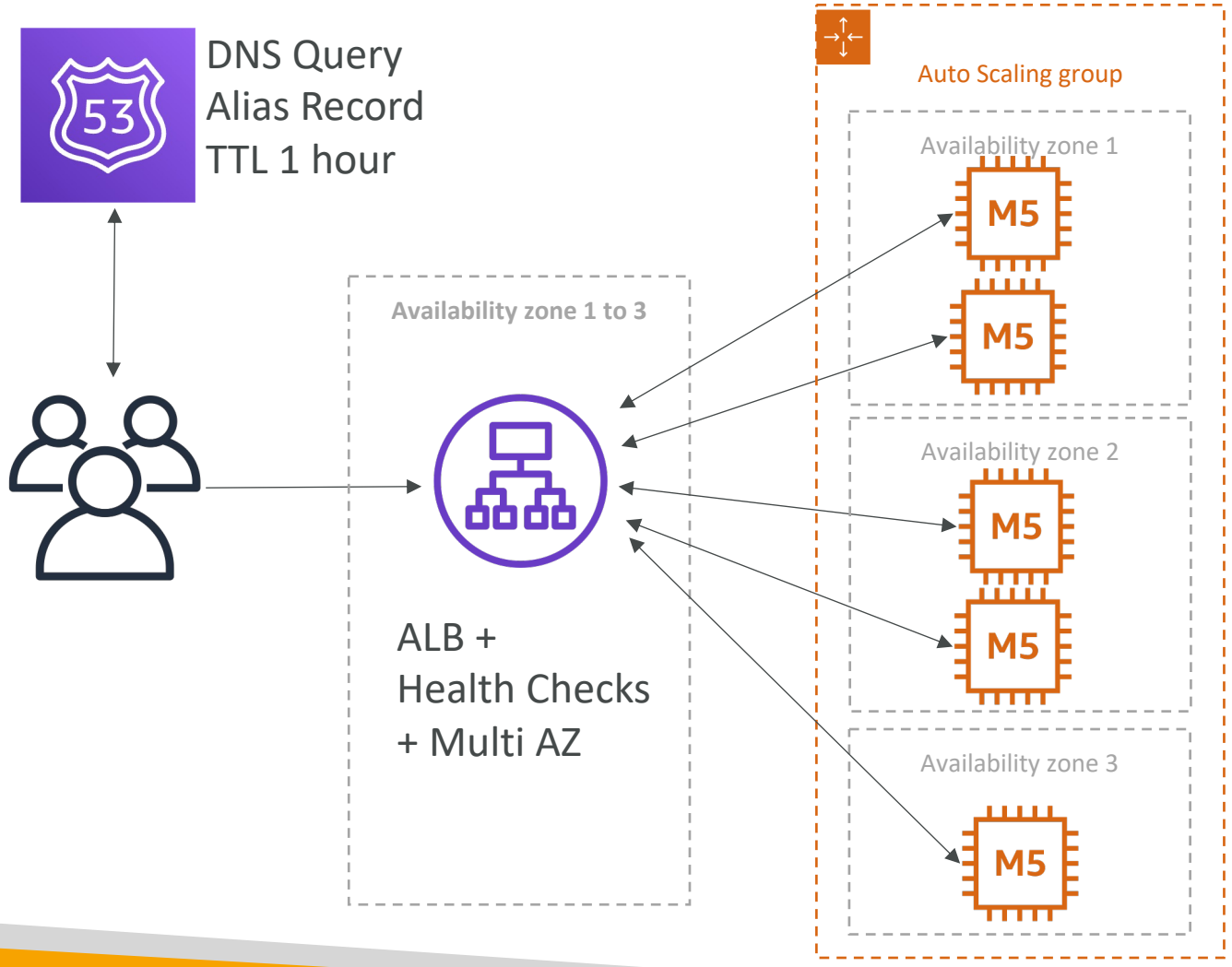


- "DNS-based load balancing"
- Ability to use multiple instances
- Route53 TTL implies client may get outdated information
- Clients must have logic to deal with hostname resolution failures
- Adding an instance may not receive full traffic right away due to DNS TTL

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

ALB + ASG



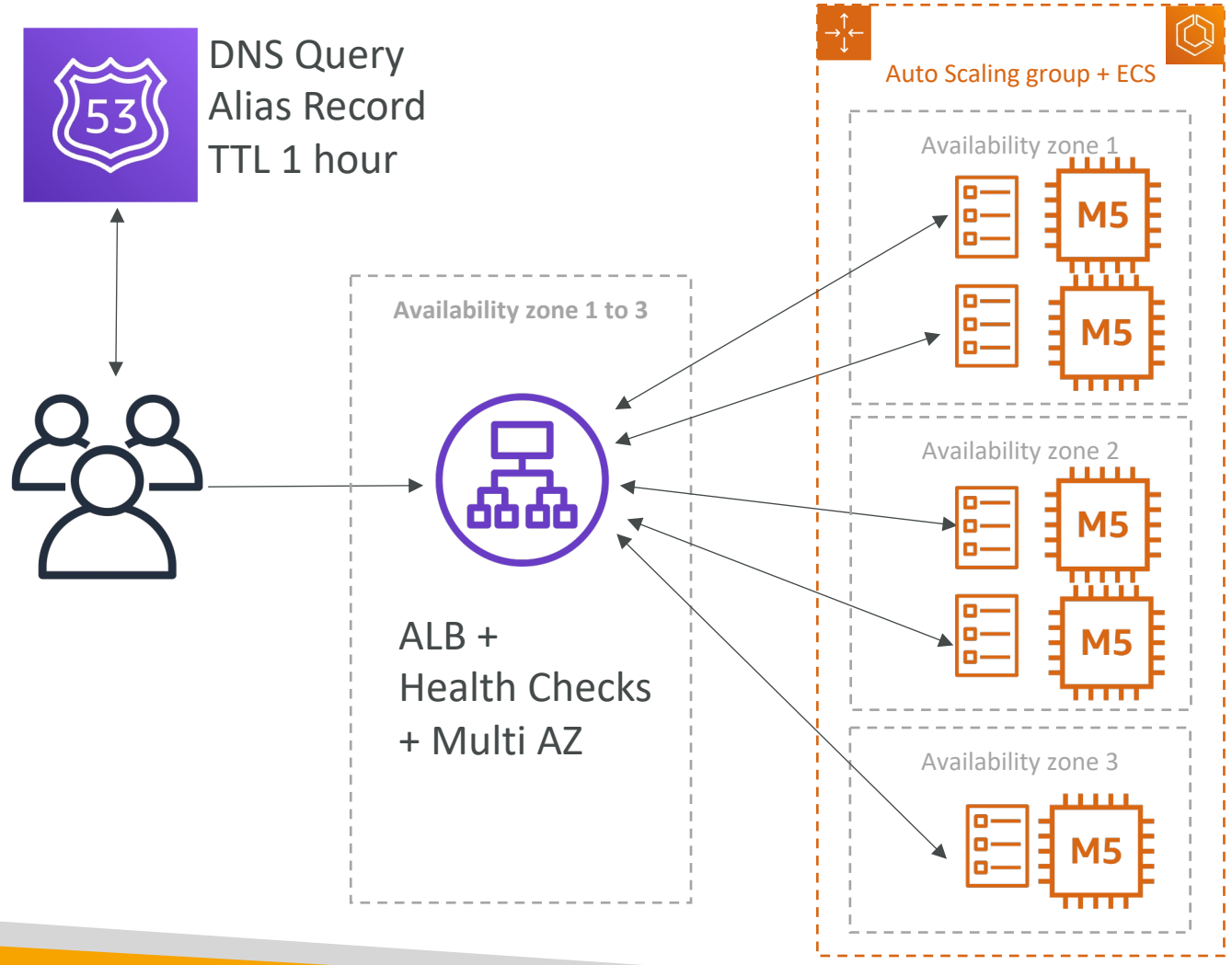
- Scales well, classic architecture
- New instances are in service right away.
- Users are not sent to instances that are out-of-service
- Time to scale is slow (EC2 instance startup + bootstrap) – AMI can help
- ALB is elastic but can't handle sudden, huge peak of demand (pre-warm)
- Could lose a few requests if instances are overloaded
- CloudWatch used for scaling
- Cross-Zone balancing for even traffic distribution

- Target utilization should be between 40% and 70%

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

ALB + ECS on EC2 (backed by ASG)

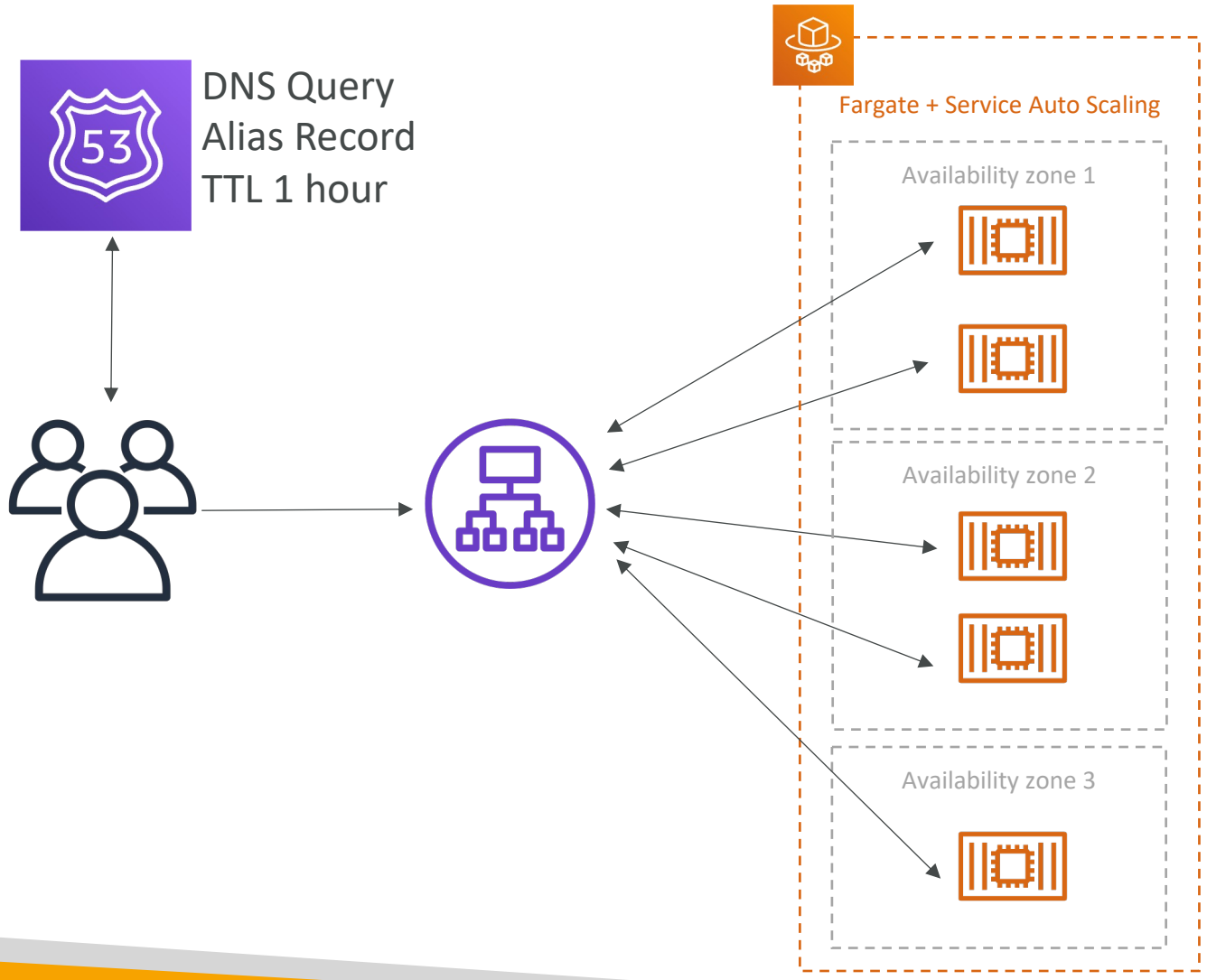


- Same properties as ALB + ASG
- Application is run on Docker
- ASG + ECS allows to have **dynamic port mappings**
- Tough to orchestrate ECS service auto-scaling + ASG auto-scaling

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

ALB + ECS on Fargate

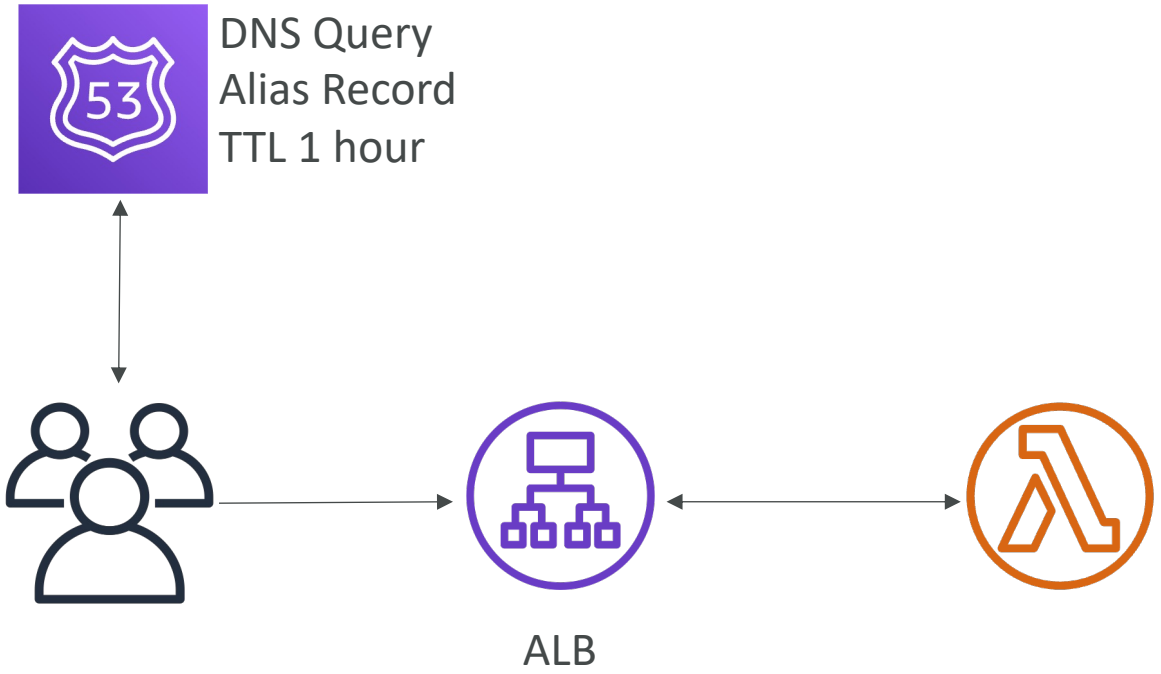


- Application is run on Docker
- Service Auto Scaling is easy
- Time to be in-service is quick (no need to launch an EC2 instance in advance)
- Still limited by the ALB in case of sudden peaks
- "serverless" application tier
- "managed" load balancer

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

ALB + Lambda

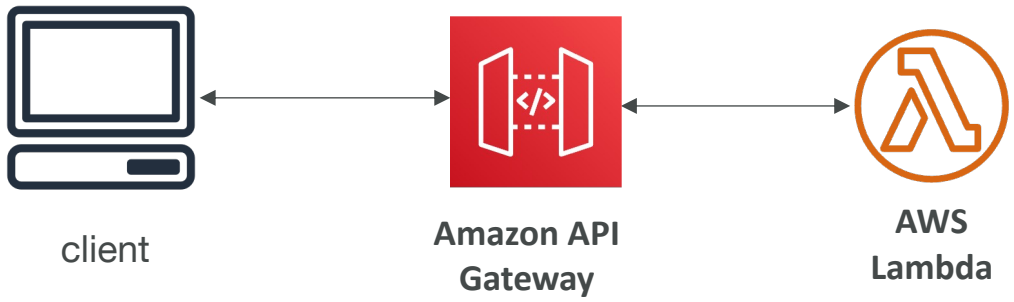


- Limited to Lambda's runtimes
- Seamless scaling thanks to Lambda
- Simple way to expose Lambda functions as HTTP/S without all the features from API Gateway
- Can combine with WAF (Web Application Firewall)
- Good for hybrid microservices
- Example: use ECS for some requests, use Lambda for others

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway + Lambda



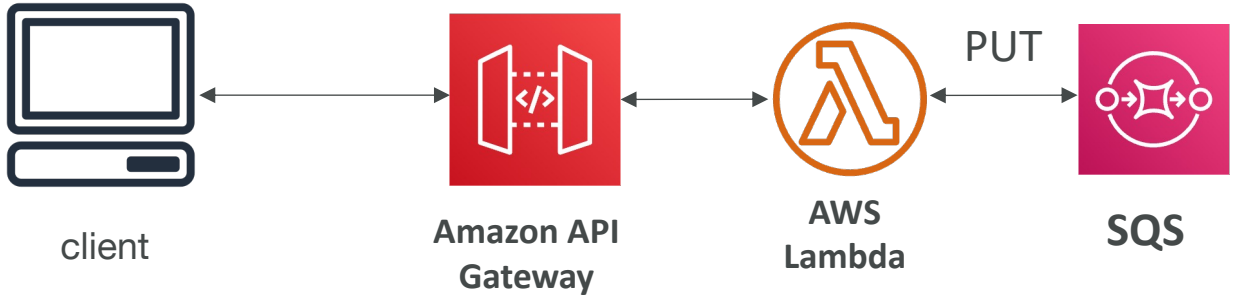
- Pay per request, seamless scaling, fully serverless
- Soft limits: 10000/s API Gateway, 1000 concurrent Lambda
- API Gateway features: authentication, rate limiting, caching, etc…
- Lambda Cold Start time may increase latency for some requests
- Fully integrated with X-Ray

© Stephane Maarek

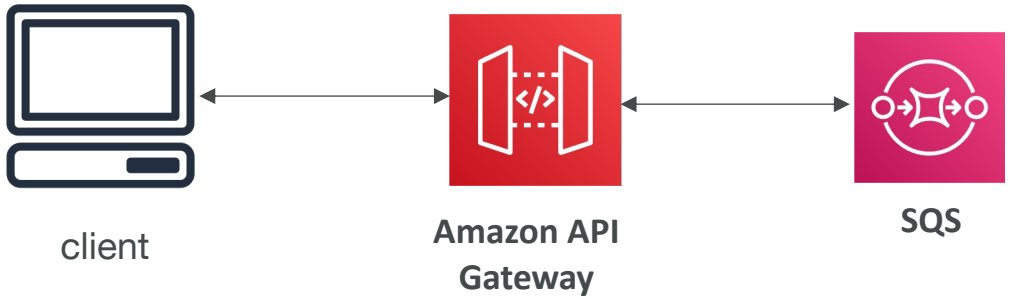
NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway + AWS Service (as a proxy)

OK



BETTER

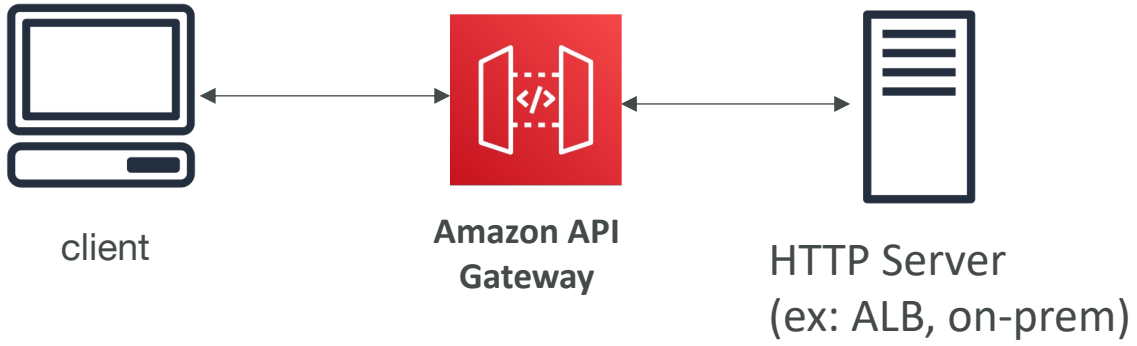


- Lower latency, cheaper
- Not using Lambda concurrent capacity, no custom code
- Expose AWS APIs securely through API Gateway
- SQS, SNS, Step Functions…
- Remember API Gateway has a payload limit of 10 MB (can be a problem for S3 proxy)

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

API Gateway + HTTP backend (ex: ALB)



- Use API Gateway features on top of custom HTTP backend (authentication, rate control, API keys, caching…)

- Can connect to…
- on-premises service
- Application Load Balancer
- 3rd party HTTP service

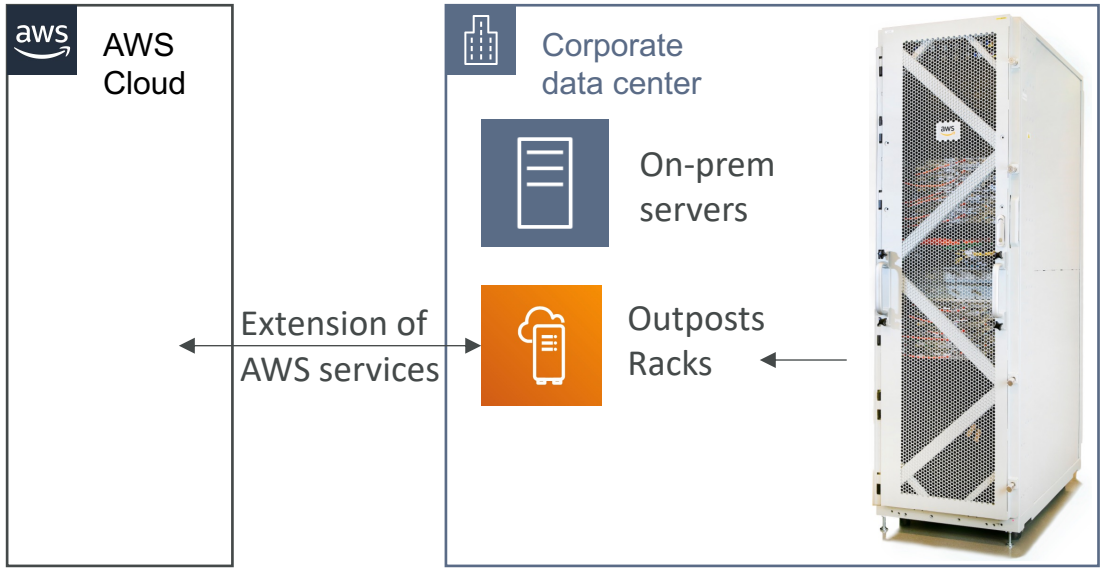
© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AWS Outposts



- **Hybrid Cloud**: businesses that keep an on-premises infrastructure alongside a cloud infrastructure
- Therefore, two ways of dealing with IT systems:
  - One for the AWS cloud (using the AWS console, CLI, and AWS APIs)
  - One for their on-premises infrastructure
- **AWS Outposts are "server racks"** that offers the same AWS infrastructure, services, APIs & tools to build your own applications on-premises just as in the cloud
- **AWS will setup and manage "Outposts Racks"** within your on-premises infrastructure and you can start leveraging AWS services on-premises
- You are responsible for the Outposts Rack physical security



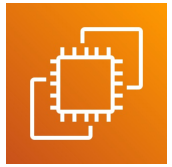
© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AWS Outposts



- Benefits:
  - Low-latency access to on-premises systems
  - Local data processing
  - Data residency
  - Easier migration from on-premises to the cloud
  - Fully managed service
- Some services that work on Outposts:


Amazon EC2


Amazon EBS


Amazon S3


Amazon EKS


Amazon ECS


Amazon RDS

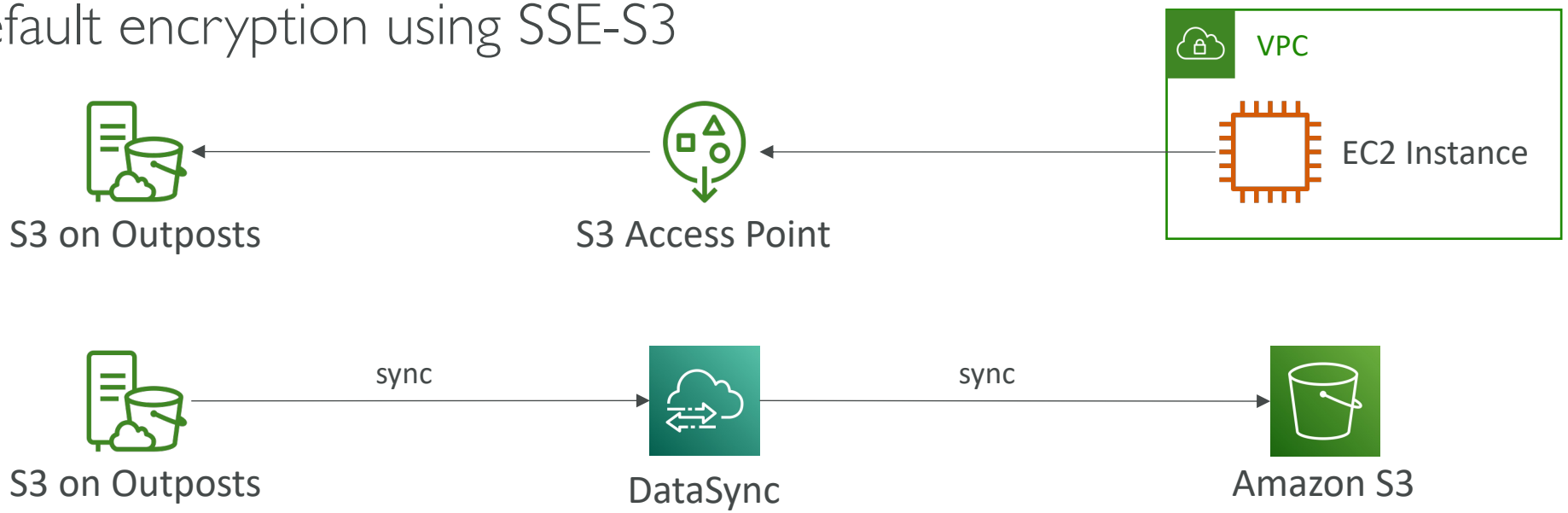

Amazon EMR

© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

S3 on AWS Outposts

- Use S3 APIs to store and retrieve data *locally* on AWS Outposts
- Keeping data close to on-premises applications
- Reduce data transfers to AWS Regions
- S3 Storage Class named **S3 Outposts**
- Default encryption using SSE-S3

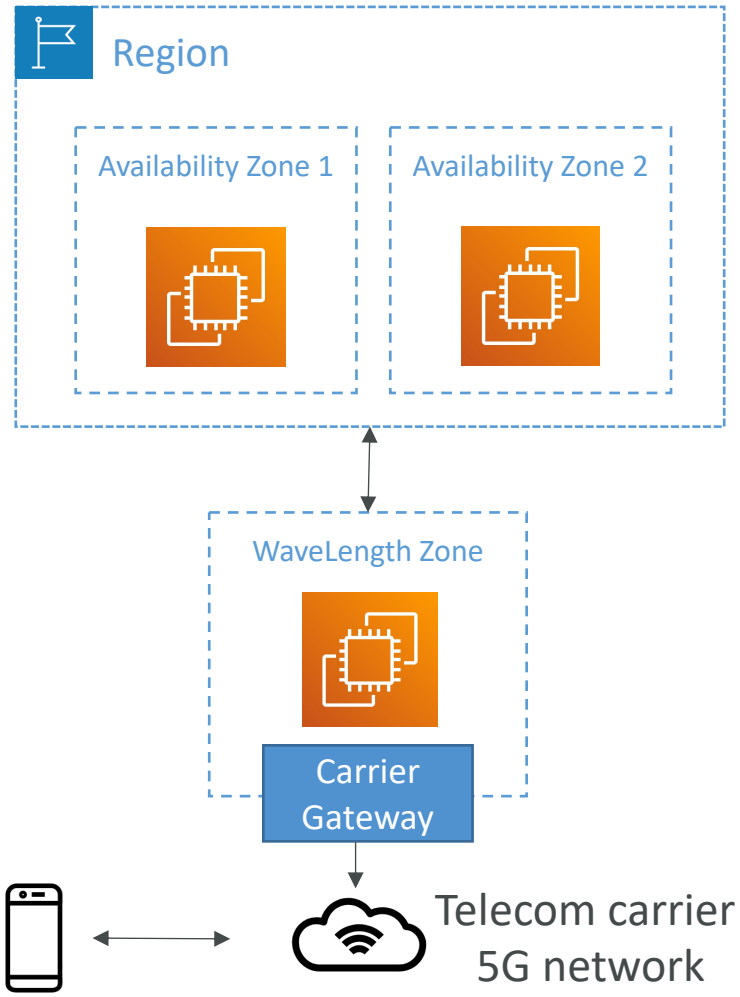


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AWS WaveLength

- **WaveLength Zones** are infrastructure deployments embedded within the telecommunications providers' datacenters at the edge of the 5G networks
- Brings AWS services to the edge of the 5G networks
- Example: EC2, EBS, VPC…
- Ultra-low latency applications through 5G networks
- Traffic doesn't leave the Communication Service Provider's (CSP) network
- High-bandwidth and secure connection to the parent AWS Region
- No additional charges or service agreements
- Use cases: Smart Cities, ML-assisted diagnostics, Connected Vehicles, Interactive Live Video Streams, AR/VR, Real-time Gaming, …

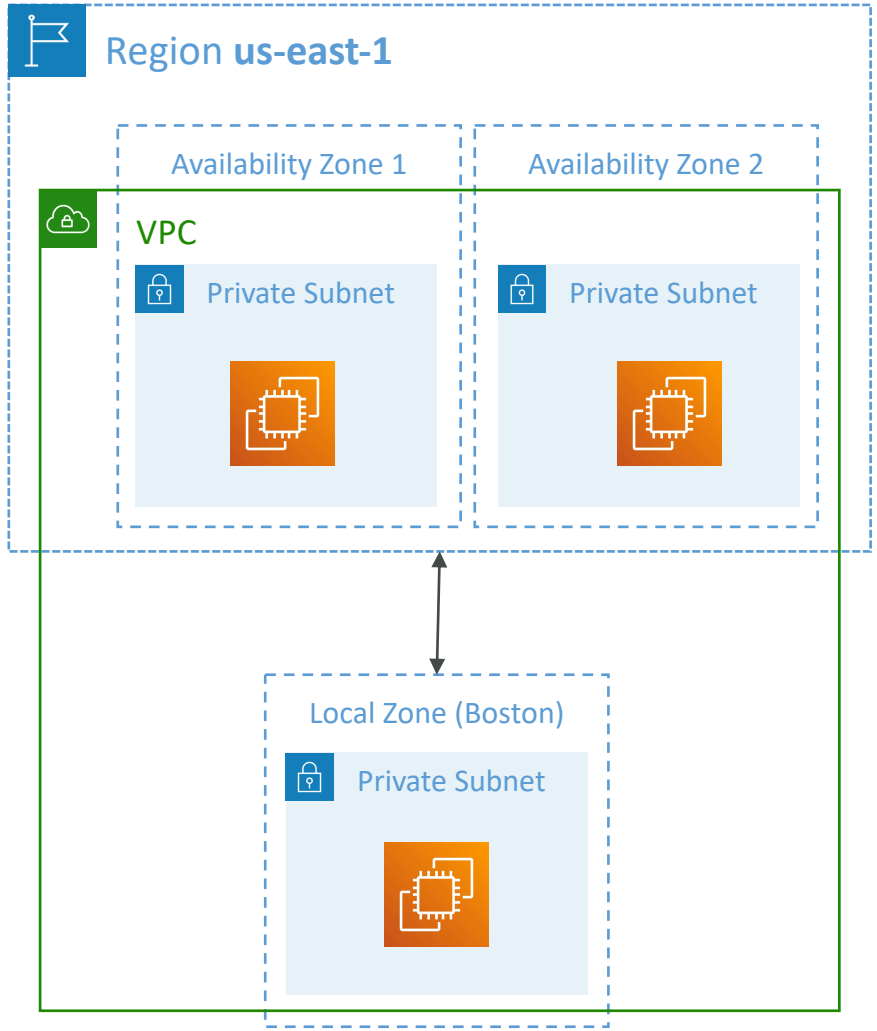


© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com

AWS Local Zones

- Places AWS compute, storage, database, and other selected AWS services **closer to end users to run latency-sensitive applications**
- Extend your VPC to more locations – "**Extension of an AWS Region**"
- Compatible with EC2, RDS, ECS, EBS, ElastiCache, Direct Connect …
- Example:
  - **AWS Region:** N. Virginia (us-east-1)
  - **AWS Local Zones:** Boston, Chicago, Dallas, Houston, Miami, …



© Stephane Maarek

NOT FOR DISTRIBUTION © Stephane Maarek www.datacumulus.com